Parsek — Game Actions & Resources System Design

Comprehensive design specification for Parsek's game resource tracking system — covering science, funds, reputation, milestones, contracts, kerbals, facilities, and strategies across the rewind timeline.

Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to a timeline, rewind to earlier points, and see previously recorded missions play back as ghost vessels alongside new ones. This document specifies how the game's resource state (everything except vessel trajectories) is tracked, reconciled, and kept consistent across rewinds.

Version: current to Parsek 0.10.5. The ledger engine (Phases 1-8) was completed in 0.6; later releases added the items listed under Status, the stock-screen reservation layer (section 14) and the pre-Parsek save backup (section 4.6). Section 3.15 (event identity, ruled 2026-10-26) is built through build order step 6 and step 8; step 7 is planned. **Status:** All phases implemented and tested (4700+ tests). Full ledger-based recalculation engine with 8 resource modules (Science, Funds, Reputation, Milestones, Contracts, Facilities, Strategies, Kerbals; the Routes module was removed in 0.10.6 when route events left the ledger). KSP state patching for all resource types including contract restoration from ConfigNode snapshots and milestone reversal via reflection. Kerbal lifecycle management with reservation chains, stand-in generation, and retirement is fully integrated into the engine walk via IResourceModule (T42 complete): KerbalsModule is a registered first-tier module that consumes KerbalAssignment in ProcessAction and builds the replacement chains + retired set in PostWalk. Shipped since the earlier draft: contract deadline-expiry penalties (ContractsModule injects a synthetic ContractFail with funds + reputation penalties at the deadline UT), contract-completion science crediting (ScienceModule credits TransformedScienceReward), the second-tier RouteModule (logistics/supply routes, milestones M1-M6), rescue detection, warp facility-visual patching, and MIA respawn override. Verification tooling is shipped: the ledger ground-truth harness (CareerSaveParser / LedgerGroundTruthDiff / in-game LedgerGroundTruthHarness) diffs recalc output against the on-disk save, and the event-driven LedgerTrace tracer instruments the apply boundary. The re-fly work binds ledger state to recording lifecycle via EffectiveState ERS/ELS routing, RecordingSupersedeRelation rows, and recording-scoped LedgerTombstones. **Out of scope:** Vessel recording system, DAG structure, ghost rendering, distance zones. See `parsek-flight-recorder-design.md` (recording, recording trees / DAG, playback, distance zones) and `parsek-ghost-trajectory-rendering-design.md` (ghost rendering) for those.

---

1. Introduction

This document specifies how Parsek tracks everything on the timeline that is not a vessel trajectory: science experiments, funds, reputation, milestones, contracts, kerbals, KSC facilities, and strategies. It covers:

- What happens to your science, funds, and reputation when you commit a recording and when you rewind
- How the system prevents time-travel paradoxes (duplicate kerbals, overspending, double science credit)
- The reservation system that locks resources committed to future recordings
- Kerbal identity management across rewinds (replacement chains, stand-ins, retirement)
- KSC spending (tech nodes, facility upgrades, kerbal hiring) and how it interacts with the timeline
- Contract and strategy slot management across rewinds
- How KSP's internal state is patched to stay consistent with the timeline

1.1 What happens when you fly a mission

During flight, Parsek silently captures every resource-relevant event: science experiments transmitted or recovered, milestone achievements, contract completions, and kerbal assignments. These events are held in memory — they don't affect the timeline yet.

When you **commit** the recording, all captured events are added to the timeline's ledger. The system recalculates everything from the beginning of time: science subject caps are applied, fund balances are recomputed, milestone credit is assigned to whichever recording achieved it first, and KSP's internal state (R&D science, fund balance, reputation, facility levels, crew roster) is patched to match.

1.2 What happens when you rewind

Rewinding loads an earlier save state, but all committed recordings survive. The system recalculates the full timeline up to the rewind point. You may see different numbers than before — if you committed a new recording to the past, it gets chronological priority for science subjects and milestones, reducing credit for later recordings. Fund balances shift accordingly.

Resources committed to future recordings are **reserved**. If you rewound to before a vessel launch, that future launch still constrains the spendable funds shown now; you cannot spend money that the projected future cashflow still needs. Kerbals assigned to future missions are reserved and replaced by temporary stand-ins. Contract and strategy slots committed in the future are held from the start.

1.3 What the player sees

Situation	What happens
Commit a recording	Science, funds, milestones, kerbal XP from that mission join the timeline. KSP state updates immediately.
Rewind to earlier point	All committed data survives. Balances recalculated. Reserved resources reduce what you can spend. Stand-in kerbals fill reserved crew slots.
Two recordings collect the same science	Chronologically first recording gets full credit. Later recording's effective science is reduced (subject cap). Total never exceeds the cap.
Kerbal used in a committed recording	Reserved from the start of time until recovery. A free stand-in fills their roster slot. Original returns when their recording resolves.
Tech node unlocked at KSC	Science cost is reserved against the future science cashflow. Can't unlock if the projected science balance would dip negative.
Facility upgraded at KSC	Fund cost reserved. Building visuals update during fast-forward.
Contract accepted	Active from its accept UT. After a rewind, Mission Control refuses an accept that would leave a later committed accept without a slot (section 10.2).
KSP load (hard reset)	Everything resets to the loaded save. Ledger is pruned to match. This is the only destructive operation.

1.4 The three layers of paradox prevention

Parsek prevents time-travel paradoxes through three complementary layers, all working by prevention rather than detection:

Layer 1: No-delete invariant. Recordings can only be added to the timeline, never removed. Total earnable resources never decrease. Existing spendings that were valid when committed cannot become invalid. Re-Fly is the one operation that rewrites past events; it keeps this layer by only ever retiring losses (`parsek-rewind-to-separation-design.md` section 2.12).

Layer 2: Spending reservation. Committed spendings (past, present, and future) are reserved against the resource cashflow. New spendings (vessel builds, tech nodes, facility upgrades, kerbal hires) are blocked if they would create a deficit at any point on the timeline. Applies to science and funds. The owner rulings of 2026-10-21 extend it to reputation and to every open contract's fail penalties, and stop a loss nobody could reserve at the reserved floor, with the rest carried as a debt (section 3.13, built in 0.10.6).

Layer 3: UT=0 reservation and stock-control blocks. Kerbal identity is locked from the start of time. Kerbals used in any recording are reserved as a continuous block - no gaps, no reuse until all recordings resolve. Contract and strategy slots are not reserved from UT=0: Mission Control and the Administration building refuse an accept or activation that would leave a committed one without a slot (sections 10.2, 13.3, 14). This prevents duplicate kerbals, slot overflow, and resource diversion conflicts.

The player never sees a broken state. If something goes truly wrong, KSP load (the hard reset) is always available.

1.5 Resource modules at a glance

Game mode	Module	What it tracks	Earnings	Spendings
All modes	Kerbals	Crew identity, assignments, XP, death/rescue	Assignment, XP gain, death/MIA	Hiring cost (career only)
Science+	Science	Per-subject experiment credit, tech tree	Experiments (transmit/recover)	Tech node unlocks
Career	Funds	Monetary balance	Contracts, recovery, milestones	Vessel builds, facility upgrades, hires
Career	Reputation	Global score affecting contracts	Contracts, milestones	Contract failures, strategies

Game mode	Module	What it tracks	Earnings	Spendings
Career	Milestones	World Firsts (once-ever per save)	Achievement triggers (fund + rep awards)	—
Career	Contracts	Accept/complete/fail/cancel lifecycle	Advance payments, completion rewards	Penalties on fail/cancel
Career	Facilities	KSC building levels and destruction state	—	Upgrade cost, repair cost
Career	Strategies	Resource conversion policies	Ongoing contract reward transforms	Setup cost

"All modes" means SANDBOX, SCIENCE_SANDBOX and CAREER. In MISSION, MISSION_BUILDER, SCENARIO and SCENARIO_NON_RESUMABLE games Parsek is inert (owner ruling S9, 2026-09-26; `ParsekGameModeGate`): no ledger capture, no state patching, no stock-control blocks, and a `ParsekScenario` node such a save carries is written back verbatim.

1.6 Example: a career Mun landing

A complete Mun landing and return in Career mode, showing every game action the ledger tracks:

```
UT=0 (KSC, frozen time)
  CONTRACT    Accept "Explore the Mun"                    +0 funds advance
  CONTRACT    Accept "Collect science from Mun surface"   +2,000 funds advance
  FUNDS       Vessel build "Mun Lander I"                 -18,000 funds
  KERBAL      Assign Jeb to command seat
  KERBAL      Assign Bill to crew cabin

UT=100 (launch)
  MILESTONE   First launch (if first ever)                +2,000 funds, +5 rep

UT=300 (Kerbin orbit)
  SCIENCE     crewReport@KerbinInSpaceLow                 earned 5
  MILESTONE   First orbit of Kerbin (if first)            +5,000 funds, +10 rep

UT=2000 (Mun SOI)
  SCIENCE     evaReport@MunInSpaceHigh                    earned 8
  MILESTONE   First flyby of the Mun                      +5,000 funds, +10 rep

UT=5000 (Mun orbit)
  SCIENCE     crewReport@MunInSpaceLow                    earned 8
  MILESTONE   First orbit of the Mun                      +7,000 funds, +12 rep

UT=8000 (Mun surface, Midlands)
  SCIENCE     surfaceSample@MunSrfLandedMidlands          earned 30
  SCIENCE     crewReport@MunSrfLandedMidlands             earned 10
  SCIENCE     evaReport@MunSrfLandedMidlands              earned 8
  MILESTONE   First landing on the Mun                    +10,000 funds, +15 rep
  MILESTONE   First EVA on the Mun                        +5,000 funds, +8 rep
  CONTRACT    "Explore the Mun" completed                 +40,000 funds, +15 rep
  CONTRACT    "Collect Mun surface science" completed     +12,000 funds, +8 rep

UT=15000 (Kerbin recovery)
  SCIENCE     (onboard experiments returned at full value)
  FUNDS       Vessel recovery (% of cost by distance)     +12,000 funds
  MILESTONE   First Mun return (if tracked)               +8,000 funds, +10 rep
  KERBAL      Jeb gains XP (Mun landing + return)
  KERBAL      Bill gains XP (Mun landing + return)

UT=15000 (KSC, frozen time — post-recovery)
  SCIENCE     Tech node "landing" seq=0                   -45 science
  SCIENCE     Tech node "survivability" seq=1             -45 science
  FACILITY    Upgrade tracking station lvl 2              -150,000 funds
  KERBAL      Hire Valentina                              -25,000 funds
```

If you then rewind to UT=0 and fly a different mission, every action above participates in the recalculation — science subject caps, milestone once-ever flags, fund balance checks, kerbal reservations — all reconciled against whatever the new recording contributes.

2. Design Philosophy

These principles govern every design decision in the game actions system. They are listed here because they inform every section that follows.

2.1 Timeline Model

1. **Commit is permanent.** Once a recording is committed, its resource data is part of the timeline. The timeline is additive — recordings can be added but never deleted through Parsek. Like a Git commit.

2. **Rewind is non-destructive.** All committed data survives a rewind. Derived values (effective science, running balances, kerbal status) are recomputed from scratch. Like `git checkout` — you move the pointer, not the history.

3. **KSP load is the hard reset.** Loading a save discards everything not in that save. The ledger is pruned to match. This is the only destructive operation, and it's KSP's native behavior — Parsek accepts it rather than fighting it. Like `git reset --hard`.

4. **Immutable recordings, derived state.** Raw values from KSP (science awarded, fund amounts, milestone triggers) are written once and never changed. All computed values (effective science, running balances, affordability, kerbal status) are recomputed from scratch on every timeline change.

2.2 Paradox Prevention

5. **Prevention, not detection.** Every constraint works by making paradoxes structurally impossible, not by detecting them after the fact.

6. **Spending reservation locks resources globally.** Committed spendings (past and future) are reserved against projected cashflow. The player can only add new spendings if the projected minimum balance covers them. Open contracts' fail penalties are reserved too, a committed outcome cannot be moved earlier, and a loss that cannot be reserved stops at the reserved floor (section 3.13, owner rulings 2026-10-21).

7. **Identity reservation locks from UT=0.** Kerbals are reserved from the start of time as continuous blocks. No gaps, no reuse until resolved. Contract and strategy slots are checked against the committed timeline at the stock control instead (sections 10.2, 13.3).

8. **Conservative over precise.** Where Parsek's model diverges from KSP's exact mechanics (e.g., hard cap vs diminishing returns curve for science), it errs on the side that prevents overcredit.

2.3 Player Experience

9. **Commitment is the deliberate choice.** Recordings only affect the timeline when the player explicitly commits. The player can always revert to launch and discard. The harshness of permanent consequences (kerbal death, resource commitment) is mitigated by the player's control over what gets committed.

10. **The player never sees a broken state.** No manual resolution needed, no unresolvable deficits, no duplicate kerbals. If something goes truly wrong, KSP load is the escape hatch.

11. **Available balance shows what you can spend.** KSP's fund and science displays are patched to show the spendable balance after cashflow-aware reservations, not the gross running balance. No surprises at launch time.

---

3. Architecture Overview

The game actions system is a **standalone module** that tracks every resource-related event on the timeline. It is completely separate from the vessel recording system. It does not know about trajectories, DAGs, ghosts, or flight recordings. The vessel recording system does not know about science, funds, or tech nodes.

3.1 Coupling between systems

The original coupling between the two systems was a single field: `recordingId` on earning actions. When a recording is committed, its associated earning actions join the timeline. The recording system doesn't know about science values. The ledger doesn't know about trajectories. They share a key.

The re-fly work broadened this coupling beyond that one field. Ledger reads of committed recordings and ledger actions now route through `EffectiveState.ComputeERS` / `ComputeELS` (effective-recording-set / effective-ledger-set) rather than the raw stores, so ledger-derived state follows recording visibility. Re-fly also writes `RecordingSupersedeRelation` rows keyed by recording id and builds `LedgerTombstone`s scoped to superseded recording subtrees, so a recording's lifecycle (supersede, subtree closure) directly shapes which ledger actions are effective. See `SupersedeCommit.cs` and `LedgerTombstone.cs`.

3.2 API between systems

Recording system → Ledger:

- `OnRecordingCommitted(recordingId, startUT, endUT)`

There is no deletion event. Parsek doesn't delete recordings. KSP load handles removal natively by not including the recording in the loaded save.

Ledger → Recording system:

- `GetAvailableResources(ut)` → current balances at a point in time
- `GetRecordingDeltas(recordingId)` → list of game actions for UI display

Implementation (v0.4): The actual API is `LedgerOrchestrator.OnRecordingCommitted(recordingId, startUT, endUT)` for the commit trigger, plus `NotifyLedgerTreeCommitted(RecordingTree)` for batch commits across a recording tree. `GetAvailableResources` and `GetRecordingDeltas` are not yet implemented as named methods — module query methods (`FundsModule.GetAvailableFunds()`, `ScienceModule.GetAvailableScience()`, etc.) provide this data. Additional API: `OnSave()` / `OnLoad()`, `OnKspLoad(HashSet<string>, double)`, `OnKscSpending(GameStateEvent)`, `CanAffordScienceSpending(float)`, `CanAffordFundsSpending(float)`, `HasFacilityActionsInRange(double, double)`.

3.3 Three kinds of game actions

Earning actions are associated with a recording ID. Science collected during a mission, funds from vessel recovery, reputation from milestones, kerbal assignments. Tagged with the recording that produced them. Removed from the timeline only if the recording is removed (which only happens via KSP load/hard reset — Parsek has no delete operation).

Spending actions are not associated with any recording. Tech node purchases, facility upgrades, kerbal hires. These happen at KSC between missions. They persist on the timeline. Removed only by loading a save from before their UT.

System-generated actions are created by Parsek itself in response to earning or spending actions. Kerbal stand-in generation is the primary example — when a kerbal is temporarily reserved by a committed recording, Parsek generates a replacement kerbal to maintain roster stability. These are not player actions and have no UT on the timeline, but they do modify KSP's game state (roster entries).

3.4 Timeline operations

Operation	Effect on game actions
COMMIT	Add recording's earning actions to timeline, trigger recalculation
HIDE	Stop ghost playback; game actions remain on timeline
UNHIDE	Restore ghost playback; no change to game actions
REWIND	Non-destructive jump to a recording's launch point; all actions survive, trigger recalculation
LOAD	Destructive reset to save state (KSP native); Parsek rebuilds from save contents

There is no delete button. No deficit system is needed. Load is the only destructive path.

3.5 The no-delete invariant

Because recordings can only be added to the timeline (never removed by the player through Parsek), the total earnable resources can only stay the same or increase over time. A retroactive commit can redistribute credit between recordings but cannot reduce the total credited amount for any capped resource. This means **existing** spending actions that were valid when created cannot be retroactively invalidated by normal play.

However, the invariant does not prevent the player from adding **new** spendings after a rewind that would exceed the available budget. Each new recording adds a vessel build cost (funds), and the player can unlock new tech nodes (science) or hire kerbals

(funds) at KSC. Without further protection, these new spendings could create a deficit downstream.

The **reservation system** (sections 6.5 and 7.6) covers this gap: committed spendings — past, present, and future — are reserved against projected cashflow. The player can only add new spendings if the projected minimum balance is sufficient. Together, the no-delete invariant protects existing spendings, and the reservation system prevents new overspending.

STASH / Re-Fly interaction: Re-Fly supersede keeps old ledger rows physically append-only but may tombstone reviewed, recording-scoped career actions from the superseded subtree so they leave ELS. Superseded recording events remain in `Ledger.Actions` for audit/recovery, while recalculation consumes only the post-tombstone ELS and patches stock KSP state for removed science, contracts, facilities, and safe roster-created kerbals. Null-scoped KSC/system rows, initial seeds, already-paid rollout costs, and unknown future action types remain preserved until reviewed.

Tombstoning a credit is exactly the retroactive invalidation this section rules out: the credit may already have paid for a later committed spend, which the recalculation then charges into a negative balance (funds) or refuses (a tech node never unlocked). From 2026-05-06 (the broad tombstone scope) until the 2026-10-19 rulings the Re-Fly path did this. The rulings (`parsek-rewind-to-separation-design.md` section 2.12, designed, not yet implemented) restore this section's invariant for Re-Fly: a slot can be re-flown only when every row its merge would retire is a loss (crew deaths and bookkeeping, crew-death and vessel-loss reputation, building damage); any gain, contract outcome or flight-tagged spend seals the slot. A merge therefore only ever raises a balance, and a dominance check before the merge refuses any that would not. Other operations that still retire committed credit (a quickload trim that keeps later KSC spends, revert-to-launch pruning, a recording that fails to load, route credits after a rewind) are tracked as todo `LEDGER-RETIREMENT-PATHS-OUTSIDE-REFLY` ; the owner ruled every one of them on 2026-10-21 (section 3.13).

3.6 Recalculation triggers

#	Trigger
1	Recording committed
2	Rewind (load Parsek quicksave)
3	Fast-forward crosses a game action
4	Save loaded (KSP load — Parsek rebuilds from save contents)

3.7 Sort order for the unified walk

All game actions across all resource modules are collected and sorted for the recalculation walk. The sort key is:

Priority	Field	Notes
Primary	UT ascending	When the action occurred on the timeline
Secondary	Type: earnings before spendings	At the same UT, credits before debits
Tertiary	Capture sequence `cseq`	Real capture order at the same UT (e.g. a Rebuild then an Upgrade, or several tech unlocks, at one frozen KSC UT)
Fallback	Sequence index	Order between rows with no `cseq` , or with the same one

Built (section 3.15 step 2): `cseq` is the tie key, so KSC rows written at one frozen UT keep their real click order. A row with no `cseq` (0 from an older save, or -1 when converted from an event an older build captured) sorts before every sequenced row at its UT, because it was captured before this build stamped anything; this also keeps an older save's same-UT order unchanged. Two such rows fall back to the sequence index. The sort is stable, so rows equal on every key keep their input order.

`RecalculationEngine.CompareWalkOrder` is the one comparer. Every reader that must agree with the walk on which of two rows comes first reads it: the committed-future index (first / last committed row of a key, so the click gates and marks pick what the walk picks), the facility destruction fold, the KSC reputation-curve reconcile, the Re-Fly dominance check and the Timeline. The cashflow projection and the reserved floor read the walk's own sorted list.

3.8 Recalculation model

On any timeline change (commit, rewind, fast-forward event, KSP load), every resource module recalculates its derived state from scratch by walking all actions from UT=0 forward. No module retains cached state between recalculations — the ledger file is the sole input, the derived state is the sole output.

The general pattern is:

```
RecalculateAndPatch():
  0. Seed initial balances (once per session):
     - SeedInitialFunds from Funding.Instance.Funds
     - SeedInitialScience from ResearchAndDevelopment.Instance.Science
     - SeedInitialReputation from Reputation.Instance.reputation
     Capture the pre-ledger progress seed (once per save, once ProgressTracking
     has loaded; see 4.5).
     Update slot limits from current facility levels.
  1. Reset all derived state to zero/default.
  2. Collect all game actions from the ledger file.
  3. Sort by (UT, type, sequence).
  4. Walk forward from UT=0:
     For each action:
       Dispatch to the appropriate module.
       Module updates its derived state (running balances, caps, reservations, visual state).
  5. Derived state at the end of the walk = current state.
  6. Patch KSP state to match derived state.
  7. PostWalk: modules finalize derived state (e.g. KerbalsModule builds chains/retired set).
```

Each module owns its portion of the walk:

- **Science**: tracks per-subject credited totals and running science balance. Applies subject caps.
- **Funds**: tracks running funds balance. Validates affordability.
- **Reputation**: tracks running reputation balance.
- **Milestones**: tracks once-ever flags. Chronologically first recording gets credit.
- **Kerbals**: computes reservations, chain occupancy, XP totals.
- **Facilities**: derives building level and visual state (upgraded/destroyed/repaired).

Dependency ordering. Some modules produce values that other modules consume. The recalculation respects a priority order:

Tier	Modules	Role
First (independent)	Science, Milestones, Contracts	Determine what was earned, achieved, or assigned. Compute effective values.
Transform	Strategies	Apply active strategy transforms to effective contract rewards before second-tier crediting.
Second (dependent)	Funds, Reputation	Aggregate effective (and transformed) earnings and spendings from first-tier modules.
Parallel	Facilities	Derives building state and feeds repair/upgrade costs into Funds.
First tier	Kerbals	Reservation/chains from KerbalAssignment actions. PrePass reads RecordingStore for mutable recording metadata. PostWalk builds replacement chains and retired set. ApplyToRoster called by orchestrator after engine walk.

First-tier modules run their walk and produce effective values. Strategy transforms then modify contract rewards within their active windows. Once transforms are applied, the milestone and contract fund/reputation awards flow into the Funds and Reputation running balances in the second tier. This means a single recalculation pass walks all actions from UT=0 forward, but within each UT, first-tier modules resolve, then strategy transforms apply, then second-tier modules consume the results.

Modules remain isolated in logic — they don't call each other. The dependency is purely data flow: first-tier modules write effective values onto actions, second-tier modules read those values during their portion of the same walk.

Events on the timeline are locked once committed — their immutable values never change. But the **derived** values are recomputed on every recalculation. Adding a new recording to the past can change derived values on future events (e.g. a retroactive science collection reduces a later collection's effective credit). The events themselves are fixed; only the interpretation changes.

3.8.1 Implementation Notes (Post-Build)

Derived field reset: Before each walk, the engine resets all derived fields on every action to defaults (`Effective=true`, `EffectiveScience=0`, `Affordable=false`, `EffectiveRep=0`, `TransformedFundsReward=FundsReward`, etc.). This prevents stale values

from a previous recalculation from leaking into the next walk. Critical for idempotency — calling Recalculate twice on the same actions must produce identical results.

PrePass phase: Between Reset and the walk, the engine calls `PrePass(actions)` on every module. `ScienceModule` and `FundsModule` use this to compute committed spendings for the current walk. In a full-ledger walk that is the whole timeline; in a UT-cutoff walk it is only the visible/current action scope. Cutoff availability is then installed from a separate full-timeline cashflow projection. Other modules (`ContractsModule`, `MilestonesModule`, `FacilitiesModule`, `ReputationModule`, `StrategiesModule`) have no-op PrePass implementations.

Strategy transform mechanism: Strategy transforms modify `TransformedFundsReward`, `TransformedScienceReward`, and `TransformedRepReward` derived fields — NOT the persisted `FundsReward` / `ScienceReward` / `RepReward`. Second-tier modules read the Transformed fields. This prevents data corruption if the ledger is saved between recalculations.

Orchestration: `LedgerOrchestrator` (not in the original design) coordinates the full pipeline: creates module instances, registers them with the engine, converts `GameStateEvent` data to `GameAction` entries at commit time, and invokes the recalculate-then-patch sequence on all four triggers.

Event conversion bridge: `GameStateEventConverter` maps `GameStateEvent` entries from `GameStateStore` to `GameAction` entries for the ledger. This replaces the design doc's "extract from sidecar" flow — game actions are captured by `GameStateRecorder` into `GameStateStore` during flight, then converted to `GameAction` entries at commit time. No sidecar involvement for game actions.

3.9 Spending reservation pattern

Any resource that has both earnings and spendings requires a **spending reservation system** to prevent the player from adding new spendings that would create a deficit at any point on the timeline.

When the player rewinds to an earlier UT, the visible game state is recomputed only through that UT, while committed future spendings still constrain what can be spent now. The player can only add a new spending if the projected resource balance would stay non-negative from the current UT through the future committed ledger.

```
currentBalance(ut) = balance after replaying actions with action.UT <= ut
projectedBalance starts at currentBalance(ut)
for each future earning/spending in chronological order:
  projectedBalance += futureDelta

available(ut) = max(0, min(currentBalance(ut), every future projectedBalance))
```

This applies to:

- **Science**: available science is the minimum projected science balance from the current UT through future science earnings and spendings. Future science earnings cannot make more science spendable now than the current balance, but they can cover later committed tech unlocks before those unlocks happen.
- **Funds**: available funds are the minimum projected fund balance from the current UT through future fund earnings and spendings (vessel builds, facility upgrades, hires, etc.). Future earnings cannot inflate current spendability above the current balance, but they can cover later committed fund spendings before those spendings happen.

Reputation gets a reservation too (owner rulings 2026-10-21, section 3.13, built in 0.10.6). Stock holds nothing back in reputation: it can go negative without blocking any player action, and the only reputation spending that requires a minimum balance is strategy activation, which stock gates on the live reputation itself (`Strategy.CanBeActivated`). Parsek reserves the reputation that committed costs still need and, while such a cost lies ahead, every open contract's reputation penalty: a strategy setup or a Mission Control click that would dig into that reservation is greyed with the reason and refused, and any other reputation loss stops at the reserved floor, the rest carried as a debt. The pool itself still shows the real value (section 3.13, part 2).

The reservation pattern is analogous to the kerbal reservation system — future committed events lock resources retroactively.

The no-delete invariant protects existing spendings. Adding a recording can only add new earnings (and for funds, a vessel cost). It cannot remove existing earnings. For science, subject caps mean the total credited science is stable — redistribution happens, but the total doesn't decrease. So existing spendings that were valid when committed remain valid.

The reservation system protects against new spendings. Even though existing spendings stay valid, the player could attempt to add new spendings (tech nodes, facility upgrades) at a UT where the running balance cannot support them alongside existing future cashflow. The reservation blocks this.

Together, the two mechanisms keep new player spendings from making science or funds negative anywhere on the committed timeline. Section 3.13 extends the protection to reputation and to losses a new flight adds after a rewind (built in 0.10.6); with no

committed cost ahead reputation can still go negative, as in stock.

3.10 Three layers of paradox prevention

Parsek prevents time-travel paradoxes through three complementary design layers:

Layer 1: No-delete invariant. Recordings can only be added to the timeline, never removed. This guarantees that total earnable resources never decrease. Existing spendings that were valid when committed cannot become invalid — there is no scenario in normal play where an earlier-valid spending becomes unaffordable due to a new recording being added. Re-Fly, the only operation that rewrites past events, keeps the layer by retiring losses only and refusing any merge that would lower a balance (section 3.5, `parsek-rewind-to-separation-design.md` section 2.12).

Layer 2: Spending reservation. Committed spendings (past, present, and future) are reserved against projected cashflow. The player can only add new spendings (vessel builds, tech nodes, facility upgrades, kerbal hires) if the projected balance never dips negative. This prevents the player from creating new deficits after rewinding to an earlier UT. Applies to science and funds; the 2026-10-21 rulings extend it to reputation and to open contracts' fail penalties, and add the reserved floor with a debt for losses nobody can reserve (section 3.13).

Layer 3: UT=0 reservation and stock-control blocks. Kerbal identity is locked from the start of time when committed anywhere on the timeline. Kerbals used in a recording are reserved from UT=0 as a continuous block - no gaps, no reuse until all recordings resolve. Contract slots are NOT reserved from UT=0: Mission Control refuses an accept that would leave a committed future accept without a slot (section 10.2). Strategies are NOT reserved from UT=0 either: the Administration building refuses an activation the committed timeline makes later, an activation that leaves no slot for a committed activation, and a player deactivation while a committed row for that strategy is still ahead (section 13.3). This prevents duplicate kerbals, slot overflow, and resource diversion conflicts that could cascade into downstream paradoxes.

Core philosophy: conservative by design. Every restriction exists to prevent a specific paradox. The tradeoff is reduced gameplay flexibility — kerbals can't be reused freely across rewinds, a strategy can't be activated earlier or cancelled before its committed timeline changes it, available funds may show zero at early UTs because the budget is fully committed to future events. But the player never sees a broken timeline, never encounters an unresolvable deficit, and never needs to manually fix inconsistencies. If a situation becomes truly stuck, KSP load (the hard reset) is always available as the escape hatch.

3.11 Resource modules

Each resource type is an independent module with its own earning/spending rules. All modules feed into the same unified timeline walk, which dispatches each action to its module. The modules are, ordered by game mode and increasing complexity:

Game mode	Module	Earnings	Spendings
Sandbox	Kerbals	Assignment, XP gain, death/MIA	— (free creation)
Science	Science	Experiments (transmit/recover)	Tech node unlocks
Science	Kerbals	Assignment, XP gain, death/MIA	— (no funds)
Career	Funds	Contracts, recovery, milestones	Vessel builds, facility upgrades
Career	Reputation	Contracts, milestones	Contract failures, strategies
Career	Milestones	World Firsts triggers (award funds + rep)	—
Career	Contracts	Accept/complete/fail/cancel (award/penalize funds + rep)	—
Career	Kerbals	Hiring, assignment, XP gain, death/MIA	Hiring cost (funds)
Career	Facilities	—	Upgrade cost (funds), gates capabilities
Career	Strategies	Ongoing resource conversion rates	—

Supply routes are not a ledger module (0.10.6, todo LOGISTICS-WITHOUT-MONEY). Routes carry no money (phase 1: no route funds gate, dispatch charge or recovery credit; a Space Center route dispatches free in every mode), and their events are not ledger rows (phase 2): every dispatch, debit, pickup, delivery, pause, resume, hold and endpoint-lost event goes to the route-owned `RouteEventLog`, persisted as `ROUTE_EVENT_LOG` with the route store in the PARSEK scenario node, and the former `RouteModule` is deleted. The ledger therefore holds committed history only. `GameActionType` ids 23 to 30 and 36 stay reserved; a row of those types in an old save's ledger is dropped at ledger load (`LedgerLoadMigration.IsLegacyRouteNode`) with one Info line naming the count per type, so the next save holds none. The route system (logistics milestones M1-M6) has its own design spec: see `docs/parsek-logistics-supply-routes-design.md` (section 10.6 for the real-time rule).

3.12 Module documentation template

Each resource module chapter should follow this structure when fully designed:

1. **Status** and **Game mode applicability** — which modes the module is active in and how behavior varies.
2. **KSP mechanics** — how the vanilla game handles this resource (the ground truth Parsek must model).
3. **Earning action schema** — fields stored in the ledger for incoming resources.
4. **Spending action schema** — fields stored for outgoing resources (if applicable).
5. **Recalculation logic** — how the unified walk processes this module's actions, including caps, constraints, and priority rules.
6. **Verified scenarios** — worked examples showing correct behavior under normal play and edge cases (retroactive commits, rewinds, paradox prevention).
7. **Edge cases** — known tricky situations and how the system handles them.
8. **Open questions** — unresolved design decisions for future work.

3.13 The committed-value rule (owner rulings 2026-10-21)

Status: ruled 2026-10-21 (ledger rulings interview, todo `LEDGER-RETIREMENT-PATHS-OUTSIDE-REFLY`), open-penalty scope ruled 2026-10-22; built in 0.10.6 (all four parts, the stock-money paragraph and the revert and failed-load rules below). Implementation plan: `docs/dev/done/plans/committed-value-rule-implementation.md`. This section is the authority for the rule; the module sections point here.

The goal: no committed purchase is ever left unpaid, and no pool (funds, science, reputation) drops below what committed costs still need. Layer 1 (section 3.10) protects committed spends against operations that remove rows; this rule also protects them against losses a new flight adds after a rewind, and extends Layer 2 to reputation. It has four parts, applied to funds, science and reputation alike:

1. **Committed outcomes do not move earlier.** An outcome the committed timeline already holds cannot be changed before its UT. A new flight cannot fail a contract whose completion, failure or cancellation is committed: stock's fail is refused and logged. Completing it earlier stays allowed, because that only moves the reward earlier (the same direction as Re-Fly Rule 4, rewind design section 2.12). This extends ruling D7 (Cancel greyed for such a contract, section 14.4) from the button to in-flight fails. Built in 0.10.6: `Patches/ContractEarlyFailPatch.cs` prefixes stock's single fail funnel, `Contract.SetState(State.Failed)` (reached by `Fail()` and by a failed non-optional parameter), and stock's `OfferExpired` end of an Active contract whose requirements no longer hold, and stock's deadline expiry (`SetState(State.DeadlineExpired)` from `Update()`), and refuses them while the committed timeline holds a complete, fail or cancel of that contract at any UT (`StockUiReservationPredicates.CommittedContractResolution`, the any-UT form of the Cancel block's predicate). A committed outcome the clock already passed counts too: the flight scene has no clock cursor, so stock still shows the contract Active until the next recalculation applies the outcome, and a fail in that window would charge a penalty for a contract the timeline already ended (todo CONTRACT-FAIL-AFTER-REACHED-COMMITTED-OUTCOME). The contract stays Active, frozen where stock left it, until the next recalculation past the committed UT applies the committed outcome. A contract with no committed outcome expires as stock decides (the committed timeline holds the same expiry). A reached outcome holds only the lifecycle it ended: it counts from the live contract's own accept date, so a contract accepted again later is not held by its earlier outcome. The other side of the same window: after a rewind to before a committed accept, the save shows the contract on offer again; once the clock has passed both the committed accept and the committed outcome, the next career update takes that offer off the board (`KspStatePatcher.PatchContracts`, todo STALE-OFFER-RE-ACCEPTABLE-AFTER-FLIGHT-PASSES-COMMITTED-RESOLUTION), so a contract the timeline already took and finished can never be accepted for a second advance.
2. **Known obligations are reserved up front.** Committed costs (section 3.9) are always reserved. The fail penalties of open contracts (funds and reputation; stock contracts carry no science penalty) are reserved from accept to outcome **only while a committed purchase or strategy cost lies ahead** (V, decision card 2026-10-22). A committed cost ahead exists only after a rewind, so a career that never rewinds plays exactly like stock. Where each reservation lives:
   - Committed funds and science costs are held out of the stock pool, as today: the stock widget shows what is available, and the Reserved tooltip shows the amount.
   - Open-contract penalties are never taken out of the stock pool, because stock pays a fail penalty out of that same pool and clamps it at 0. They are enforced where the money would be spent instead (part 3), and the Reserved tooltip includes them. Built in phase 2 (0.10.6): the funds projection leaves an open contract's synthetic deadline fail out of the pool reserve (it is told apart from ledger rows by membership; a committed fail row is still reserved). A deadline fail the walk injects itself moves contract state only, never a pool, even once its deadline has passed: stock files its own fail row when it charges one (section 8.6). Built in phase 4 (0.10.6): the click rules of part 3 hold the penalties, and the funds tooltip's Reserved line includes them while a committed cost lies ahead (the Total does not change, so Total minus Reserved is what a purchase can spend). A contract with a committed outcome after now is not counted: the committed timeline already holds that outcome.

- Reputation has no stock affordance to hold anything in: the pool shows the real value and every reputation reservation is enforced at the clicks (part 3).

3. **A click that digs into a reservation is greyed.** It uses the stock-control annotation pattern (section 14.2): the stock button greyed, the reason on its stock tooltip, a Harmony backstop on the stock method that reads the same predicate. What a click may spend is the available pool minus the open-penalty reservation (reputation: the pool minus what committed costs still need, minus the open-penalty reservation).
   - **Purchases** are refused when their cost is above that: facility upgrades and repairs, hires, part purchases (R&D and the editor's purchase prompt), tech, strategy setup costs, and every launch path (the editor Launch button and the Space Center launchpad and runway craft picker). The funds check is enforced, no longer advisory (O7; todo GUI-P15-D7-TWO-SAFETY-GATES-THAT-EXIST-AND-ARE-NEVER-ARMED). A launch is refused only for its cost, the way stock refuses an unaffordable one.
   - **Mission Control** Accept (the advance it pays and the penalty it adds to the reservation), Decline (its reputation cost) and Cancel (its lerped funds and reputation penalty, and the release of its own reservation) are refused only when a committed cost lies ahead in a pool and the click would leave that pool below nothing to spend, or, in a pool that is already short, shorter than before. A click that improves an already short pool is never refused.
   - A pool with no committed cost ahead refuses nothing and holds no open penalty, so a career that never rewinds plays exactly like stock.
   - Built in 0.10.6 (phase 4). One decision per control reads one snapshot (`GameActions/ClickReservation.cs` : the pure rules and `ClickReservationGate` , which probes the ledger once per ledger state and clock interval and reads the live pools and the live open contracts). The greyed control and its backstop read it together: the facility menu's Upgrade, Repair and Rebuild buttons and their `SpaceCenterBuilding` methods; the Astronaut Complex Hire button and `AstronautComplex.HireRecruit` ; the part tooltip, the editor purchase prompt and R&D purchase-all, with their backstops; the Administration Accept and its backstop (the setup's funds, science and reputation); the editor Launch button, the Space Center craft picker's Launch button and the per-site Launch buttons of the launch-site flyout, with stock's own `CanAffordLaunchTest` failing as the backstop of every launch path and its dialog giving the reason (the editor mark prices the ship with the crew the launch itself installs); Mission Control's Accept, Decline and Cancel through the `ContractReserve` decoration kind (blocked, never marked: the detail panel carries the reason) and the `Contract.Accept` , `Decline` and `Cancel` prefixes. Every refusal shows one sentence, "Reserved for later costs and contract penalties, blocked by timeline.", with the numbers. A committed-row block on the same control keeps its own reason. Stock first everywhere: a click stock itself refuses (a purchase above the live pool, Accept at the active-contract limit, a contract stock will not let you decline or cancel) is left to stock, with stock's reason and no Parsek text. A reputation cost is compared after stock's curve: a strategy's reputation setup is charged through it, so the click compares what it would really take, and the reason names that amount. If the reservation itself cannot be worked out (a Parsek fault), every click falls back to stock and one warning is logged. Tech research needs nothing new: stock contracts carry no science penalty, so the science a click may spend is the pool's own target, which the research block already reads.
   - Known limits: stock's auto-hire of crew at launch is part of the launch and is not refused; a contract the committed timeline accepts later, but that is not open now, is not in the open-penalty reservation (its penalty enters the walk with its committed outcome); reputation amounts use stock's curve at the current reputation.
   - **Part upgrades** (todo PART-UPGRADE-PURCHASE-UNRESERVED, built in 0.10.6) follow the part-purchase rules. Game rule: buying a part upgrade is a funds cost on the timeline, like buying a part. Stock sells one only from the part list tooltip (R&D and the editor) and charges its `entryCost` unless entry purchases are bypassed; Parsek books what stock charged as a `FundsSpending` row (source `PartUpgrade` , the upgrade name as its key), so the walk, the reserved floor and the click reservation count it. An upgrade the committed timeline buys later cannot be bought again before then (it would be charged twice), and one whose price would dig into what later costs and open contract penalties need is refused with the sentence above; the tooltip's purchase button is greyed with the reason and `PartListTooltipController.onPurchaseUpgrade` refuses the click with the same decision. When the clock reaches a committed purchase, `KspStatePatcher.PatchUnlockedUpgrades` marks the upgrade unlocked again with no second charge (add-only; an upgrade the install does not define, or whose tech is not researched, is logged and skipped). Stock KSP 1.12.5 defines no part upgrades, so this matters only to mods that add them.
   - Every KSC purchase stock charges on its own event (hires, parts, upgrades) books its row and then runs the reconcile and recalculation one frame later, after stock's `Funding` handler has charged: `EventData.Fire` runs listeners last-added first, so Parsek's handler runs before the charge, and a recalculation in that handler patched the pool to the target and then let stock subtract the cost a second time.
   - **A committed purchase takes effect in the update that pays for it** (built in 0.10.6, todos PURCHASE-STATE-PATCH-RUNS-WHEN-THE-FUNDS-PATCH-IS-CLAMPED and CROSSED-HIRE-NEVER-MADE-AFTER-A-SKIPPED-FUNDS-PATCH). Game rule: when the clock reaches a committed part purchase, part upgrade or hire, the part unlocks, the upgrade unlocks or the kerbal joins in the same update that charges its cost, never in an earlier one that left the funds alone. A clock crossing whose funds patch is skipped leaves the row counted but unapplied in the stock pool reference, so the next update that patches funds takes the cost and makes the purchase in the same pass (`KspStatePatcher.PatchPools` runs `PatchFunds`

before the add-only part and upgrade passes and holds them while a funds pool exists and its patch skipped; `LedgerOrchestrator.ResolveCommittedHiresToMake` makes the hires that pass committed). Science and Sandbox games have no funds pool, so nothing waits there.

4. **Any other loss stops at the reserved floor; the rest is a debt.** The floor is what committed costs still ahead need. A loss nobody can reserve ahead (a crew-death reputation hit, an open contract's fail, a committed repair a new destruction made chargeable) applies only down to the floor; the part below it is held as a debt and charged from the first room above the floor (above 0 once the last committed cost is paid): the reservation releasing at the committed cost's UT, or later gains. The debt is derived in the recalculation walk from existing rows, nothing about it is saved, and the funds and science tooltips show one `Debt:` line while one is outstanding.
   - **"A committed cost lies ahead of a loss"** means a cost row with a later UT that was already on the timeline when stock applied the loss, judged by real-time capture order, not by ledger order (coordinator ruling 2026-10-22). In a career that never rewinds every purchase is captured after the losses before it in time, so nothing is ever ahead of a loss and the career stays exactly stock; after a rewind the committed future's purchases were captured before every new flight's loss. The order is a capture sequence per save (`cseq` , an additive key on game-state events and ledger rows, no schema bump): stamped when an event is captured and carried onto every row converted from it, stamped at write for a row written directly, always one above every value the ledger and the event store hold. A row without one (an older save, including a row converted later from an event an older build captured) gets no floor and is ahead of nothing. The floor of a loss is what the cost rows captured before it need, and a debt is charged only from the room above what the cost rows captured before the loss or the charging row need.
   - Built in 0.10.6 (phase 3): the walk stops the loss at the floor and carries the debt; the stock pool reference steps each row by what stock applied (its effect before the floor and the debt), so the debt is never filed as a leak and stock's own 0 clamp is not counted twice. Code: `GameActions/ReservedFloorPlan.cs` , `GameActions/CaptureSequence.cs` ; plan section 2.3.
   - A purchase or strategy cost is never turned into debt by a click: it is paid in full or refused at its click (part 3). A committed one the walk can no longer pay after a rewind is charged anyway and the running value goes below 0, a debt later gains repay (funds as built in phase 3; science since section 3.15 step 6), so no committed purchase is ever left unpaid or taken back.
   - With no committed cost ahead a loss applies as stock applies it: funds and science stop at 0 and the rest is forgiven, reputation can go negative. Built in phase 2 (0.10.6) for every loss, and since phase 3 for every loss with no cost captured ahead of it: a funds or science row that is not a purchase or strategy cost and would take the walk's value below 0 stops at 0 (off under stock's `AllowNegativeCurrency`), so a gain after such a loss is kept. A strategy's science exchange debit counts as a cost here, like its funds half, and so does the reputation leg of a strategy exchange (Bail-Out Grant, Fundraising, Unpaid Research): its paired funds or science credit pays in full, so the leg is never stopped at a floor. Where a reservation holds the pool below the walk's value, stock stops such a loss at 0 before the walk does; that forgiven part is not given back, it stays a debt in the target, and later gains repay it before the pool rises (phase 2; phase 3 adds the explicit floor, under which the walk itself stops the loss and carries the part below the floor as its debt).
   - Reputation works through stock's curve: the reservation for a strategy's reputation cost is computed at a different reputation than the one it is paid at, so at high reputation it can fall a few percent short and the strategy can leave reputation that much below 0. Accepted; nothing is left unpaid.

Money the ledger does not know about stays as stock left it (BUILT 2026-10-22, phase 2). An earning or a spend stock applied that the ledger holds no row for (another mod, an unmodelled stock path) is kept as stock applied it, on every later write: Parsek never takes it back and never re-creates it. A reservation that releases always comes back to the pool. A save from an older build keeps its pools: whatever separates them from the ledger at the first update is kept the same way. Parsek records such money as one `StockPoolAdjustment` ledger row per pool at the update that finds it (untagged and hidden from the Timeline), so the rebuilt pools carry it from then on and stock's 0 clamp and reputation curve apply at the real pool value. (Mechanism: plan `docs/dev/done/plans/committed-value-rule-implementation.md` section 2.4; code `GameActions/StockPoolReference.cs` , read by `KspStatePatcher.PatchFunds` / `PatchScience` / `PatchReputation` and by the affordability gate, saved in the PARSEK node as `STOCK_POOL_REFERENCE` .)

Changes to this rule (section 3.15, owner rulings 2026-10-26).

- Built (section 3.15 step 5): money stock pays for an identity the ledger already credits is not money the ledger does not know about. The pool reference steps such a row by what stock paid (its walk effect plus the duplicate part), so the next write, the replay's target, takes the second payment back and nothing is adopted as an adjustment. Only for a payment a rewind made possible and only for rows captured by a build that ships it; every other repeat stock pays by its own rule is kept as before.
- Planned: an earning with no ledger channel is filed at its own event UT and is live-only (step 9).
- Pool differences an older save brings keep this rule.

Worked rulings. L1.4: reputation 60, a committed strategy costing 40 reputation on day 50, a new flight's crew deaths of 30 on day 20: reputation stops at 40 on day 20 with a debt of 10, reaches 0 when the strategy is paid, and the debt is paid by the next 10 of gains. L1.2: funds 120k, an open contract with a 50k fail penalty and a committed 100k upgrade on day 50: purchases may spend

nothing (the upgrade needs 100k, the penalty another 50k); if the contract fails on day 20, funds stop at 100k with a debt of 30k, the upgrade is paid, and the debt comes from later gains.

Flights are never blocked for their risk. "Of course we should not block flights of launches" (V). The debt is the player's risk to take: "we need a way to let the player make decisions about risks of rewinding and doing new launches, so I pick debt".

Cases that need no rule of their own. Deadline expiry: the committed timeline already holds the same fail at the same UT and penalty (section 8.6), and the expiry is the clock's own event, filed under no flight (section 3.14). Strategy converter debits: they divert a share of the earning or penalty that causes them, and follow that case. Killing a kerbal a committed flight uses: the crew reservation keeps him out of new flights (section 11). A building a new flight destroys: its repair is a Space Center click covered by part 3 and the facility blocks of section 14.4, and a committed repair made chargeable falls under part 4.

What the other operations keep (same interview):

- A stock revert removes only the rows the reverted flight wrote; every committed row stays, including the committed-future rows a rewind left (ruling D2). Built in 0.10.6: Parsek remembers the ledger's row ids at the same entries stock captures its two revert targets, and the revert prune (`ParsekScenario.PruneRevertedFlightLedgerRows` over `Ledger.PruneOrphanActionsAfterUT`) removes only the untagged rows after the launch boundary that were added since the target's entry. Revert-to-Launch goes back to `FlightDriver.PostInitState`, rebuilt at every flight start, so its rows are taken at every flight-scene load that is not a revert; Revert-to-editor goes back to `FlightDriver.PreLaunchState`, rebuilt only at a fresh launch and still offered after a resume of the same vessel on the pad, so its rows are taken only at a fresh launch and kept across later loads. Stock loads the scenarios before it charges the rollout, so Revert-to-editor refunds it, after an F9 on the pad too.
- A recording that fails to load, for any reason, keeps its ledger rows: it loses its ghost and playback, never its rows (extends the GC-3 ruling from schema rejects to every load failure). Built in 0.10.6: a committed tree that the schema gate rejects whole stays inert with its tagged rows (GC-3); any other committed recording the save names but the load drops (a sidecar that fails to hydrate, a partial schema reject, a tree that loads empty) has its rows kept with the tag cleared before the reconcile (`LedgerOrchestrator.KeepRowsOfRecordingsThatFailedToLoad`), because the next save no longer names it. That is the shape a non-rewind discard leaves. A never-committed Re-Fly provisional is left to the load sweep.
- Supply routes are a real-time system and carry no money: every load back in time, a revert included, retires a route's cycles after its cutoff and the clock re-decides them (as built), and route cycles write no funds and no ledger rows (both built in 0.10.6; their events live in the route event log; logistics design section 10.6, plan `docs/dev/done/plans/logistics-without-money.md`, todo `LOGISTICS-WITHOUT-MONEY`). Space Center origin routes are free in every mode. Why: route money on the ledger and free interaction with route vessels are mutually exclusive, and holding route vessels as ghosts or blocking a destination's move were both rejected (logistics design section 10.6.1, which also lists every model traced).
- Discarding a flight that resumed a committed vessel keeps what stock kept from it: a contract finished, a milestone reached or science banked while flying the resumed vessel becomes an untagged row, like any non-revert discard, while the committed flight's own rows up to its recorded end stay as they were. A rewind or a load that drops such a flight keeps nothing of it, because the load reverted stock too. Built in 0.10.6 (todo RESTORE-CLONE-DISCARD-DROPS-IRREVERSIBLE-CAREER-ACTIONS).
- A Re-Fly session keeps showing the replaced flight's losses until the merge retires them (rewind design section 2.12).
- A quickload or load restores the save exactly; `docs/dev/design-load-restores-the-save.md` owns it.

3.14 Events the clock causes are filed under no flight (owner ruling 2026-10-24)

Status: ruled 2026-10-24; built in 0.10.7 (PRs #2229, #2233). V: an event no flight causes "is an independent unique ledger event, it should not be linked to any flight recording". Rewind design section 2.12 ("Awards are filed by cause") carries the Re-Fly side of the same rule.

Game rule. Some career events happen because the clock reached a date, whatever any flight does: an accepted contract's deadline passing, stock ending an accepted contract whose requirements no longer hold, and a strategy reaching its maximum duration (KSPCommunityFixes' StrategyDuration). Others happen because an instrument already in place works on its own: a Breaking Ground ground station transmitting its science (itself always untagged, operator ruling 2026-09-26), and a Sentinel telescope detecting a comet or an asteroid on its own scan clock. Each one, and everything stock pays or completes because of it, is filed untagged, under no flight, and written to the ledger the moment it happens, in flight too. So:

- Discarding or reverting the flight that was in progress never removes it, and a Re-Fly's merge never retires it. A deadline that passes during a Re-Fly is not the attempt's loss: it never seals the slot and never refuses the merge.
- What the event charges goes with it: the contract's fail penalty, and every strategy conversion that penalty triggers, are filed with the expiry under no flight (`ContractEarlyFailPatch` opens a clock-cause scope around an allowed `DeadlineExpired` / `OfferExpired` end, inside which every capture is untagged).
- A contract a ground station's science or a Sentinel detection completes, with its reward, and a strategy's cut of ground-station science are filed under no flight the same way (`DeployedScienceSubmitScopePatch` around stock's `SubmitScienceData` for a

deployed subject no vessel submits; a ground station's data a kerbal carried aboard is the carrying flight's, `SentinelDetectionScopePatch` around the comet spawn and the asteroid discovery). A Sentinel launched by the flight in progress counts as already in place: its detection is not the flight's, like a ground station the flight deployed.
- A contract a flight fails before its deadline (a failed parameter, a lost vessel) is still that flight's own row.
- The rest of section 3.13 is unchanged: a contract the committed timeline already resolves is never expired early (part 1).

Strategies follow the committed rows exactly. V, same day: strategies must "be precise, to not cause compounding errors". Game rule: a strategy is active exactly while the committed timeline has it active, and a strategy changes only its own rows and the Administration building's availability, never another ledger row (section 11.3).

- A committed activation switches the strategy on exactly when the clock crosses its UT, with no second setup charge and no new row (`StrategyActivationGate` , a `StrategySystem.Update` postfix).
- The active set the patcher applies is rebuilt at the current clock from the committed rows (`StrategyStatePatcher.ResolveLedgerActiveAtNow`), so a walk that reads past now never switches on a later activation early.
- A one-time exchange strategy (stock's Bail-Out Grant and the other currency exchangers, which pay once when activated and then switch themselves off) is never switched on again by the patcher or the gate, so its grant is paid exactly once across rewinds and loads (todo STRATEGY-ACTIVATION-REPLAY-REPAYS-A-ONE-SHOT-EXCHANGER).

Status of the instrument half: built in 0.10.7 (todos CONTRACT-COMPLETED-BY-DEPLOYED-SCIENCE-TAGGED-TO-THE-FLIGHT, STRATEGY-CONVERSION-OF-DEPLOYED-SCIENCE-TAGGED-TO-THE-FLIGHT).

Mod background sources, built in 0.10.7 (LEDGER-EVENTS-OF-MOD-BACKGROUND-SOURCES-TAGGED-TO-THE-FLIGHT, proven by unit tests only by owner ruling 2026-10-25). The same scope opens around three more causes no flight has: an `AutoAccept` contract's accept (stock's `ContractSystem` accepts it on its own refresh; Contract Configurator contracts can carry the flag), a science submission from an unloaded vessel the live flight does not record (the recorder's vessel and the active tree's background members are the flight; stock transmits only from loaded vessels, and a loaded vessel outside the tree stays the flight's because the flight's EVA kerbal can run its experiment; a recovery keeps its own owner), and a resource scan no stock scanner makes (a mod's, such as SCANsat's background coverage; a stock scanner keeps the flight's tag on any vessel, for the same EVA reason). A mod contract a timer parameter completes stays tied to the flight: no seam names its cause.

3.15 Event identity: the replay decides every repeat (owner rulings 2026-10-26)

Status: ruled 2026-10-26. Build order steps 1 to 6 and 8 are built (science rows dated at their capture UT, `cseq` as the walk's tie key; capture dedup on capture identity; the identity registry and its suite gate; world records once per threshold; stock stopped first and corrected second; the purchase state rule and science debt; the Re-Fly seal on credit); step 7 is planned (todo `LEDGER-EVENT-IDENTITY`). This section is the authority for how the ledger tells a repeated event from a new one. The module sections point here. Where today's code differs, the module section says so and names the step below that changes it.

Game rule. The ledger is a list of things that happened, not a list of money movements. Each economic row says three things:

- what happened: its identity, named by the key stock itself checks before it pays;
- when it happened: the capture UT;
- what stock priced it at: the raw data.

The pools are never stored. They are recomputed by replaying the rows in time order:

- The first occurrence of an identity earns its price; for science, it earns what the subject cap leaves.
- A later occurrence earns what the identity's rule leaves, often nothing.
- A purchase is charged when the walk does not already own the item at that moment.

What stock pays live during a flight is a preview. The next update writes the pools from the replay.

Example: a committed flight recovers goo at UT 1000 (cap 3.9, stock paid 3). After a rewind, the player recovers the same goo at UT 200. The replay puts UT 200 first, and the UT 1000 run earns only what the cap leaves. This holds the never-reduce rule, because the total credited for the subject does not fall (owner ruling 2026-10-26, lane SCI-1).

Two kinds of duplicate.

- **Row duplicate:** one stock payment seen twice, for example through two capture channels. It is collapsed at capture and never reaches the replay.
- **Event duplicate:** the same achievement at two moments, for example on two timelines. Both rows are kept, and the replay decides which one earns.

1. Every economic row declares an identity and a rule. One registry names, for every `GameActionType` and source, the key and the rule below. A suite gate fails when a type or source has no entry, so a new channel cannot ship without a decision.

Source	Identity	Rule
Science experiment	subject id	cap headroom, earliest first (hard cap, owner ruling 2026-10-26; stock's curve is not replayed, 12.8)
World-first	body-qualified progress node id	first wins (section 9)
World record	node id + threshold index	each threshold once, earliest crossing first (step 4, built)
Contract complete / fail / cancel	contract guid, one outcome ever (a re-accept does not reopen it)	first outcome wins (section 10)
Contract parameter	contract guid + parameter path	first wins, and only while the walk has the contract open (not completed, failed or cancelled). A mod parameter stock may pay again (Contract Configurator `disableOnStateChange = false`) keys on the condition stock checks before paying.
Contract accept advance	contract guid + accept	once per accept
Orbital survey bonus	body	once per body, once a decompile confirms stock pays once
Continuous producer (Breaking Ground station, Kerbalism background science)	producer + subject + time slice	a slice the committed timeline already holds earns nothing
Tech node, part, part upgrade, facility level, hire	item name; facility + exact level	purchase state rule (step 6, built)
Rollout cost	launch guid	as recorded
Vessel recovery	owner recording + launch guid	as recorded
Strategy setup	strategy + activation	once per activation; a one-time exchanger is never switched on again (3.14)
Strategy conversion leg	its cause row	follows its cause (step 7)
Reputation	rides its source row	nominal stored, stock's curve re-run in the walk (section 8)
Science lab payout	none; the lab bounds itself	as recorded
Income with no ledger channel	reason + event UT	live-only (step 9)
Seeds (`*Initial`)	not events	starting value

Built (build order step 3). `LedgerEventIdentityRegistry` (`Source/Parsek/GameActions/`) is the one place that names, for every `GameActionType` , the identity key and the rule. A type splits into one entry per source where the sources need different rules: `FundsEarningSource` , `FundsSpendingSource` , `ReputationSource` , `ReputationPenaltySource` , `StrategyConversionSource` on `StrategyScienceDebit` , a milestone's one-shot node against a world record, and a lab payout against the orbital survey bonus. Non-economic rows (crew, destruction, deactivation, scans, recovery evidence) are `NoEconomicEffect` , and the reserved route types are `NoLedgerRow` .

- Each entry says whether the walk enforces its rule today and where, or which step of this section plans it and what the walk does until then. Enforced today: science cap headroom, a world-first's first wins, a world record's thresholds once each, a contract's first outcome, a contract parameter's first wins while the contract is open, the repair state rule, and the purchase state rule for tech, parts, upgrades, facility levels and hires (step 6). Planned: conversion legs (step 7), channel-less income (step 9) and the survey bonus (build order step 7).
- `For(action)` returns a row's entry and `Describe()` gives a one-line text for logs. The registry changes no walk behaviour; later steps wire the modules to it.

- The suite gate `LedgerEventIdentityRegistryTests` enumerates every `GameActionType` and every value of each split source enum, and fails naming any value with no entry. A probe walk checks each enforced deciding rule against the modules, and a cell pins that the planned survey rule still behaves as its entry says.
- The one behaviour change of this step: a contract parameter row the walk reaches after its contract's credited completion is not counted (8.3).

2. Order is time, then capture order.

- The walk sorts on (UT, earnings before spendings, `cseq`). `cseq` replaces `Sequence` as the tie key, so two KSC clicks at one frozen UT keep their real order (3.7).
- Every row carries its capture UT as a double.
- A science earning is dated at its capture UT (built; it used to be dated at its recording's end UT, which misordered a transmission against a re-earn after a rewind).
- An older row whose capture UT survives only as a float `StartUT` ties on `cseq`.

3. Row duplicates collapse on capture identity. Capture dedup keys on the capture event (its `cseq` and channel), never on economic keys. A 0.1 s window over type and amount could hide a real second event; RELAUNCH-ROLLOUT-DEDUPED-AS-REVERT was one. This step lands before take-back, so a doubled capture is never read as a stock re-payment.

Built (build order step 2). Each site and what it keys on now:

- `LedgerOrchestrator.DeduplicateAgainst` (every recording commit and its preview, the discard re-home, the retired Re-Fly provisional, the recovery XP, crew-close and vessel-recovered writers): a row converted from a captured event (`GameStateEventConverter.ConvertEvent`, transient `HasEventCaptureIdentity`) collapses only onto a row with the same `cseq` and key, at any UT. A row of another capture with the same type, key and 0.1 s window is a second event and stays. Science keeps its own capture id (`ActionId`). The type + UT + key match stays the fallback when either side names no capture: rows from older saves, and rows built without an event (vessel cost, crew assignment, the death penalty, recovery rows).
- `LedgerLoadMigration.LedgerHasMatchingAction` (the save recovery migration): the event's row is the one with its `cseq`; the economic match only for a pair without one.
- `GameStateStore.EventIdentityMatches` (event detail update, retag, remove): two events with one moment, type, key and tag are told apart by `cseq`; a copy taken before the store stamped it keeps the field match.
- Kept as they are, because they cannot hold two events: the recorder's contract and parameter re-fire debounce and the converter's contract bucket (contract guid inside 0.1 s, one outcome per accept, cleared at every scene change, one recording's events), the rollout dedup (launch guid, built by RELAUNCH-ROLLOUT-DEDUPED-AS-REVERT), the recovery funds pairing (the paired event's UT, pool values and tag), the resource-event coalescing (one pool's contiguous moves inside 0.1 s merged, the sum kept), the crew-status bounce (no ledger row). A world record's thresholds folded into one milestone row stay folded: the row names each threshold it holds (step 4), so the replay tells a re-crossing from a new one.

4. World records pay once per threshold. Built.

- Game rule (owner ruling 2026-10-26, "pay once per career, earliest crossing wins"): each step of a record (500 m, 2 km, 7 km... for altitude) pays once in a career. If a rewind lets a later flight reach a step again before the committed flight's crossing, the earlier crossing gets the reward and the committed one keeps only its steps that are still new. A flight that sets a record already set that far on the timeline earns nothing for it.
- A record row names the thresholds it crossed and the reward of each one (`MilestoneRecordBand`, 7.8).
- The replay credits each threshold once, earliest crossing first in walk order (`RecordThresholdBook` in `MilestonesModule`). An overlap is a repeat only on rewind evidence (a capture order the walk inverts, or two crossings of one interval), so a career on one timeline walks exactly what stock paid, stock's own re-payments included. A row whose thresholds were all credited earlier earns 0 (`RecordThresholdsAlreadyCredited`); a row with some new thresholds earns only those, in funds, reputation and science.
- The live mirror raises stock's band to the committed count at the clock (7.9).
- Rows written before the field get their positions from the cumulative count per node, in capture order, so the earliest crossing still wins for them too (7.8).

5. Stock is stopped first and corrected second (owner ruling 2026-10-26: prevention plus take-back).

- **Prevention.** One add-only live mirror pass generalises `CommittedMilestoneReach`. For every registered kind whose stock state can record "already paid", it sets that state at the clock without paying: subject science, world-first nodes, record bands, contract parameters, scans and purchases. The flight clock cursor fires on any registered kind, so stock does not pay a known identity in the first place.
- **Take-back.** When stock paid live for a row that the replay zeroes or caps for an identity reason, the next update takes the difference back and logs one warning naming the identity.

    - The amount comes from a stock-shadow replay: what stock paid is the captured amount, with reputation run through the curve at the shadow's own reputation and stock's 0 clamp applied.
    - It is taken back only for an identity reason (a duplicate or a cap), never for a floor or a strategy share.
    - It is applied only to rows first captured after the build that ships it.
    - It is never filed as a `StockPoolAdjustment` row, because the ground-truth check counts those separately and would hide it.
    - The player has no window to spend the duplicate first. Nothing can be spent in flight, and updates are held back while a recorder or a pending tree exists. Auto-merge (always on in shipping) commits the tree at scene exit, and the Space Center's scene-entry update runs before a click is possible. A merge dialog locks all input. A payment made at the Space Center is followed by an update one frame later. If a shortfall still appears (auto-merge off, a test setting), it becomes a debt (3.13 part 4; science the same way since step 6).
- Prevention changes the raw data the replay sees: stock pays the remainder (0.69) instead of the full first reading (3). Under the hard cap the credited total is the same.

Built (build order step 5). Game rule for the player: once the clock has passed a moment the timeline already credited (a world-first, a record step, a contract or one of its parameters, science from a subject), stock does not pay it again; and if stock pays it anyway because a rewind made it forget, the next update takes that payment back, so it is earned once. A career that never rewinds ends exactly as before.

- **Prevention.** `CommittedMilestoneReach.Apply` mirrors every registered kind whose stock state records "already paid" (`IsMirroredRow`): world-first nodes and record bands as before, a subject's committed science at the clock (`CollectCommittedSubjectScienceAtOrBefore`, the walk's own cap rule; `ScienceSubjectPatch` injects the larger of it and the walk's credited total, so stock's diminishing returns count it; the mirror is replaced on every pass and cleared by a patch that is not deferred and by every load), contract parameters (the ones the walk credited, as the full patch completes them; completed without reward on their live Active contract, `KspStatePatcher.MirrorCommittedContractParameters`, sharing the full patch's planner; the contract is not handed to stock's completion path until the next full patch) and resource scans (`MirrorCommittedResourceScans`). Add-only, inside the replay suppression, skipping a live Re-Fly session's suppressed subtree. `CommittedMilestoneClockCursor` fires when the flight clock passes a row of any of these kinds. Purchases are not mirrored here: a committed purchase takes effect only in the update that charges it (3.13 part 3), and the flight scene that runs this pass offers no purchase, so the full patch's purchase passes already cover them.
- **When stock forgot.** A time-travel load (Plain rewind, Re-Fly start, Discard Re-fly, stock revert, an unannounced flight quickload; `StockShadowTakeBack.IsTimeTravelLoad`) keeps the in-memory ledger and loads an older stock state. `ParsekScenario.OnLoad` records it as a timeline break: the loaded UT and the capture high water at the load (`timelineBreak` values in the `LEDGER` node, additive, no schema bump). The plain rewind's branch returns before the refined classification, so it records its own break (at the loaded game's clock, else the rewind save's); every other kind records it on the refined kind in the in-session branch (`TimelineBreakWiringGateTests` pins both calls and that no other return in OnLoad can skip them). A science row gets its capture number when its recording commits (`CaptureSequence.StampBatchIfMissing`), so for science "captured after the break" means committed after it. A repeat is taken back only for the part whose crediting row (its witness) a break separates from it: the witness captured at or before the break's high water, the repeat after it, the witness's UT after the loaded UT. Without a break nothing is taken back, so a repeat stock pays by its own rule (a mod parameter that pays again, a duplicate event) is kept as before. A contract the walk ended at its deadline is never a witness: stock keeps the contract's own deadline across a rewind, so a reward or penalty it paid past the walk's deadline is a deadline disagreement and is kept as before (a fail stock really recorded is a witness; a fail the walk synthesized at its deadline is not). A record row is the one exception: its evidence is step 4's own (below).
- **Records.** A record threshold is taken back only on the rewind evidence step 4 zeroes it on: a named first payment of an interval an earlier capture already paid as new (`RecordThresholdBook.BlockReason.SecondFirstPayment`, `RowCredit.SecondFirstPayments`). On one timeline stock's band only advances, so a second first payment proves the band was lowered in between, and no break is needed. Never taken back: a re-payment (an `r` entry, or a re-payment folded into its scope's crossing; stock pays those on one timeline), a threshold at or below the pre-ledger seed, a re-payment of a zeroed crossing, a derived row's guess, and an older row zeroed because a later capture crossed earlier (it was paid on its own timeline, and the later capture is the credit). A capture folds every award of an interval into its entry, so an entry can hold re-payments; stock re-pays only at the interval its loop stops on, the row's highest, so the row's award count less its distinct intervals is the folded re-payments and the highest entry is divided by that count plus one (`MilestonesModule.ResolveRecordTakeBack`; a count below the intervals takes nothing back). The witness names each threshold and the capture that paid it first.
- **Era.** Only rows first captured by this build: the first capture number this build issues for a save starts the era (`takeBackFromCseq`, additive), and a row below it, unsequenced, or Parsek-authored is never taken back.
- **Amounts.** The walk modules decide (a whole repeat of a world-first, a contract outcome or a parameter; one payment per second first payment of a record row; the part of a capped science row that forgotten credit covers) and the pool modules compute what stock paid beyond the walk: the captured amount, funds and science with stock's 0 clamp, reputation through stock's curve. Both the clamp and the curve run at stock's own pool when it paid the repeat (`StockShadowStart`): what the pool

reference last wrote or read back (`LastWritten`, which already holds every committed row an undeferred write put back after the rewind, and sits at the projected minimum after a measured write), plus the rows stock paid live since that patch (not counted by the reference, stock-applied, captured before the repeat), each at its walk effect plus its own take-back part. A committed row the reference saw unwalked was never paid by stock and adds nothing. With no reference yet, the walk's value. A repeated fail or cancel penalty is given back the same way (a negative take-back). Never for a floor or a strategy share.

- **Applied by the write.** `StockPoolReference.Measure` steps such a row, the first time it sees it, by what stock paid, so the duplicate is no leak and the write (the replay's target) takes it back. One Warn per row names the identity, the row and the witness (`[TakeBack] Took back ...`). Nothing is filed as a `StockPoolAdjustment`. An authoritative patch writes the target anyway and logs the same line.
- **Shortfall.** If the duplicate was spent before the update (auto-merge off only), funds keep the walk's running value below 0 and the floored write leaves it as a debt repaid by later gains (3.13 part 4), with one Warn; reputation has no 0 clamp and is taken in full; science is the same debt as funds since step 6 (one Warn, "later gains repay it first").
- Tests: `StockShadowTakeBackTests`, `CommittedMirrorPreventionTests`, `TimelineBreakWiringGateTests`; the registry entries carry the stock side (`StockMirror`, `TakesBackStockRepeat`) and a gate ties every identity reason the take-back accepts to a take-back entry.

6. Purchases follow a state rule, and science can go into debt.

- A purchase is charged when the walk state at its (UT, `cseq`) does not already own the item: the tech researched, the part or upgrade unlocked, the facility at that exact level, the kerbal on the roster.
- A legitimate repeat charges again: rebuilding a level down and upgrading it again, re-hiring after a dismissal, activating a strategy again, accepting a contract again.
- A purchase the walk already owns is shown in the Timeline as not counted. The reservations, the reserved floor and the click gates read the same predicate.
- A hire keeps the price captured with it, even when an earlier new hire changes the roster size its price depended on (owner ruling 2026-10-26).
- A committed tech that the walk can no longer afford is charged into debt like a funds row, instead of staying researched for free (12.6; owner ruling 2026-10-26).

Built (build order step 6). Game rule for the player: after a rewind, a purchase the timeline already made earlier is not charged a second time, and the Timeline greys it with "Not counted: already bought earlier on timeline, so nothing was charged."; a committed tech the career can no longer pay for stays researched and its cost becomes a science debt that later science repays before the pool rises, the same way funds work. A career that never rewinds is charged exactly as before.

- **The state rule.** `PurchaseStateRule` (`Source/Parsek/GameActions/`) names the item a row buys (`TryGetItem`: a tech node, a part, a part upgrade, a facility at its exact level, a kerbal), and `PurchaseOwnershipBook`, fed in walk order (`RecalculationEngine.CompareWalkOrder`), records what the walk owns. A row is not charged (`PurchaseAlreadyOwned`, Effective false, one Info line naming the holder) when the walk already owns its item and the purchase that holds it was captured after this row (`IsCapturedAfter`, both `cseq` above 0): a rewind separated them. A holder captured before the row means stock itself sold the item again in real time, which happens only when the walk cannot see the step that gave it up (a dismissal writes no ledger row; an older save's Rebuild writes none), so the row charges. A row without a capture number is never judged. So in a career that never rewinds nothing is ever owned at a purchase.
- **Facility levels.** The walk holds a facility at one level, with the row that took it to each level since the last Rebuild. Only a Rebuild (a `FacilityUpgrade` row flagged `FacilityDowngrade`) takes a facility down. An ordinary upgrade row to a level at or below the held one is an old timeline's step the walk already has: it is owned when the row that took the walk to that level (or started the run above it) was captured after it, and a Rebuild is owned only at its exact level. An owned row is neither charged nor applied (`FacilitiesModule` skips it through the same book), so the level never dips below one the walk paid for.
- **Legitimate repeats.** A Rebuild ends the holding of the higher level, so raising it again charges, and a committed Rebuild after a rewind re-bought the levels still takes the facility down at its UT. A re-hire after a dismissal charges (stock's `KerbalRoster.SackAvailable` makes the kerbal an applicant again; the dismissal has no row, and the holder was captured before the re-hire). A strategy and a contract keep their own once-per-activation and once-per-accept rules and are not decided here. A hire is charged at the price captured with it.
- **Sites.** `ScienceModule.ProcessSpending` (tech), `FundsModule.ProcessFundsSpending` (part, part upgrade), `ProcessFacilityUpgrade`, `ProcessKerbalHire`. An owned row is removed from the prepass committed-spendings total and from `TryGetRowDelta`, so the projection, the reserved floor, the click probe and the stock pool reference leave it out; the stock-screen reservation index (`CommittedFutureIndex.Build`) skips the rows `PurchaseStateRule.CollectAlreadyOwned` returns, the same book over the same rows in the same order. An owned facility row is the exception and stays in the index as a holder of the facility's controls until its UT: a later Rebuild or upgrade click could turn it back into a level change, while nothing the player does un-owns a tech, part, upgrade or kerbal. An owned hire makes nobody when the clock reaches it (`LedgerOrchestrator.ComputeClockCrossing`, `CollectHireRowsNotAppliedByFunds`); an owned tech, part or upgrade is added by the add-only patch passes that already hold it from its earlier owner, so it changes nothing.

- **Science debt.** `ScienceModule.ProcessSpending` charges a tech in full; when the running science cannot cover it the running value goes below 0 (`ResolveDebtShortfall` , one Info line "Spending charged into science debt") and `Affordable` stays true, so the tech patch keeps it researched and the re-lock guard (`ShouldRefuseUnaffordableRelock`) is never armed by a walked row. The write is floored at 0 by `StockPoolReference.ResolveCurrencyWriteTarget` exactly as for funds, the tooltip shows the negative balance the same way, and step 5's science take-back shortfall Warns as the same debt (`KspStatePatcher.NoteScienceTakeBackShortfall`).
- Tests: `PurchaseStateRuleTests` (each kind already owned, the legitimate repeats, the hire price, the index and click probe, a randomized never-rewound career that walks exactly as before), `ScienceDebtTests` , `LedgerEventIdentityRegistryTests` (the purchase entries are enforced and probed).

7. Conversion legs follow their cause.

- A strategy conversion leg captured inside stock's currency-modifier query for a known cause row carries that cause's id. The replay scales the leg by the cause's effective share, so a zeroed duplicate takes its cut with it (11.4).
- A leg with no positive evidence of its cause stays as captured and is never taken back, because a mod may pay with a strategy reason (`StrategyOutput` , `StrategyInput`).
- Moving a cause outside a strategy's window after a rewind shifts value between currencies; the player gets the unconverted payout instead. Accepted (owner ruling 2026-10-26).

8. Contract twins stay out of the offer pool.

- After a rewind, stock can offer the same goal again under a new guid.
- A contract the committed future accepts stays out of the offer pool, which also covers a contract the clock accepts on its own (`AutoAccept`).
- Once-ever goals are never keyed on contract type and target, because that would zero repeatable contracts: part tests, tourism, rescues, Contract Configurator `maxCompletions > 1` .

9. Income with no ledger channel is live-only (owner ruling 2026-10-26).

- An earning stock applied for a reason no channel converts (a mod reward, cheat funds) is filed as an untagged row at its own event UT with its reason. Today it is filed as an adjustment dated at the next update.
- A rewind to before it drops it. A mod whose state rolled back with the save then pays it again once. A mod whose state does not roll back loses it.
- Pool differences an older save brings stay `StockPoolAdjustment` rows (3.13).

10. A Re-Fly seal reads credit, not presence. A gain row seals a slot when the replay counts it, even at zero value, so a first-reached milestone with no award still seals (rewind design section 2.12). A duplicate the replay does not count does not seal, so prevention and take-back give the same slot state for the same play (owner ruling 2026-10-26). A not-counted row stays out of the seal only when its identity has a counted holder row the timeline keeps AND stock's payment of it was prevented or is taken back; any other not-counted row seals, because a payment stock kept stays as a `StockPoolAdjustment` that sits in no retire set, and a re-fly could earn it again.

Built (build order step 8). Game rule for the player: a crashed flight is still offered for Re-Fly when everything it gained after its separation was something the timeline had already credited (a world-first or record step already reached, a contract already completed, a part already bought, science from a subject already at its cap); a flight that earned anything the timeline counts, even a first milestone that paid nothing, is sealed as before. A career that never rewinds is unaffected: on one timeline every gain the replay walks is counted.

- **The predicate.** `ReplayCredit.NotCountedLabel` reads a row as a walk has just left it. A row stays out of the seal when both hold:
  - Holder: `Effective` is false with a reason that names a counted holder row (`ReplayCredit.IsNotCountedReason` : already completed, already resolved, outcome after end, milestone already achieved, parameter after completion, record thresholds already credited, purchase already owned), or a science earning with a positive award whose subject cap left it nothing (only when a science module took part in the walk; the holder is the earlier credit of the subject). `ContractDeadlinePassed` has no holder (the walk's deadline is synthesized, and stock keeps the reward it paid past it, which is never taken back), nor does `FacilityAlreadyIntact` ; both seal. A reason added later seals until it is reviewed (gate `ReFlySealCreditTests.EveryNotCountedReason_IsReviewed` , which pins each reason's verdict).
  - Settled payment (`ReplayCredit.IsStockPaymentSettled`): the row carries no captured amount (stock was prevented and paid nothing), or the walk takes its payment back (`GameAction.HasTakeBack` , step 5), or it is a purchase already owned (what stock kept of it is a charge, the player's loss, never a credit a re-fly could collect twice). A paid repeat with no take-back seals: below the take-back era, with no timeline break between it and its holder, or the same-tick explicit resolution (a completion the pre-pass zeroes because a failure at the same UT resolved the contract; it has no witness, so nothing takes it back).

    - An `Effective` false with no reviewed reason, a partly capped science row and a zero-value first credit all count. A paid repeat captured before its holder (no take-back) seals: the safe side.

- **Which walk.** Never the flags some earlier walk left on the shared rows: a cutoff walk, a probe, or a walk of a list that held an attempt's earlier copy can leave a stale `Effective = false` on a row that holds a real counted gain. `SupersedeCommit.ResolveReplayNotCounted` runs its own full walk (no cutoff) of the ELS on clones (`RecalculationEngine.WalkBalancesOnClones`, which now reports `NotCountedByActionId`), plus the rows a commit is about to file when the gate reads them (Site A's pending rows, the merge preview's attempt rows). The ELS-only walk is cached with the gate's own cache, cleared on every ledger, store, supersede and tombstone version bump; so is the walk with a tree commit's pending rows (one walk for every gate read inside the scope, which `CommitTree` opens once per promoted slot, cleared when the scope opens or closes); any other walk with extra rows is not cached. Two rows sharing one ActionId (older saves) keep the id sealing when either copy seals. A retire set holding only loss rows runs no walk. A walk that throws, or no walk modules, leaves every non-loss row sealing (the presence rule).
- **Sites.** `SupersedeCommit.SelectSealingRows` is the one filter: the value gate `RetireSetHoldsSealingRow` (Site A, Stash, Site B-1, `RewindInvoker.CanInvoke`, the load-time value-seal sweep, the auto-seal preview) and the merge's Rule 1 re-check in `ReFlyMergeDominance.Evaluate`, which judges the retire set on its own walk of `L` plus any retire-set row the ledger has not filed yet (`ComposeCreditRows`). The merge still retires a not-counted row; that retires nothing the replay counts, and Rule 5's balance check still runs on `L` and `L'`.
- **Logs.** The gate line adds `notCounted=<n> credit=<cached|cached+extra|els|els+extra|walk-failed|none>` and names the rows it let through (`notCountedRows=[Type:ActionId(reason)]`); one `Seal credit walk` Verbose line per walk; the dominance verdict line ends `notCounted=<n>`.
- Tests: `ReFlySealCreditTests` (zero-award first milestone, counted contract outcome and spend, each taken-back or prevented duplicate, the deadline-passed completion and its merge refusal, a repeat below the take-back era or with no break, the same-tick resolved completion, a shared ActionId, losses, stale flags, cache invalidation, a walk that throws, pending rows and their shared walk, the Rule 1 re-check).

Out of scope for the first release (owner ruling 2026-10-26).

- Kerbal Construction Time and RP-1 charge at queue time and refund scraps as positive rollout deltas.
- Milestone rows store amounts after the difficulty multiplier, so changing difficulty mid-career shifts what an earlier copy is worth. This is a known limit.

Ledger readers move with each step.

- The harness ledger oracle (`once_key`) learns science remainders, thresholds and leg scaling. Built with step 8 (`harness/lib/oracle.py`, rewind-aware model): `subjectCap` gives later copies of a subject only the remainder, earliest first; a record is keyed per threshold; a purchase kind follows the state rule by capture order (`seq`); a once-shadowed or capped entry is the replay's take-back, never an adjustment entry, and stock's live payment of it still corroborates. Leg scaling waits for step 7.
- The Timeline shows each new not-counted reason.
- LedgerGroundTruth reads take-back as part of the replay, never as an adjustment. Checked with step 8: the reconstruction reads the walk's running pools, which are the replay, and only `StockPoolAdjustment` rows are left out of the comparison (`LedgerGroundTruthAfterRewindTests.TakeBack_IsPartOfTheReplay_NeverAnAdjustment`).

Build order.

1. Capture UT (double) on every row, and `cseq` as the tie key. Built.
2. Capture dedup on capture identity. Built.
3. The registry and its suite gate. Built: `LedgerEventIdentityRegistry`, gate `LedgerEventIdentityRegistryTests`, and the parameter-after-completion fix.
4. Records per threshold. Built: `MilestoneRecordBand`, `RecordThresholdBook`, the `CommittedMilestoneReach` band raise.
5. Stock-shadow take-back. Built: the generalised `CommittedMilestoneReach` mirror and cursor, `StockShadowTakeBack` (era, timeline breaks, witnesses), the pool modules' stock shadow, the pool reference's take-back step.
6. Science debt, then the purchase state rule. Built: `PurchaseStateRule`, `PurchaseOwnershipBook`, `PurchaseAlreadyOwned`, the science debt in `ScienceModule.ProcessSpending`.
7. Conversion legs, contract twins, the survey bonus and continuous producers.
8. The Re-Fly seal on credit. Built: `ReplayCredit`, `SupersedeCommit.ResolveReplayNotCounted` / `SelectSealingRows`, the dominance check's Rule 1 re-check on its own walk.

Each step ships its oracle and Timeline changes.

Edge cases checked by hand (simulated career, 2026-10-26): Rewind-to-Launch, then Re-Fly of a booster, then F9 inside a flight, then a second Rewind-to-Launch.

- Today's code ends that career about 8,000 funds and 12 reputation over: a contract paid twice, a record threshold paid three times, and a strategy cut kept from a zeroed parameter.
- The plan ends it exact.
- An F9 needs no rule: it is a cold load, and pools, ledger and stock state all come from the quicksave (`docs/dev/design-load-restores-the-save.md`).

---

4. Persistence Architecture

4.1 File layout

The game actions system persists its data in a **dedicated ledger file** on disk, separate from both the KSP save file and the recording sidecar files. The ScenarioModule in the .sfs save file holds a reference to this ledger file (same pattern as recording sidecar files).

```
saves/
  savegame/
    persistent.sfs              ← KSP save file (ScenarioModule references the ledger file)
    Parsek/
      GameState/
        ledger.pgld             ← single file: all earnings + spendings, all resource types
      recordings/
        rec_001.dat             ← sidecar: trajectory + raw science events (rich, verbose)
        rec_002.dat
        ...
```

The ledger file contains all game actions across all resource modules — science earnings, science spendings, fund transactions, milestones, contracts, kerbal events, facility upgrades. One file, one source of truth for recalculation.

The save carries its own copy (2026-10-20, `docs/dev/design-load-restores-the-save.md` 4.3): every OnSave also writes the ledger, the game-state events, the milestones, the baselines and the pending science into a `GAME_STATE` node of the Parsek scenario node, and a cold load reads them from there, so loading an older save gets that save's ledger rather than the last one written. The `Parsek/GameState/*` files stay as a mirror of the latest save (offline tools and the fallback for a save written before the node existed).

4.2 Data flow

During flight: KSP callbacks (e.g. `OnScienceReceived`) fire as events happen. Event data is captured by `GameStateRecorder` into `GameStateStore` (in-memory). The ledger file is not touched during flight.

On commit: `GameStateEventConverter` transforms `GameStateStore` events into `GameAction` entries and appends them to the ledger. The `GameStateStore` events are the source; no sidecar files are involved for game actions.

On recalculation: The ledger file is the sole data source. All entries are loaded, sorted, and walked. No sidecar files are opened.

Spending actions (KSC): Tech node unlocks, facility upgrades, kerbal hires — these happen outside any flight recording. They are written directly to the ledger file with no sidecar involvement.

Implementation divergence: The actual data flow uses `GameStateRecorder` → `GameStateStore` (in-memory) → `GameStateEventConverter` → Ledger (at commit time). Raw events are NOT written to recording sidecar files — they are captured in memory during flight and converted to `GameAction` entries when the recording is committed. The sidecar retains trajectory/part event data only.

Commit pipeline (v0.4): At commit time, `LedgerOrchestrator.OnRecordingCommitted` runs a multi-step pipeline: (1) convert `GameStateStore` events to `GameAction` entries, (2) deduplicate against existing ledger entries (`DeduplicateAgainstLedger` — prevents double-adding KSC events like tech unlocks, facility upgrades, and hires that may overlap with the recording's time range), (3) create vessel cost actions from `Recording.PreLaunchFunds` delta, (4) create `KerbalAssignment` actions from crew snapshot data, (5) append to ledger, (6) recalculate and patch.

KSP load migration: `LedgerOrchestrator.OnKspLoad` includes a migration path — if the ledger is empty but committed recordings exist, old save events are converted to bootstrap the ledger.

4.3 KSP load reconciliation

When KSP loads a save, the ScenarioModule provides the ledger file reference and the list of committed recording IDs. Parsek reconciles the ledger file against the loaded state:

1. **Seed actions** (`FundsInitial`, `ScienceInitial`, `ReputationInitial`) are never pruned — they represent the career starting state.
2. **Earning actions:** Any entry whose `recordingId` is not in the loaded save's recording list is pruned from the ledger file.
3. **Spending actions:** Any entry whose UT is after the loaded save's current UT is pruned.
4. **Recalculation** runs on the pruned dataset.

After reconciliation, the ledger file reflects the loaded save's state. The pruned entries are lost (the sidecar files still have the raw earning data as a passive backup, but Parsek never reads from them to restore ledger state).

Owner ruling 2026-10-21 (section 3.13, built in 0.10.6): a recording the loaded save names but that fails to load, for any reason, still keeps its rows in step 2. It loses its ghost and playback, never its rows: an inert tree (GC-3) keeps them tagged, any other dropped committed recording keeps them with the tag cleared.

4.4 Why not ScenarioModule or sidecar-only

The ledger data was considered for storage inside the ScenarioModule (embedded in the .sfs save file) or solely within recording sidecar files. Both were rejected:

ScenarioModule (inside .sfs): Would make KSP load semantics automatic (load = correct state for free), but embeds structured data inside KSP's own save format, harder to inspect and manage externally.

Sidecar-only: Would give a single source of truth with zero duplication, but makes recalculation depend on the file system (reading N sidecar files per recalc), creates coupling between the ledger module and recording internals, and makes the system fragile to sidecar corruption or deletion.

Dedicated ledger file: Self-contained for recalculation (no file system traversal), independent from recording internals (data is extracted at commit time and never read from sidecars again), inspectable on disk, and follows the same reference pattern already established for sidecar files.

4.5 Initial state seeding

Some resources start at non-zero values when a career save is created. The ledger must be seeded with these initial values, extracted from the save file — not assumed from defaults, as the player may use custom difficulty settings or mods that change starting values.

All three core resources are seeded: **funds** (career starting funds vary by difficulty), **science**, and **reputation**. While science and reputation typically start at 0 in stock difficulty presets, seeding them is necessary to handle mid-career Parsek installation — without seeding, installing Parsek on an existing career save would wipe the player's accumulated science and reputation balances to 0 during recalculation.

The seeds are written to the ledger file once when Parsek first initializes on a career save. They are immutable. Action types: `FundsInitial`, `ScienceInitial`, `ReputationInitial`.

Reputation already inside the seed. A reputation seed read off the live pool holds every reputation change stock has applied so far, so a ledger row for one of those changes must not be applied on top of it (the row's reputation leg is skipped, `insideRepSeed`; its funds and science legs still apply). Two cases mark a row inside: (1) the commit that files it is the one that read the seed, or the seed is still waiting for its first read (production order, never game time); (2) the seed was read while the row's recording was recorded but not yet committed, in the active tree or the pending tree a quickload restores, and the row's UT is strictly before that read. For (2) the seed row stores the uncommitted recording ids and the read's UT (`seedPendingRec`, `seedCaptureUT`, sparse); a read during a load takes the loaded save's clock, and a recording the optimizer splits keeps both halves on the list. Any award a recording earns after the read, and every row of a recording that did not exist at the read (a later flight, a Re-Fly), still counts. When in doubt a row counts: a double count is caught by the stock pool guard as an adjustment, while a row wrongly skipped would take away reputation the player earned.

Progress seed. Stock progress has the same mid-career problem: the milestone patch rebuilds every progress node from `MilestoneAchievement` rows, and progress earned before Parsek has no row. `PreLedgerProgressSeed` records it once per save, at the first recalculation after ProgressTracking's OnLoad has provably run (`CurrencyScenarioReadiness.IsScenarioModuleLoaded`; before that the live tree reads every node unreached). It holds (a) the qualified ids (bare at the top level, `Body/Node` in a body subtree) of one-shot nodes that are complete in the live tree and that no ledger row in any state and no captured

`MilestoneAchieved` game-state event names (a row anywhere on the timeline, or a pending flight's event, means the ledger owns the node), and (b) per world-record node, the reward thresholds the live node implies stock already paid minus the crossings that node's ledger rows and pending events own, clamped at 0. Owned is what the walk counts for them (7.8): the distinct intervals named as first payments (a re-payment owns none) plus the positions of each row or event that names none. A pending event is any event that is not a row's own event: no row of its (id, recording) scope has its capture number (or, when either side has none, its UT within 0.1 s, the match the recorder's coalescing folds by). A scope can hold a committed row and a pending event at once, since a recording that keeps its id past an in-flight commit starts a new seed event that commits as its own row; each pending event becomes its own row at commit. So the seed plus the walk's count is the live band at capture, and every named crossing sits above the seed. It carries no rewards: the currency seeds already hold whatever stock paid. It is stored as a child of the ledger file, additive so older builds ignore it:

```
PROGRESS_SEED
{
    captured = True
    node = FirstLaunch
    node = Mun/Landing
    RECORD
    {
        id = RecordsAltitude
        paidCount = 3
    }
}
```

No `PROGRESS_SEED` node means not captured yet (a new save, or one saved by an older build), and the next recalculation captures it; a captured seed may be empty, which on a save that has always run Parsek is the normal case. A ledger load replaces it with the file's content and a test reset clears it; `Ledger.Clear()` keeps it, because its callers restore an action list of the same save. Until it is captured, `PatchMilestones` skips entirely, so nothing is cleared before it can be recorded. How the walk uses it is in 9 (7.7).

4.6 Pre-Parsek save backup

Parsek rewrites `persistent.sfs` (patched funds, science, reputation, crew, tech, contracts, facilities, plus its own `ParsekScenario` node), so the first time it opens an existing save it keeps a copy of that save as it was before Parsek touched it (`PreParsekBackup.cs`). The backup is unconditional; its old setting was removed.

- **When.** On the first cold `ParsekScenario.OnLoad` of a save (the `!initialLoadDone` path), before Parsek's first read or recalculation and before any Parsek write, so the copied `persistent.sfs` carries no Parsek gameplay state. It is skipped when the save already has a Parsek footprint (a `Parsek/` folder or a populated `ParsekScenario` node), when the advisory done-marker `Parsek/pre-parsek-backup.txt` exists, when the save is itself a backup (sentinel file `parsek_backup_source.txt` or the `(pre-Parsek` name fragment), when it has no on-disk `persistent.sfs`, and when it parses as a brand-new empty career (no vessels, science, milestones or active contracts; an unparseable save is backed up).
- **What.** `persistent.sfs`, `persistent.loadmeta` and the `Ships/` and `Subassemblies/` craft folders, plus the sentinel. Quicksaves, `Parsek/` and KSP's own backups are not copied.
- **Where.** A sibling save folder `saves/<Name> (pre-Parsek <yyyy-MM-dd_HHmm>)/`, which KSP lists in its Load menu as an ordinary resumable save. The copy is staged in a `.parsek-backup-staging-<guid>` folder inside the source save (not under `Parsek/`, so a failed copy leaves no false footprint) and published with one directory move; orphaned staging folders are swept on the next cold load.
- **Fail-open, fail-loud.** A failure never stops the load. It logs an Error, shows a one-shot screen message asking the player to back the save up by hand, deletes the staging folder and writes no marker, so the next cold load retries. A success writes the marker and shows a screen message naming the backup. After publishing, the published folder is re-read and the capture line reports whether it is gameplay-pristine (an Error if not, a Warn if it could not be checked; the folder is kept either way).

---

5. KSP State Patching & Warp Model

After every recalculation, the ledger's derived state may differ from what KSP's internal systems believe is true. Parsek must patch KSP's state to match. This section describes what gets patched, when, and how the warp cycle works.

15.1 What gets patched

Each module has a corresponding KSP internal state that must agree with the ledger's derived values at the current UT:

Module	KSP internal state	What Parsek patches
Science	`ResearchAndDevelopment.Instance`	Per-subject collected totals, total science balance, tech tree unlock state
Funds	`Funding.Instance`	Fund balance
Reputation	`Reputation.Instance`	Reputation balance
Milestones	`ProgressTracking.Instance`	Set of achieved milestones
Kerbals	`HighLogic.CurrentGame.CrewRoster`	Roster entries (active, reserved, retired, stand-ins), XP per kerbal, alive/dead/MIA status
Facilities	`ScenarioUpgradeableFacilities`	Building levels, destroyed/intact state
Resource scans (no module)	`ResourceMap` / `ResourceScenario`	Scanned bodies and biomes, add-only

Implementation note: resource scans (owner ruling 2026-10-06). A resource scan is committed career progress. Harmony prefix / postfix pairs on `ResourceMap.UnlockPlanet` (orbital survey) and `ResourceMap.UnlockBiome` (surface scanner) capture each locked-to-unlocked transition as a `ResourceScanCompleted` event (key = body name, `biome=` detail for a biome), tagged and forwarded like any other career event, which converts to a non-economic `ResourceScanUnlocked` ledger action (`scanBody` , `scanBiome`). `KspStatePatcher.PatchResourceScans` unlocks every effective row at or before the part-purchase cutoff (the supplied cutoff clamped to the live clock), add-only: stock has no re-lock, a future scan must not arrive early, and a rewind needs no re-lock because its quicksave predates every later scan. A Re-Fly supersede retires a superseded flight's scan row; it never strict-blocks. Todo RESOURCE-SCAN-UNLOCKS-NOT-LEDGERED.

Implementation note: science lab payouts. The Mobile Processing Lab pays its stored science out through a bare `ResearchAndDevelopment.AddScience` with no science subject (decompiled KSP 1.12.5: `ModuleScienceLab.OnTransmissionComplete` under `ScienceTransmission` , `ModuleScienceLab.RecoverScienceLabs` under `VesselRecovery` ; triggered data never opens an `RnDCommsStream` , so no `OnScienceRecieved` fires). Harmony prefix / postfix pairs on those two methods read the R&D pool and the gross payout stock is about to pass (the lab's `storedScience` , or the recovered labs' summed `storedScience`) and, when the call moved the pool, emit a `ScienceLabCredited` event (key = the stock reason, value before = the pool, value after = the pool plus the gross), tagged and forwarded like any other career event, which converts to a `ScienceLabEarning` ledger action (`scienceAwarded` , `method`). `ScienceModule` credits it uncapped (there is no subject; the lab's own `scienceCap` already bounded the payout), so a rewind past the transmission re-applies it once the clock gets there again. The paired stock `ScienceChanged` stays dropped and remains the earnings reconcile's store side, which the new row now matches. The credit is the gross, like a `ScienceEarning` 's subject value: a converter strategy (Patents Licensing) takes its share inside `AddScience` and the ledger records that take as its own `StrategyScienceDebit` , so a net credit would charge it twice. A Re-Fly supersede retires a superseded flight's lab row; it never strict-blocks. A non-rewind discard re-homes a discarded flight's lab credit as an untagged row (the pool keeps it), like a completed contract. Todo LAB-SCIENCE-NEVER-LEDGERED. A tree with a `ScienceLabCredited` event tagged to any of its recordings is never idle on pad, so the scene-exit idle auto-discard keeps it, like a converter toggle (`ParsekFlight.TreeHasLabPayoutsForIdle`); a flight Recover pays its lab only at the Space Center, after the idle checks and untagged, so the recovery request carries the lab science read from the live vessel and keeps the tree too (`ParsekFlight.TreeHasPendingLabRecoveryForIdle` ; owner ruling 2026-10-17, todo IDLE-ON-PAD-DISCARDS-LAB-PAYOUT-FLIGHT).

Implementation note: the orbital survey bonus. `ModuleOrbitalSurveyor.CompleteSurvey` is the one other stock payout with no science subject (decompiled KSP 1.12.5: `AddScience(SciBonus * mainBody.scienceValues.RecoveryValue, ScienceTransmission)` , then the body's resource map unlock). It rides the lab channel: a prefix / postfix pair reads the pool and that bonus and emits a `ScienceLabCredited` event whose detail starts with `survey=<body>` , which converts to a `ScienceLabEarning` row with the body in `ScanBodyName` (shown as "Orbital survey of "; a lab row has none, so its key and its save shape are unchanged). It is tagged to the live flight like a lab transmission: the survey runs on a loaded vessel the flight or its EVA kerbal works. Before this the pool check kept the bonus as an untagged adjustment dated to the check. Todo ORBITAL-SURVEY-SCIENCE-NEVER-LEDGERED.

The Science Archive in the R&D building reads from `ResearchAndDevelopment.Instance` . Patching the per-subject collected totals ensures the Science Archive progress bars reflect the recalculated timeline, not KSP's stale state.

Implementation status (v0.6): All resource types fully patched — Science (balance + per-subject collected totals), Funds, Reputation, Facilities (level + destruction/repair visual state), Milestones (`ProgressTracking`), and Contracts (`ContractSystem` with ConfigNode snapshot restoration). Kerbals are patched via `KerbalsModule.ApplyToRoster()` (called by orchestrator after the engine walk, separate from `KspStatePatcher`).

Critical: use `SetReputation` , NOT `AddReputation` . KSP's `AddReputation` applies a nonlinear gain/loss curve. If the recalculation engine already computed the final reputation value through the curve simulation, calling `AddReputation` with a correction delta

would apply the curve TWICE, causing significant distortion. `SetReputation` directly assigns the value. This was discovered via decompilation (see spike findings).

15.2 Two-phase pattern

Every timeline change follows the same two phases:

```
Phase 1: RECALCULATE
  Walk ledger from UT=0 to current UT.
  Compute all derived values. Pure computation — no KSP mutation.
  Output: correct state per module at current UT.

Phase 2: PATCH
  For each module:
    Read derived state at current UT.
    Write into KSP's corresponding internal state.
  KSP now agrees with the ledger.
```

The recalculation is pure — it reads the ledger and produces derived state. The patching is impure — it mutates KSP's runtime objects. Keeping these separate means the recalculation logic can be tested without KSP running, and the patching is a thin translation layer.

15.3 Warp cycle

Status: PARTIALLY IMPLEMENTED. The warp exit recalculation trigger IS wired, and the facility-visual portion of the model IS wired. On warp start `ParsekFlight.OnTimeWarpRateChanged` calls `KspStatePatcher.PatchFacilities` so buildings render their correct (derived) level/destruction state during warp; on warp exit it runs the full recalculation and uses `LedgerOrchestrator.HasFacilityActionsInRange(warpStartUT, warpEndUT)` (true when any `FacilityUpgrade`, `FacilityDestruction`, or `FacilityRepair` action falls in the crossed range) to log facility crossings. What is NOT yet implemented: true real-time per-frame facility visual updates during warp (the current approach only patches to the final derived state at warp start and again at warp exit, not per crossed UT), the dedicated during-warp game-action event queue, and the warp-exit player-feedback summary described below. Those remain design-only.

During time warp (fast-forward), the player is not in control. KSP state mutation is deferred until the player exits warp. The warp cycle follows the same collect-during-warp, process-on-exit pattern used for vessel spawning.

```
ENTER WARP:
  KSP state is already correct (patched after last recalculation).
  Start visual playback. No recalculation needed.

DURING WARP:
  Visual-only updates (no KSP state mutation):
    - Ghost vessels replay from sidecar trajectory data.
    - Facility visuals update at the correct UT (building destroyed,
      repaired, upgraded — rendered in real time like ghosts).
  Event queue collects:
    - Game action UTs crossed during warp.
    - Vessel spawn/despawn events (existing recording system).
  No recalculation. No KSP state patching.

EXIT WARP:
  Full recalculation from UT=0 to current UT.
  Patch all KSP state (R&D, funds, rep, roster, facilities, milestones).
  Process vessel spawn queue (spawn/despawn real vessels).
  Player takes control with correct state.
```

Key properties of this model:

One recalculation per warp session. The full walk from UT=0 runs once on exit, not per event crossing. This is a significant performance benefit — warping through 50 science events runs one recalculation, not 50.

Visuals are real-time, state is deferred. Ghost vessels and building visual changes render at the correct UT during warp, exactly as the player would expect. But KSP's internal state (fund balance, science totals, roster) only updates on exit. The player never interacts with KSP state during warp, so the deferral is invisible.

Two parallel queues. The vessel spawn queue and game action queue follow the same pattern — collect during warp, process on exit. The game action queue primarily serves as a trigger for recalculation and a source for the player feedback summary on exit ("During warp: 3 recordings completed, 45 science earned, First Mun Landing achieved, Jeb recovered").

15.4 Patching after rewind

Rewind is a special case. KSP loads a quicksave, which restores KSP state to the moment the quicksave was taken. But the timeline may have recordings committed after that quicksave was created. The quicksave doesn't know about those recordings.

```
REWIND:
  1. KSP loads quicksave (KSP state = quicksave state).
  2. Parsek recalculates full timeline from UT=0 to rewind UT.
  3. Parsek patches KSP state to match ledger's derived state.
  Player takes control with state reflecting all committed recordings.
```

The quicksave is the starting point, but Parsek overrides it with the ledger's truth. This is why per-subject science totals, fund balances, and kerbal roster may differ from what the quicksave contained — the ledger accounts for recordings that were committed after the quicksave was taken.

15.5 Patching after KSP load

KSP load is authoritative — the loaded save defines truth. Parsek reconciles the ledger (prunes entries not in the save), then recalculates and patches:

```
KSP LOAD:
  1. KSP loads save (KSP state = save state).
  2. Parsek reconciles ledger (prune orphaned earnings plus future spendings and contract lifecycle rows).
  3. Parsek recalculates from UT=0 to loaded UT.
  4. Parsek patches KSP state.
```

In most cases after a KSP load, the patched state will match what's already in the save. The recalculation is a verification step — it ensures consistency even if the save was edited or corrupted.

AS BUILT (2026-10-06): step 2 runs only on a COLD load (`LedgerOrchestrator.OnKspLoad` from the cold branch of `ParsekScenario.OnLoad`), and it prunes rows tagged to recordings absent from the save but does NOT prune future spendings and contract lifecycle rows in FLIGHT, at the Space Center or in the Tracking Station: `Ledger.Reconcile` is called with `preserveFutureTimelineActions` there (and whenever the clock is not ready yet), so committed future rows survive and the current-UT cutoff filters their effects until the clock reaches them, which is the committed-is-permanent rule (owner ruling D2, `docs/dev/research/stock-ui-reservation-overlays-2026-09-25.md`). Since 0.10.7 the VAB and SPH read the same scene predicate (todo EDITOR-LOAD-WALKS-PAST-THE-CLOCK: the in-session load, the cold-load reconcile and the deferred seed), so they keep those rows and take the cutoff too; only a stock revert to the editor still walks without one. The Tracking Station was aligned with FLIGHT and the Space Center on 2026-10-15 (owner ruling 2026-10-06, todo TRACKING-STATION-LEDGER-CUTOFF-ALIGN): its cold and in-session loads take the cutoff, and like the Space Center it applies a committed action when its clock reaches it during warp (`CareerLedgerClockCursor`). Game rule behind the editor change: no scene shows money the committed timeline earns later (a contract advance, a reward) as available now, so entering the VAB or SPH after a rewind or a Re-Fly merge cannot make a committed future earning spendable. An in-session load (a scene change) does not re-run this reconcile at all; a load of a save file (F9, the pause-menu or main-menu Load) is a cold load and does, at the save's own clock (below); stock Revert additionally prunes the reverted flight's untagged launch-pad rows (`Ledger.PruneOrphanActionsAfterUT`). A player's load of an earlier save (F9, the Load menu) undoes the untagged rows made after the save (owner ruling 2026-10-19, "a quickload also undoes Space Center purchases made after the quicksave", reversing the earlier QL-R2 keep for loads) because it is a cold save-file load that reads the save's own ledger, which never held them (`docs/dev/design-load-restores-the-save.md`). Phase 3 of that design deleted the in-session retire that used to do this (`RetireUntaggedRowsAfterSaveAtInSessionLoad` and the `UNTAGGED_FUTURE_ROWS` id list `OnSave` wrote; an old save still carrying the node loads with it ignored), so an in-session load no stock path announces (the `saveStamp` canary warns on one) keeps every untagged row. Route events are not ledger rows (0.10.6); the in-session route reconcile retires the route event log's events after the loaded save (always on an F9 in flight, on evidence otherwise). Rewinds, the Re-Fly loads and a cold load keep ruling D2 here. A quickload in flight that resumes a flight used to retire that flight's own abandoned future when the recorder resumed it (owner ruling OQ-2, 2026-10-06), and the committed-copy restore rule (`DecideCommittedCopyRestore`) reached the same flight from the Space Center; Phase 3 of the load design deleted both, since a save-file load reads that future out of the save. What stays is the trajectory trim (`ParsekScenario.TrimForQuickloadResume`), which leaves committed history whole: neither a recording still committed in memory nor one the quicksave itself already shows as committed history (`ParsekScenario.IsCommittedHistoryAtQuicksave` , see `QUICKLOAD-TRIM-CUTS-COMMITTED-HISTORY-OF-A-RESUMED-CLONE` ; a member that had ended at the quicksave and was restamped after it, say recovered, is not history,

`ParsekScenario.IsEndRestampedAfterQuicksave`). The remaining F9 consequences are tracked in `docs/dev/todo-and-known-bugs.md` (the QUICKLOAD-* entries). **Save-file loads are cold (owner ruling 2026-10-19, `docs/dev/design-load-restores-the-save.md` 4.1):** every save carries its own ledger, events, milestones, baselines and pending science in a `GAME_STATE` node of the Parsek scenario node, and a load of a save file stock announces (F9, the pause-menu Load, the main-menu Load) that is not a revert classifies `Cold` (`coldReason=save-file`) and reads them from there. Its reconcile and recalculation run at the save's own `flightState` UT: inside a Space Center or Tracking Station OnLoad the Planetarium still reads the clock from before the load (stock `Game.Load` runs the scenario modules before `FlightState.Load` sets the clock), so `ParsekScenario.ResolveColdLoadClockUT` takes the save's UT whenever the Planetarium reading is ready, and keeps the not-ready reading (the boot load) so the rules above for a clock that is not ready still hold. The in-session retires that used to stand above (the untagged-row retire, the resume reconcile, the committed-copy rule) were not reached by such a load, and QL-R1 (a discarded flight records again) and QL-R2 (Space Center purchases after the save kept) are reversed for it: the save's ledger is the ledger of its moment, and a flight in progress at the save resumes with its recording. Rewinds and Re-Fly loads keep D2. The in-session machinery stays for scene changes and stock reverts; Phase 3 of the load plan deleted what no load kind reaches.

15.6 Ledger ground-truth verification and apply-boundary tracing

Two shipped subsystems verify and observe the patch pipeline described above.

Ledger ground-truth harness (non-circular). `LedgerOrchestrator.RecalculateAndPatch` derives KSP state from the ledger; the ground-truth harness checks that derivation against KSP's own on-disk save rather than against live state after patching (which would be a tautology). Layer A is pure and headless-testable: `CareerSaveParser.Parse(ConfigNode)` independently parses a career `.sfs` GAME node (funds, science pool + per-subject, reputation, facility level fractions, contract guids, qualified milestone ids, per-vessel resource totals) into a `CareerSaveSnapshot`, and `LedgerGroundTruthDiff.Compare` diffs a `LedgerReconstructionSnapshot` against it per facet, producing a `LedgerDivergenceReport` (unit-tested in `LedgerGroundTruth{Parser,Diff}Tests.cs`). Layer B is the in-game `LedgerGroundTruthHarness` test (category `LedgerGroundTruth`, career + FLIGHT only): it quicksaves the live career, parses it via `CareerSaveParser`, runs a recalculation, and diffs the reconstruction against the parsed save. The seeded pools (funds/science/reputation) and guid-corroborated vessel-recovery consistency are hard-asserted; per-identity facets and phantoms are report-only by default. See `docs/dev/design-ledger-groundtruth-harness.md`.

Ledger apply-boundary tracer (`LedgerTrace`). Gated, event-driven observability for the code that rewrites career scalars every recalc (off by default behind the `ledgerTracing` setting). Unlike the map render tracer it has no per-frame probe: the read-back reconcile runs synchronously inside each `KspStatePatcher.Patch*`. Three tiers: a per-recalc structural snapshot (Tier A, one grep-stable `phase=Structural funds/science/rep/facilities/...` line emitted from `ApplyRecalculatedStateToKsp`); per-identity change lines (Tier B, keyed `resource|id`); and a read-back `ledger-vs-truth` anomaly (Tier C, `Warn`) via pure predicates comparing the patched-in target against the value actually read back from KSP. A monotonic `recalcSeq` stamps every line so one recalc burst is grep-sliceable.

---

6. Science Module

Status: Designed — earning/spending schemas, recalculation algorithm, reservation system, and verified scenarios complete.

Science is the simplest resource module and the first to be fully designed.

Game mode applicability: Not applicable in sandbox (all tech unlocked, no science tracking). Active in Science mode and Career mode with identical behavior in both.

12.1 KSP's science pipeline

1. Player uses a science instrument at a specific body + situation + biome.
2. KSP generates a `ScienceData` with a subject ID (e.g. `crewReport@MunSrfLandedMidlands`).
3. Player transmits (partial value, scaled by `transmitScalar`) or recovers the vessel (full value).
4. KSP adds science via `ResearchAndDevelopment.Instance.SubmitScienceData()`.
5. Value diminishes on repeat — the diminishing returns formula is roughly `nextValue = baseValue × subjectMultiplier × (1 − subjectScience / scienceCap)`.

12.2 Single subject cap

There is one `scienceCap` per subject. Both transmit and recover draw from the same pool. Transmitting depletes the subject by the awarded amount (post-scalar). Recovery depletes by the full awarded amount. There are no separate transmit and recovery budgets — all methods eat from the same cap.

The `scienceAwarded` value already has the transmit scalar baked in. It is the post-method number. No raw pre-transmit value needs to be tracked.

12.3 Science earning action schema

```
ScienceEarning
  ut:                double — when it happened
  recordingId:       string — which recording earned this
  subjectId:         string — full KSP subject string (e.g. "crewReport@MunSrfLandedMidlands")
  experimentId:      string — the experiment type (e.g. "crewReport")
  body:              string — e.g. "Mun"
  situation:         string — e.g. "SrfLanded"
  biome:             string — e.g. "Midlands"
  scienceAwarded:    float — what KSP actually credited (IMMUTABLE, never changes)
  method:            enum — TRANSMITTED | RECOVERED
  transmitScalar:    float — the experiment's transmission efficiency (0.0 to 1.0)
  subjectMaxValue:   float — total science this subject can yield (scienceCap)
```

`scienceAwarded` is immutable. Written once at recording time. What KSP reported. Never changes. Captured by `GameStateRecorder` during flight, converted to ledger entries on commit.

`effectiveScience` is derived. Always computed, never stored. What the ledger actually credits to the budget after applying subject caps against other recordings on the timeline. Recalculated from scratch every time the timeline changes.

Built (section 3.15 step 2): the row's `ut` is the capture UT, as a double. `GameStateEventConverter` dates each science earning at the moment stock paid it, so a transmission sorts by when it happened, not by when its recording ended. `startUT` keeps the capture moment and `endUT` the recording end. Two cases stay at the recording end: a subject with no capture UT, and a capture after the recording's trajectory window (the end bound is the row's owner). Rows an older build filed at the end UT are not migrated; their capture survives only in the float `startUT`, and a tie with them is broken on `cseq` (3.7).

`method` and `transmitScalar` serve the UI (showing the player "you transmitted this at 30% efficiency") but do not drive the recalculation math.

12.4 Science spending action schema

```
ScienceSpending
  ut:          double — frozen KSC time
  sequence:    int — order of unlock within that UT (0, 1, 2...)
  nodeId:      string — tech tree node ID (e.g. "survivability")
  cost:        float — science points spent
```

12.5 Science reservation system

Science uses the same cashflow reservation system as funds (section 7.6). Committed science spendings (tech node unlocks) — past, present, and future — constrain the amount that is spendable at the current UT. The player can only spend an amount that leaves the science balance non-negative from the current UT through the future committed ledger.

Available science at any UT:

```
currentScience(ut) = science balance after replaying actions with action.UT <= ut
projectedScience starts at currentScience(ut)
for each future science earning/spending in chronological order:
  projectedScience += futureScienceDelta

availableScience(ut) = max(0, min(currentScience(ut), every future projectedScience))
```

Note: this uses `effectiveScience` (post-cap derived values), not raw `scienceAwarded`. The recalculation must run an isolated full-timeline shadow walk first to compute future effective values, then the cutoff walk patches the current visible state and installs the projected available value.

Why this is needed: without reservation, a player who rewinds to a UT where only some earnings have accumulated could see a science balance that appears sufficient to unlock a new tech node, but existing future spendings may already need that current headroom.

```
Example without reservation:
  Recording A earns 30 at UT=500. Recording B earns 20 at UT=1000.
  Tech node at UT=1500 costs 45. Balance after walk: 5.

  Player rewinds, fast-forwards to UT=600. Running balance: 30 (only A's earnings).
  Player sees 30 science, unlocks 25-cost node. Allowed.
  Walk: UT=500 +30, UT=600 -25, UT=1000 +20, UT=1500 -45 → -20. DEFICIT.

With reservation:
  Current at UT=600: 30.
  Project future: UT=1000 +20 -> 50, UT=1500 -45 -> 5.
  Minimum from now forward is 5, so availableScience(UT=600)=5.
  A 25-cost node is blocked.

Future earnings can cover future spendings before they happen:
  Current at UT=600: 30.
  Future earning at UT=1000: +40, future tech at UT=1500: -45.
  Projected balances: 30 -> 70 -> 25.
  Minimum from now forward is 25, so availableScience(UT=600)=25.
  Future earnings did not raise available above the current 30, but they did reduce the reservation needed for the fut
ure tech node.
```

At KSC spending time: the ledger checks `availableScience(currentUT) >= cost` before allowing a new tech node unlock. If insufficient, the unlock is blocked.

UI patching: Parsek patches KSP's R&D science balance to show `availableScience(currentUT)` clamped to 0, not the raw running balance. The player sees what they can actually spend right now. Parsek's own UI can show the full breakdown (current balance, projected reservation, available).

12.6 Science recalculation

The timeline can have multiple recordings that collected science from the same subject. Each recording's `scienceAwarded` was computed by KSP in isolation (the player may have rewound to a fresh save state between recordings). The sum of `scienceAwarded` across recordings can exceed the subject's maximum. The recalculation prevents this.

```
RecalculateScienceTimeline():
  1. Reset every subject's creditedTotal to 0.
  2. Reset runningScience to 0.
  3. Collect ALL science actions (earnings + spendings) from ALL committed recordings.
  4. Sort by (UT, type [earning before spending], sequence).
  5. Walk forward:

     For each action in sort order:
       if EARNING:
         subject = SubjectLedger[action.subjectId]
         headroom = subject.maxValue - subject.creditedTotal
         action.effectiveScience = min(action.scienceAwarded, headroom)
         subject.creditedTotal += action.effectiveScience
         runningScience += action.effectiveScience

       if SPENDING:
         if the walk already owns the tech (3.15 step 6): not counted
         else:
           runningScience -= action.cost   (below 0 is a science debt)
           action.affordable = true        (the tech stays researched)
```

Chronological order on the timeline determines priority. Earlier recordings get full credit. Later recordings get the remainder. If a recording is added to the past (player rewinds and commits a new mission), the recalculation gives it priority and reduces later recordings' effective values.

The `affordable` flag on a spending action means "researched by the walk": it is true on every row the walk charged, in full or into debt, and on a row it did not charge because it already owns the tech.

Built (section 3.15 step 6, owner ruling 2026-10-26). A committed tech the walk can no longer pay is charged into a science debt like a funds row (section 3.13 part 4): it stays researched, the running science goes below 0, the write is floored at 0, and later earnings repay the debt before the pool rises. Until step 6 such a tech was not deducted and `KspStatePatcher.ShouldRefuseUnaffordableRelock` kept it researched, so it cost nothing; that guard stays as a backstop and a walked row no longer arms it. Paths that leave a committed tech short after a rewind: credit moving between two flights of one

subject now that rows are dated at capture time (step 2), a strategy's science cut moving out of its window, and take-back after a spend when auto-merge is off, a test setting (step 5). A tech the walk already owns at its (UT, `cseq`), bought by a purchase captured after it, is not charged again (`PurchaseAlreadyOwned`).

12.7 Verified scenarios

Basic earning and retroactive priority:

```
Subject: crewReport@MunSrfLandedMidlands, scienceCap = 15

Step 1: Player flies T1 at UT=1000, earns 10. Commits.
  Recalculate: T1 effective = 10. Total = 10.

Step 2: Player rewinds to UT=500. Flies T0 at UT=700, earns 10. Commits.
  Recalculate (sorted by UT):
    T0 at UT=700:  headroom=15, effective=10, credited=10
    T1 at UT=1000: headroom=5,  effective=5,  credited=15
  Total = 15.

Committing T0 retroactively reduced T1's effective from 10 to 5.
Total correctly capped at subject maximum of 15.
```

Mixed transmit/recover:

```
Subject scienceCap = 10

Step 1: Fly at UT=1000, transmit. KSP awards 3. Commit.
  UT=1000: headroom=10, effective=3, credited=3. Total=3.

Step 2: Rewind to UT=300, fly at UT=500, transmit. KSP awards 3. Commit.
  UT=500:  headroom=10, effective=3, credited=3
  UT=1000: headroom=7,  effective=3, credited=6. Total=6.

Step 3: Rewind to UT=100, fly at UT=200, recover. KSP awards 10. Commit.
  UT=200:  headroom=10, effective=10, credited=10
  UT=500:  headroom=0,  effective=0,  credited=10
  UT=1000: headroom=0,  effective=0,  credited=10. Total=10.

Recovery at UT=200 consumes the entire cap.
Both transmissions remain on the timeline (scienceAwarded is immutable)
but their effectiveScience drops to 0. Correct.
```

Earnings with interleaved spending:

```
Subject scienceCap = 10 for subjectA. Tech node costs 8.

Step 1: Fly at UT=1000, recover subjectA, earn 10. Commit. Balance=10.
Step 2: Unlock tech node at UT=1500, cost 8. Balance=2.
Step 3: Rewind to UT=100, fly at UT=200, recover same subjectA. Commit.
  Walk:
    UT=200:  effective=10, balance=10
    UT=1000: effective=0 (cap hit), balance=10
    UT=1500: affordable (10 >= 8), balance=2.
  Earnings shifted between recordings but total held. Spending still valid.
```

Reservation blocks new spending on rewind:

```
Recording A earns 30 science at UT=500. Recording B earns 20 at UT=1000.
Tech node at UT=1500 costs 45. Balance after walk: 5.

Player rewinds, fast-forwards to UT=600.
  Current science at UT=600: 30 (recording A only).
  Future projection: UT=1000 +20 -> 50, UT=1500 -45 -> 5.
  Available: min(30, 50, 5) = 5.
  Player cannot unlock a node costing more than 5. Correct.

Player fast-forwards to UT=1100.
  Current science at UT=1100: 50 (both recordings).
```

```
  Future projection: UT=1500 -45 -> 5.
  Available: min(50, 5) = 5.
  Player can unlock a node costing up to 5. Correct.
```

If the future earning happens before the future spending, it preserves current headroom:

```
Current UT=600. Current science: 100.
UT=1000: Future science earning +80.
UT=1500: Future tech unlock -80.

Projected balances: 100 -> 180 -> 100.
Available science at UT=600: 100.
```

12.8 Edge cases

Multiple collections within one recording. A player transmits an experiment during a mission, then recovers the vessel with the same experiment still onboard (some instruments are rerunnable, or they ran the experiment again after transmitting). KSP treats these as separate `SubmitScienceData` calls with separate awards. The ledger captures them as two `ScienceEarning` rows with the same `subjectId` and `recordingId` but different UTs. The hard-cap walk handles this naturally — they're just two entries in the sort order.

Hard cap vs KSP's diminishing curve. The recalculation uses a hard cap (`scienceCap`) rather than simulating KSP's asymptotic diminishing returns curve. This is conservative — it may sometimes credit slightly more than KSP's curve would in a sequential playthrough, but never more than `scienceCap`. Simulating the exact curve would require replacing the clean hard-cap walk with curve simulation across recordings, adding significant complexity for marginal accuracy. The hard cap is correct in the way that matters: no overcredit, no paradox.

Research and Development building. On rewind or fast-forward, Parsek patches the R&D state to the current UT. Tech nodes whose `ScienceSpending` action is after the current UT are locked again in the player-facing research tree, even though their future spending remains on the committed ledger for projection. Only technology available at or before the current UT should appear unlocked. The Science Archive tab also reflects the recalculated timeline: Parsek patches per-subject collected totals via `KspStatePatcher.PatchPerSubjectScience`, so progress bars correctly account for retroactive recordings.

---

7. Funds Module

Status: Designed — seeded balance, reservation system, earning/spending schemas, and verified scenarios complete.

Game mode applicability: Not applicable in sandbox (unlimited funds) or Science mode (no funds tracking). Active in Career mode only.

5.1 KSP's funds system

Funds are the monetary currency of Career mode. They are a simple linear balance — no diminishing returns curve, no per-subject caps. +50,000 funds is always +50,000, regardless of current balance.

Career mode starts with an initial fund balance (default 25,000 at Normal difficulty, configurable). This is the seed value for the ledger.

5.2 Seeded balance

Unlike science (which starts at 0), funds start at a non-zero value determined by the career difficulty settings. The ledger must be seeded with the actual starting funds from the save file — this value varies by difficulty preset (Easy, Normal, Moderate, Hard) and can be customized by the player or mods.

```
FundsInitial (seed — not a player action)
  initialFunds:   float — extracted from the save file at career creation
```

The seed is extracted once when Parsek first initializes on a career save. It is immutable. Parsek must not assume a default value — it reads whatever the save file reports as the career starting funds.

5.3 Earning sources

Fund earnings come from multiple first-tier modules:

- **Milestones**: `fundsAwarded` from effective milestone achievements (first-tier, flows into funds walk).
- **Contracts**: completion rewards, advance payments on acceptance.
- **Vessel recovery**: percentage of vessel part costs returned based on distance from KSC. KSP calculates the recovery value; the ledger stores it as immutable.
- **Other**: miscellaneous KSP events that award funds.

All earning values are immutable — what KSP reported at recording time. No curve or diminishing returns.

5.4 Spending sinks

- **Vessel construction**: part costs deducted at launch. Recording-associated — tied to a `recordingId`.
- **Facility upgrades**: KSC building level purchases. KSC spending action.
- **Facility repairs**: repair cost after building destruction. KSC spending action.
- **Kerbal hiring**: crew recruitment costs. KSC spending action.
- **Contract penalties**: funds deducted on contract failure or cancellation.
- **Strategy costs**: if applicable (deferred to Strategies module).

5.5 Fund action schemas

```
FundsEarning (recording-associated or first-tier-derived)
  ut:             double
  recordingId:    string or NULL — NULL if derived from milestone/contract action
  fundsAwarded:   float — amount earned (IMMUTABLE)
  source:         enum — CONTRACT_COMPLETE | CONTRACT_ADVANCE | RECOVERY |
                         MILESTONE | OTHER

FundsSpending (recording-associated or KSC spending action)
  ut:             double
  sequence:       int — order within same UT (for KSC spendings)
  recordingId:    string or NULL — non-NULL for vessel builds, NULL for KSC spendings
  fundsSpent:     float — amount spent (IMMUTABLE)
  source:         enum — VESSEL_BUILD | FACILITY_UPGRADE | FACILITY_REPAIR |
                         KERBAL_HIRE | CONTRACT_PENALTY | STRATEGY | OTHER

FundsInitial (seed — created once at career start)
  ut:             0 (always)
  initialFunds:   float — extracted from the save file at career creation

ScienceInitial (seed — handles mid-career Parsek install)
  ut:             0 (always)
  initialScience: float — extracted from ResearchAndDevelopment.Instance.Science

ReputationInitial (seed — handles mid-career Parsek install)
  ut:             0 (always)
  initialRep:     float — extracted from Reputation.Instance.reputation
```

15.6 Reservation system

Funds use a cashflow reservation system to prevent overspending when the player rewinds to an earlier UT. Committed spendings on the timeline — past, present, and future — constrain the amount spendable at the current UT. The player can only spend an amount that leaves the fund balance non-negative from the current UT through the future committed ledger.

Available funds at any UT:

```
currentFunds(ut) = fund balance after replaying actions with action.UT <= ut
projectedFunds starts at currentFunds(ut)
for each future fund earning/spending in chronological order:
  projectedFunds += futureFundsDelta

availableFunds(ut) = max(0, min(currentFunds(ut), every future projectedFunds))
```

The key insight: future committed spendings reserve current funds only to the extent the future cashflow would otherwise dip below the current balance. Future earnings cannot increase what the player can spend now above `currentFunds(ut)`, but future earnings that occur before future spendings can cover those spendings without reserving current funds.

At commit time: the ledger checks whether the recording's vessel build cost fits within the available budget at launch UT. If `availableFunds(launchUT) >= vesselCost`, the commit is allowed. Otherwise it is blocked — the player must earn more funds, fast-forward to a later UT where more earnings are available, or revert and use a cheaper vessel.

At KSC spending time: the ledger checks `availableFunds(currentUT) >= cost` before allowing facility upgrades, kerbal hires, or other KSC purchases.

Owner rulings 2026-10-21 (section 3.13, built in 0.10.6): the reservation also holds every open contract's funds fail penalty from its accept to its outcome while a committed cost lies ahead; the KSC check above is enforced at the stock control (greyed with the reason, backstop refusal), launch costs included, instead of advisory; and a funds loss nobody could reserve stops at the reserved floor, with the rest carried as a debt. (A KSC supply-route dispatch funds gate built earlier that day was removed in 0.10.6 with all route money, todo LOGISTICS-WITHOUT-MONEY.)

15.7 Why available can appear low or zero at early UTs

After committing multiple recordings, the future cashflow may need some or all of the funds available at an early UT. This is not a bug: each spending was individually valid when committed. But viewed from an earlier UT, a future spending before the future earnings that cover it can reserve current headroom.

```
Example:
  UT=0:    Seed 25k.
  UT=100:  Earn 50k (milestone). Balance: 75k.
  UT=200:  Spend 30k (vessel). Balance: 45k.
  UT=500:  Earn 40k (contract). Balance: 85k.
  UT=600:  Spend 60k (facility). Balance: 25k.

  Player rewinds to UT=0.
    Projected balances: 25k -> 75k -> 45k -> 85k -> 25k.
    Available at UT=0: 25k.

  Player fast-forwards to UT=150 (after milestone).
    Projected balances: 75k -> 45k -> 85k -> 25k.
    Available at UT=150: 25k.

  Player fast-forwards to UT=550 (after contract).
    Projected balances: 85k -> 25k.
    Available at UT=550: 25k.

  If the facility spend at UT=600 were 100k instead:
    Projected balances from UT=0: 25k -> 75k -> 45k -> 85k -> -15k.
    Available at UT=0: 0; the timeline is overcommitted and new spendings are blocked.
```

The available value is clamped to 0 in the KSP UI. The player sees "no funds available" rather than a negative number. Parsek's detail panel can show the full breakdown (balance, reserved, available) for players who want to understand why.

15.8 Funds recalculation

Funds is a second-tier module. It runs after first-tier modules (Milestones, Contracts) have set their effective flags.

```
RecalculateFunds():
  runningBalance = initialFunds

  For each fund-affecting action in UT order:
    if EARNING:
      if source == MILESTONE:
        check milestone module's effective flag
        if effective: runningBalance += fundsAwarded
      else:
        runningBalance += fundsAwarded

    if SPENDING:
      runningBalance -= fundsSpent
      action.affordable = (runningBalance >= 0)  // defensive check
```

```
    // Available at a cutoff UT is derived by replaying future fund deltas from
    // runningBalance(ut) and taking the minimum projected balance.
```

The `affordable` flag is defensive. With proper seeding and the reservation system preventing overspending, it should always be true. If it's false, it indicates a bug or data corruption.

Implementation handles more action types: Beyond `FundsEarning` and `FundsSpending`, the walk also processes: `ContractAccept` (advance payment), `ContractComplete` (rewards, using TransformedFundsReward, see section 13.4), `ContractFail` / `ContractCancel` (penalties), `FacilityUpgrade` / `FacilityRepair` (costs via FacilityCost field), `KerbalHire` (hiring cost), and `StrategyActivate` (setup cost). All spending-type actions are included in the reservation total.

15.9 UI patching

On warp exit / rewind, Parsek patches KSP's fund display:

- `Funding.Instance.Funds` is set to `availableFunds(currentUT)`, not `runningBalance(currentUT)`. The player sees what they can spend now, not the gross balance.
- Parsek's own UI can show the breakdown: current balance, projected reservation, and available amount.

This means the KSP toolbar funds display always reflects what the player can actually spend on a new vessel or facility. No surprises.

15.10 Vessel build cost — when and how

KSP deducts vessel cost at launch. The recording captures this:

- The vessel cost is captured during flight (at launch time).
- On commit, the cost is extracted into the ledger as a `FundsSpending` with `source=VESSEL_BUILD` and the recording's `recordingId`.
- The spending UT is the recording's launch UT.

The player doesn't choose the cost — it's determined by the parts on the vessel. KSP calculates it. The ledger stores it as immutable.

15.11 Vessel recovery — earning on return

When a vessel is recovered, KSP returns a percentage of the vessel's part costs based on distance from KSC (closer = higher percentage). The recovery value is:

- Captured by KSP's `OnVesselRecoveryProcessing` callback.
- Captured during the recording.
- Extracted into the ledger as a `FundsEarning` with `source=RECOVERY` on commit.
- The earning UT is the recovery UT.

The recovery percentage formula is KSP's domain — the ledger stores only the final awarded amount.

Implementation note: which recording carries a recovery, and when (2026-09-27). There are two producers of the `FundsEarning(Recovery)` row, and which one fires depends on whether the flight is still pending when stock recovers the vessel:

- Pending tree (manual merge: the merge dialog waits at the Space Center, so `onVesselRecovered` fires first). `UpdateRecordingsForTerminalEvent` stamps the pending leaf `Recovered`, and `CreateVesselCostActions` pairs the `FundsChanged(VesselRecovery)` event into the row at commit.
- Committed recording (auto-merge on, the shipping setting: an in-flight Recover commits the tree at the scene change with the situation terminal, usually Landed, BEFORE stock recovers the vessel at the Space Center; and every Tracking Station recovery of an older flight). Committed recordings are never re-stamped by a terminal event, so the terminal stays Landed / Orbiting. `onVesselRecovered` writes the row directly (#444, `LedgerOrchestrator.OnVesselRecoveryFunds`, tagged by `PickRecoveryRecordingId`) plus one `KerbalRecovered` row per recovered kerbal (see the 9.3 recovery note) and one `VesselRecovered` row per owning recording (next paragraph).

Implementation note: the `VesselRecovered` row (owner rulings 2026-10-03). The funds row needs stock's `FundsChanged(VesselRecovery)`, which a sandbox save never raises, and the `KerbalRecovered` row needs crew aboard, so an uncrewed sandbox recovery of a vessel Parsek spawned left no trace and the next flight scene's `RunSpawnDeathChecks` respawned it. Every PLAYER recovery (Tracking Station, KSC marker, flight Recover, kRPC) of a vessel that continues a committed recording

therefore writes one non-economic `VesselRecovered` ledger action per owner, in every game mode (`LedgerOrchestrator.OnRealVesselRecovered`, called from `ParsekScenario.OnVesselRecovered`): the owners are `CrewRecoveryReservationClose.SelectOwnerRecordings` over the committed ERS (a POSITIVE launch-guid match or a genuine Parsek spawn pid; one owner per tree, the latest end at or before the recovery), so a recording the pending tree still owns gets none (its own terminal stamp records the recovery). Deduplicated per owner recording. Parsek's own programmatic recoveries run under `SuppressionGuard.Crew()` and write none, and neither do ghost-map vessels or rewind strips. The row moves no pool (not resource-impacting), never strict-blocks a supersede, is retired with its owner by a Re-Fly supersede (`TombstoneEligibility.IsSupersedeTombstoneEligible`), and gets no Timeline entry (it is spawn evidence; on a career save the funds row already shows the recovery). It is read by `RecoveredRecordingEvidence` with no UT filter, so the recording's vessel never spawns again, also after a rewind to before the recovery; and `RunSpawnDeathChecks` treats a vanished spawned vessel whose recording reads recovered as a recovery, clearing the spawn claim without counting a death.

So the ledger, not the terminal state, is the one surface that records every recovery. Anything that asks "was this recording's vessel recovered after UT X" must accept either the `Recovered` terminal or a recovery row after X. The Re-Fly invoke hold (`ReFlyInvokeHold.Classify`, Step 3b of `RewindInvoker`, rewind design section 2.12 Rule 3) reads exactly that evidence: a vessel the Re-Fly puts back in the world whose recording (POSITIVELY the survivor's launch: pid and both launch guids) ended `Recovered` after the rewind cutoff, or carries a recovery row after it on a terminal that left the vessel in the world (not Destroyed / Docked / Boarded / Disassembled), is held back until that recovery, and its recovery rows stay. Before 2026-10-20 the same evidence drove `ResurrectionRetirementEligibility.Classify`, which tombstoned the recovery at invoke (todo REFLY-RESURRECTED-RECOVERY-STAYS-BANKED); that retirement ran outside any merge and outlived a Discard, and is gone. The classifier remains as the reference for that evidence rule and for `VesselOutlivedTerminal`.

15.12 Verified scenarios

Basic career with seeded balance:

```
Seed: 25,000.

UT=10:  Vessel build -5,000.   Balance: 20,000. ✓
UT=50:  Milestone +8,000.      Balance: 28,000.
UT=60:  Recovery +3,000.       Balance: 31,000.

Available at UT=100: 31,000 - 0 future spendings = 31,000.
Player can spend up to 31,000 on next vessel.
```

Reservation uses the minimum projected balance on rewind:

```
Seed: 25,000.

UT=10:  Vessel A -5,000.   Balance: 20,000.
UT=50:  Earn +8,000.       Balance: 28,000.
UT=60:  Earn +3,000.       Balance: 31,000.
UT=100: Vessel B -15,000.  Balance: 16,000.
UT=200: Earn +20,000.      Balance: 36,000.
UT=250: Earn +12,000.      Balance: 48,000.

Player rewinds to UT=50:
  Current funds at UT=50: 28,000.
  Future projection: -15,000 -> 13,000, +20,000 -> 33,000, +12,000 -> 45,000.
  Available: 13,000.
  Can build 10k vessel? Yes.
  Can build 20k vessel? No.
```

Facility upgrade and kerbal hire interleaved:

```
Seed: 50,000.

UT=50:  Vessel A -20,000.     Balance: 30,000.
UT=100: Earn +30,000.         Balance: 60,000.
UT=200: Vessel B -15,000.     Balance: 45,000.
UT=300: Earn +25,000.         Balance: 70,000.
UT=400: Vessel C -30,000.     Balance: 40,000.
UT=500: Earn +40,000.         Balance: 80,000.
UT=600: Hire kerbal -25,000.  Balance: 55,000.
UT=700: Facility -35,000.     Balance: 20,000.
```

```
Walk: all balances non-negative ✓.
Available at UT=0: min(50, 30, 60, 45, 70, 40, 80, 55, 20) = 20,000.
Available at UT=500: min(80, 55, 20) = 20,000.
```

New recording adds earnings, expanding budget:

```
Seed: 25,000. Existing timeline:
  UT=100: Earn +100,000. UT=500: Vessel -40,000. UT=1500: Facility -50,000.
  Current funds at UT=200: 125,000. Projected minimum: 35,000. Available at UT=200: 35,000.

Player rewinds to UT=200, commits rec_B (earns 30k contract, vessel costs 15k):
  New current funds at UT=300: 140,000.
  Future projection: old vessel -40,000 -> 100,000, facility -50,000 -> 50,000.
  Available at UT=300: 50,000.
  Net effect: +15,000 available (earned 30k, spent 15k).
```

15.13 No-delete invariant for funds

The no-delete invariant provides the same structural guarantee for funds as for other spendable resources:

- **Earnings are monotonically growing.** Adding a recording can only add earnings (and its own vessel cost). Existing earnings are never removed.
- **Spendings can grow.** Each new recording adds a vessel build cost. KSC spendings (facility upgrades, hires) are added independently.
- **Existing spendings are never invalidated.** Adding a recording may redistribute fund earnings (e.g. milestone credit shifting), but the total earned stays the same or increases. Spendings that were valid when committed remain valid.
- **New spendings are gated by the reservation system.** The player cannot add a spending that would create a deficit at any point on the timeline.

This is the same pattern as science (section 6.8). Both resources need the reservation system because the player can add new KSC spendings (tech nodes for science, facility upgrades and vessel builds for funds) that could create deficits without it.

15.14 Open questions

- **VAB/SPH editing sessions:** Does the ledger need to track vessel cost changes during editing (parts added/removed), or only the final cost at launch? Only the final cost at launch seems necessary — the ledger doesn't track what happened in the editor.
- **Vessel cost with recoverable parts:** If a vessel is partially recovered (some stages survived), does the recovery value need to account for which parts were recovered? KSP handles this natively — the ledger just stores the awarded recovery amount.
- **Strategy conversions:** Strategies can convert reputation to funds as an ongoing rate. Deferred to the Strategies module.

---

8. Reputation Module

Status: Designed — schema, curve-dependent recalculation model, and dependency on first-tier modules specified. Exact curve formula TBD (requires decompilation).

Game mode applicability: Not applicable in sandbox or Science mode. Active in Career mode only.

6.1 KSP's reputation system

Reputation is a global score ranging from approximately -1000 to +1000. It affects gameplay in several ways:

- Higher reputation increases contract difficulty and rewards.
- More contract slots become available at higher rep.
- Strategy availability in the Administration building requires minimum rep thresholds.
- Low/negative rep can create a death spiral — bad contracts, low funds, hard to recover.

Non-linear gain/loss curve. KSP does not add reputation linearly. Gains diminish as reputation approaches the upper limit (1000) — a "+50 rep" reward might only add 45 at low rep but only 5 at high rep. Losses are asymmetrically harsher — the same magnitude of penalty has a larger impact than a gain. KSP's `AddReputation` method implements this by splitting amounts into small "grains" and tracing the curve incrementally.

The exact curve formula is not publicly documented. The coding agent should decompile KSP's `Reputation` class (specifically `AddReputation` and the grain system) to extract the precise formula. Parsek must replicate this formula in the recalculation walk to produce correct per-event effective values.

6.2 Why the curve matters for recalculation

Unlike science (where `scienceAwarded` is the post-curve actual value), reputation rewards are context-dependent — the same nominal "+50 rep" produces different actual gains depending on the current rep level. When the timeline changes and events shift order, the running rep at each event's UT changes, which changes the effective rep for every subsequent event.

If the ledger only stored and summed nominal values, per-event effective values would be wrong. The player couldn't see "this milestone actually gave you 5 rep" vs "this one gave you 45 rep." The walk must simulate the curve to produce correct intermediate results.

6.3 Immutable vs derived values

`nominalRep` — immutable. The reward or penalty amount as defined by KSP's contract/milestone configuration. Written once, never changes. This is the number KSP would display as the reward before applying the curve.

`effectiveRep` — derived. The actual reputation change after applying the gain/loss curve against the running rep at that UT. Recalculated from scratch every time the timeline changes.

6.4 Reputation action schemas

```
ReputationEarning (recording-associated or first-tier-derived)
  ut:             double
  recordingId:    string or NULL — NULL if derived from a milestone/contract action
  nominalRep:     float — reward amount before curve (IMMUTABLE)
  source:         enum — CONTRACT_COMPLETE | MILESTONE | OTHER

ReputationPenalty (recording-associated or spending action)
  ut:             double
  recordingId:    string or NULL — NULL for KSC actions (contract decline)
  nominalPenalty: float — penalty amount before curve (IMMUTABLE)
  source:         enum — CONTRACT_FAIL | CONTRACT_DECLINE | KERBAL_DEATH | STRATEGY | OTHER
```

Note: `KERBAL_DEATH` is a recording-associated penalty — killing kerbals costs reputation. This connects the kerbals module to the reputation module.

Milestone and contract rep values flow in from first-tier modules. The reputation module does not create separate entries for these — it reads the `repAwarded` fields on `MilestoneAchievement` and contract actions, checks their `effective` flags, and applies the curve.

6.5 Reputation recalculation

Reputation is a second-tier module. It runs after first-tier modules (Milestones, Contracts) have set their effective flags.

```
RecalculateReputation():
  runningRep = 0
  For each rep-affecting action in UT order:

    if REPUTATION_INITIAL (seed):
      runningRep = action.initialRep

    if MILESTONE (from milestones module):
      if action.effective:
        effectiveGain = applyGainCurve(action.repAwarded, runningRep)
        action.effectiveRep = effectiveGain
        runningRep += effectiveGain

    if CONTRACT_COMPLETE (from contracts module):
      if action.effective:
        effectiveGain = applyGainCurve(action.nominalRep, runningRep)
        action.effectiveRep = effectiveGain
        runningRep += effectiveGain

    if PENALTY (contract fail, kerbal death, etc.):
```

```
      effectiveLoss = applyLossCurve(action.nominalPenalty, runningRep)
      action.effectiveRep = effectiveLoss  // negative value
      runningRep += effectiveLoss          // effectiveLoss is negative
```

`applyGainCurve(nominal, currentRep)` and `applyLossCurve(nominal, currentRep)` must replicate KSP's internal curve. These are the functions that need to be extracted from decompiled code.

6.6 Patching

On warp exit / rewind, Parsek sets `Reputation.Instance.reputation` to `runningRep` at the current UT. Since the recalculation simulates the curve, the patched value should match what KSP would have computed natively.

6.7 Verified scenario

```
Assume simple gain curve: effectiveGain = nominal × (1 - currentRep / 1000)

Step 1: Recording A at UT=1000 completes contract, nominal +50 rep. Commit.
  runningRep = 0. effectiveGain = 50 × (1 - 0/1000) = 50. runningRep = 50.

Step 2: Recording A also achieves "First Mun Landing," nominal +15 rep.
  runningRep = 50. effectiveGain = 15 × (1 - 50/1000) = 14.25. runningRep = 64.25.

Step 3: Rewind to UT=500. Recording B at UT=700 achieves "First Mun Landing." Commit.
  Recalculate:
    UT=700 (B milestone):  runningRep=0. effective=true.
      effectiveGain = 15 × (1 - 0/1000) = 15. runningRep = 15.
    UT=1000 (A contract):  runningRep=15.
      effectiveGain = 50 × (1 - 15/1000) = 49.25. runningRep = 64.25.
    UT=1000 (A milestone): effective=false (already credited to B). +0.

  Total rep = 64.25 (was 64.25 before — close but not identical due to curve reordering).
```

Note: the total can shift slightly when events reorder because the curve is non-linear. This is correct behavior — the curve depends on order, and the timeline determines order.

6.8 Order-dependent totals and the no-delete invariant

Unlike science (where subject caps make totals stable regardless of event order), reputation totals are order-dependent due to the non-linear curve. Reordering events by adding a retroactive recording can shift the final rep value slightly, because the curve produces different effective gains at different running rep levels.

This is expected and harmless. There is no ledger-enforced invariant that depends on a minimum or exact reputation value (unlike funds, where spending affordability must be validated). Reputation affects contract offerings and strategy availability, but those are KSP-native checks at the current moment — not constraints the ledger enforces during the walk.

The no-delete invariant still holds in a weaker sense: rep-affecting events are never removed from the timeline (only added), so the set of inputs to the curve is monotonically growing. But the output of the curve (final rep) is not monotonically stable — it can shift slightly with each retroactive commit.

Reserved floor and debt (owner rulings 2026-10-21, section 3.13, built in 0.10.6). The rulings add the invariant the paragraph above says does not exist: reputation does not drop below what committed costs still need. The walk reserves the reputation a committed strategy activation costs, and the clicks reserve every open contract's reputation penalty while such a cost lies ahead (section 3.13, part 3). A loss that would take reputation below that floor (a crew death, a vessel loss) applies only down to it; the rest is a debt the walk charges from the first room above the floor, when the reservation releases at the committed cost's UT or later gains arrive. The debt is re-derived from the existing rows on every walk; the only new saved value is each row's capture order (section 3.13), which decides whether a committed cost lies ahead of a loss. V: "reputation just can't go lower than what is reserved, no matter what happens".

12.9 ~~Open questions~~ Resolved

- **Exact curve formula:** RESOLVED. Extracted from decompiled `Reputation` class. Uses `addReputation_granular` with Hermite spline AnimationCurve. 5-key addition curve (gain ~2x at rep=-1000, ~1x at rep=0, ~0x at rep=+1000) and 4-key subtraction curve (loss ~0x at rep=-1000, ~1x at rep=0, ~2x at rep=+1000). Implemented in `ReputationModule.ApplyReputationCurve` with the exact keyframes. See `docs/dev/plans/game-actions-spike-findings.md` for full keyframe data.

- **Reputation floor:** Loss curve multiplier approaches ~0.0x near rep=-1000 (symmetric with gain ceiling). No hard clamp in `AddReputation`, but `SetReputation` clamps to [-1000, 1000].

---

9. Milestones Module

Status: Designed — schema, recalculation logic, and verified scenarios complete.

Game mode applicability: Not applicable in sandbox (no progression tracking). In Science mode, milestones are tracked by KSP's Progress Tracking system but award no funds or reputation (those resources don't exist in Science mode). In Career mode, milestones award funds and reputation. The ledger tracks milestones in both Science and Career modes — in Science mode for progression gating, in Career mode for progression gating and resource awards.

7.1 KSP's milestone system

KSP's Progress Tracking / World Firsts system fires achievements for spaceflight firsts: first orbit of Kerbin, first Mun landing, first EVA on Duna, etc. Each milestone fires once ever per save. In Career mode, the World Firsts contract strategy automatically awards funds and reputation for each milestone.

Milestones are triggered automatically during flight by game events. The player does not choose to achieve them — they happen as a consequence of what the vessel does.

7.2 Milestone action schema

```
MilestoneAchievement (recording-associated action)
  ut:             double — when it was achieved during flight
  recordingId:    string — which recording triggered it
  milestoneId:    string — e.g. "FirstOrbitKerbin" / "FirstMunLanding"
  fundsAwarded:   float — what KSP awarded (IMMUTABLE, career only, 0 in Science mode)
  repAwarded:     float — what KSP awarded (IMMUTABLE, career only, 0 in Science mode)
```

`fundsAwarded` and `repAwarded` are immutable — written once at recording time based on KSP's milestone reward configuration. They are 0 in Science mode.

`effective` is derived — recalculated every time. True for the chronologically first recording that achieved this milestone, false for all later duplicates. A world-record row is decided per threshold instead: it counts while any of its thresholds is new, and earns only those (7.8).

There is no spending side. Milestones are earn-only.

Implementation note: a milestone a vessel recovery completes (2026-10-08). Stock completes some progress nodes inside the recovery itself (`KSPAchievements.CrewRecovery`, i.e. `FirstCrewToSurvive`, completes from its own `onVesselRecovered` listener), at the Space Center or the Tracking Station where no recorder is live, and before Parsek's recovery handler or stock's payout run (`EventData.Fire` walks listeners last-added-first and the node subscribes last, when `ProgressTracking` deploys it on scenario load). Owner decision: such a milestone belongs to the recovered recording, like the recovery payout. `RecoveryDispatchScope` is opened by Harmony prefixes around the stock methods that fire `onVesselRecovered` for one vessel (`VesselRetrieval.recoverVessel`, `SpaceTracking.OnRecoverConfirm`, `ShipConstruction.RecoverVesselFromFlight`, `ProtoVessel.Clean`; not for ghost map vessels, rewind strips or Parsek's crew-suppressed housekeeping recoveries). Inside it, a `MilestoneAchieved` event, and a funds / reputation / science change keyed `Progression` (the reward's own legs, so the post-walk reconciler still pairs them), that no live recorder tags is tagged to the recovered vessel's recording, picked like the payout: a pending-tree recording that still owns the vessel (exactly one terminal target; its commit books the row), else `PickRecoveryRecordingId` (the row is written at once, tagged, through `LedgerOrchestrator.OnKscSpending(evt, recordingId)`). A vessel no recording matches keeps the untagged KSC row. The tag makes the milestone follow the payout everywhere the recording id is the key: a load of a quicksave taken before the recovery reads that save's own ledger and events, which hold neither (save-file loads are cold; the in-session abandoned-future reconcile was deleted in Phase 3 of `docs/dev/design-load-restores-the-save.md`), and a Re-Fly that supersedes the recording tombstones both. Todo QUICKLOAD-UNTAGGED-RECOVERY-MILESTONE-SURVIVES-F9.

7.3 Milestone recalculation

The milestone walk is a binary cap — each milestone can be credited exactly once. This is the same pattern as science subject caps, but simpler: headroom is either 1 (not yet achieved) or 0 (already achieved).

```
RecalculateMilestones():
  1. Reset all milestone flags to uncredited.
  2. Collect all MilestoneAchievement actions.
  3. Sort by UT (within same UT, order doesn't matter —
     a recording can't trigger the same milestone twice).
  4. Walk forward:

     For each action in UT order:
       if milestoneId not yet credited:
         action.effective = true
         mark milestoneId as credited
       else:
         action.effective = false
```

This is a first-tier walk. Once effective flags are set, the milestone's `fundsAwarded` and `repAwarded` values flow into the Funds and Reputation modules' running balances in the second tier of the same recalculation pass.

7.4 Verified scenarios

Basic milestone achievement:

```
Recording A at UT=1000 achieves "First Mun Landing." Commit.
  Recalculate: effective=true. +10000 funds, +15 rep.
```

Retroactive priority shift:

```
Step 1: Recording A at UT=1000 achieves "First Mun Landing." Commit.
  effective=true. +10000 funds, +15 rep.

Step 2: Rewind to UT=500. Recording B at UT=700 also achieves "First Mun Landing." Commit.
  Recalculate:
    UT=700 (B):  not yet credited → effective=true. +10000 funds, +15 rep.
    UT=1000 (A): already credited → effective=false. +0 funds, +0 rep.
  Credit shifted from A to B. Total unchanged.
```

Multiple milestones across recordings:

```
Recording A at UT=1000: "First Mun Landing" + "First Mun EVA." Commit.
  Both effective. +10000 funds +15 rep, +5000 funds +8 rep.

Rewind to UT=500. Recording B at UT=700: "First Mun Landing" only. Commit.
  Recalculate:
    UT=700 (B):  "First Mun Landing" → effective.
    UT=1000 (A): "First Mun Landing" → NOT effective. "First Mun EVA" → effective.
  B took the landing credit. A keeps the EVA credit. Total funds and rep unchanged.
```

7.5 No-delete invariant for milestones

The same no-delete safety applies: adding a recording can only shift which recording gets credit for a milestone, never reduce the total number of milestones achieved. If milestone X was effective on any recording before, it's still effective on some recording after adding a new one. Total milestone rewards are monotonically stable.

7.6 ~~Open questions~~ Resolved

- **Milestone detection during recording:** RESOLVED. `GameStateRecorder` subscribes to `GameEvents.OnProgressComplete`. Events are converted to `MilestoneAchievement` actions via `GameStateEventConverter.ConvertMilestoneAchieved` at commit time. Known limitation: `OnProgressComplete` provides the progress node but not the associated fund/rep reward values — reward amounts must be extracted from the World Firsts contract strategy configuration (deferred item D17).
- **Science mode milestones:** In Science mode, milestones track progression but award nothing. The ledger tracks them for progression gating consistency. KSP's native Progress Tracking remains the authoritative source for milestone state.
- **Milestone list:** The ledger only tracks achieved milestones. It does not maintain a list of all possible milestones — that's KSP's domain.

7.7 Progress earned before the ledger

`MilestonesModule.Reset` folds the save's pre-ledger progress seed (4.5) into every walk. A seeded one-shot id starts credited, so the patch keeps its node achieved; it predates every ledger row, so a later row for the same id (stock re-awarding a node something un-achieved) is a duplicate under the first-hit rule and is not counted. A seeded world-record count N credits that node's thresholds 1 to N before any row, so a named crossing at or below N is a repeat and earns nothing (on one timeline none exists: the seed owns exactly the band stock held beyond the ledger's crossings, 4.5), and the patched band is N plus the crossings the ledger counts above it, on a normal and on an authoritative (rewind) patch alike (7.8). Nodes outside the seed are patched exactly as before. A node with a ledger row anywhere on the timeline is never in the seed, so a rewind before that row still clears it, and a rewind into a tree that lacks it does not set it.

7.8 World-record rows and their threshold count

Stock's four world-record nodes (`RecordsAltitude`, `RecordsDepth`, `RecordsSpeed`, `RecordsDistance`) pay a reward each time the record crosses one of `ContractDefs.Progression.RecordSplit` thresholds (500 m, 2 km, 7 km, 22 km, 70 km for altitude on Kerbin), one `AwardProgress` call per threshold, and complete the node after the award loop once the maximum is reached. `GameStateRecorder.TryCoalesceWorldRecordReward` folds every award of one (record id, recording) scope into one `MilestoneAchieved` event and one `MilestoneAchievement` row, summing the rewards. Each row also carries how many thresholds it stands for, `MilestoneRecordThresholds` (ledger key `milestoneRecordThresholds`, event detail key `thresholds`, both written only when not 1): the first award writes 1, each fold adds 1 to the event and, when the direct path already forwarded it, to that event's own ledger row (same id, scope and UT; an older committed row of the same scope is never folded into, since the event commits as its own row), and the completion event (`OnProgressComplete`, no award follows it) writes 0 and is not left pending for reward enrichment.

Which thresholds a row holds (section 3.15 step 4, built). Each row and event also names its thresholds, `MilestoneRecordBand` (ledger key `milestoneRecordBand`, event detail key `band`, written only when present): a comma list of `index:funds:rep:sci` entries, InvariantCulture round-trip numbers. The index is the 1-based threshold stock is paying, read in the `AwardProgress` postfix from the node's private `rewardInterval` field (`GameStateRecorder.ReadRecordRewardInterval`): the decompiled `RecordsAltitude` / `Depth` / `Speed` / `Distance.iterateVessels` loop calls `AwardProgressInterval(..., rewardInterval, RecordSplit, ...)` and only then advances `rewardInterval` with `FindNextRecord`, so inside the award the field still names the threshold paid. A fold adds an entry, or adds to the existing entry when stock pays the same index again in one scope. Stock re-pays: its award block runs on every iteration and the loop breaks without advancing after the last interval (or after a threshold rounded to the maximum or above), so while the record sits at or past that threshold below the maximum (a home body whose maximum is not a multiple of the rounding) it pays the interval again every frame until the node completes. An award of an interval the live node instance has already paid is marked a re-payment (`GameStateRecorder.NoteRecordIntervalPaid`, weak per-instance memory; the entry's index carries an `r` suffix, `5r:5000:0:0`) when it opens a scope's entry; a re-payment folded into the entry of the scope's own crossing rides that entry. The mark is exact on the timeline that paid it: a fresh node instance (a scene load) never re-pays, because its `OnLoad` sets `rewardThreshold = FindNextRecord(record)`, strictly above the record, so its first award of an interval is a crossing. The list is all or nothing: when one award's index cannot be read (no field, a value below 1, `int.MaxValue`), the event and row carry the token `-` from then on and the row is read as an older row. Additive: no schema bump; an older build ignores the key.

The replay credits each threshold once, and one timeline walks what stock paid. `MilestonesModule` feeds every record row in walk order to a `RecordThresholdBook`, which books each row's entries per node:

- A row that names its thresholds stands for those indices, each a crossing or a re-payment.
- An older row (no list, or `-`) is derived: it stands for `MilestoneRecordThresholds` positions counted on from a base, the highest position among the pre-ledger seed and the same node's rows that are earlier in walk order and were captured before it (`CapturedBefore` on `cseq`; a row with no capture number, 0 or -1, counts as captured before every numbered row, and two such rows keep walk order). After a rewind the re-crossing is captured later but walks first, so its base skips it and the committed older row keeps its own positions, which the re-crossing already holds. The positions are a guess: an older build's scope that stayed `-` across the upgrade may hold intervals 1 and 3 while a newer scope named 2.
- An overlap zeroes a position only on evidence that stock's band was lowered between the two payments, which one timeline never shows (the band only advances, and a fresh node resumes above its saved record):
  - the position's counted holder walks earlier but was captured after this row (`CapturedLaterWalkedEarlier`: a later timeline re-crossed it earlier);
  - both are named crossings of one interval (`SecondFirstPayment`: stock paid it as new twice, so a rewind lowered the band and the mirror did not raise it);
  - a named crossing at or below the pre-ledger seed (`PreLedgerSeed`);
  - a re-payment is zeroed only when the latest named crossing of its interval captured before it was zeroed (`RepayOfZeroedCrossing`: it rides the crossing of its own timeline, which walks before it because one timeline's UT and capture order agree). So a derived guess never zeroes a named row captured after it, and a re-payment in a second scope of the same session counts.

- All counted: the row counts in full with its captured amounts. None: the row is not counted (`RecordThresholdsAlreadyCredited`, Timeline hover "Not counted: this record was already set that far earlier on timeline, so no reward was paid."). Some: the row counts and earns only its counted entries in funds, reputation and science (`RecordPartlyCredited`, `RecordCreditedFunds` / `Rep` / `Science`, read through `GameAction.CreditedMilestoneFunds` / `Rep` / `Science` by the funds, reputation and science modules and the commit-window deltas; Timeline hover "Partly counted: ..."). A named row earns the rewards of its counted entries. A derived row earns its totals less, for each zeroed position, the reward the named crossing holding that position was paid (stock's price for the same threshold on the other timeline), or an even share of its totals where a derived row holds it, clamped to between 0 and its totals; its per-threshold rewards were never stored (they follow difficulty multipliers and strategies, so the summed reward cannot be split exactly). A partly counted row is skipped by the post-walk reconciler.
- A row that stands for no threshold (the completion event) stays counted with nothing to pay.
- Pinned by `NeverRewoundCareer_WalksExactlyWhatStockPaid`, which simulates stock on one timeline (bands that only advance, interleaved scopes, an old save, the step 1-3 builds, this build with lists that go `-`, re-payments, scene loads) and requires every row counted in full, the walk paying exactly what stock paid, and a count never below stock's band.

A record id's effective count is the seed N plus the counted crossings (a derived row's positions each count once; a re-payment adds nothing), and `KspStatePatcher.PatchRepeatableRecordNode` rebuilds the node's band from that count, so a flight that crossed every threshold leaves the node complete, a re-crossing never doubles a band, and on one timeline the count is never below the band stock holds, so the patch never lowers it (an older build folded re-payments into its count, the one way it can read above the band). Rows and events written before the threshold count existed read as one threshold each. A coalesced row sits at its first fold's UT, so a rewind to a UT inside the folded span keeps all of that row's thresholds and reward on the pre-rewind side.

Accepted limits: two older rows that both lack a capture number keep walk order and both pay (no evidence says which crossing came first). An older row partly overlapped by another older row earns an even share rather than the exact rewards of the thresholds it keeps. After a rewind whose mirror failed, a re-crossing in a `-` row captured after the committed crossing is not recognised (its derived positions sit above the committed band), so it pays.

7.9 A world-first the committed timeline already credits

Stock pays a once-ever world-first only while its progress node is not complete (every once-ever handler runs `if (!IsComplete) { Complete(); AwardProgressStandard(...); }`). After a rewind the loaded save has the node unreached and the committed row lies ahead of the clock. `PatchMilestones` sets the node achieved once a patch walks past the row, but every patch is deferred while a recorder lives and a flight at 1x runs no recalculation, so a later flight that earns the node again would be paid by stock while the walk zeroes its row as a duplicate (7.4): paid twice. `CommittedMilestoneReach.Apply` closes that gap: once the clock is at or past a committed once-ever row (ELS, the walk's own input), it marks the stock node reached and complete through the private fields, with no `Complete()`, no event and no reward. It is add-only (it never clears a node, so a live flight's own uncommitted progress stays), it runs from the deferred branch of the recalculation (clock: the walk's real cutoff, else the live clock only outside OnLoad and a pending rewind UT adjustment) and from the flight scene's `CommittedMilestoneClockCursor` when the clock passes the next such row, and it skips rows of a live Re-Fly session's suppressed subtree, because a merge tombstones the origin's milestone rows and the attempt must still be able to earn the node. A re-earn before the committed row's UT is untouched: the clock has not reached the row, stock pays the earlier flight, and the walk moves the credit to it (first in UT order).

World-record bands (section 3.15 step 4, built). Stock pays a record threshold while the node's band (`rewardThreshold` / `rewardInterval`) sits below it, so after a rewind a flight that climbs to a height the committed timeline already reached would be paid by stock again while the walk zeroes the repeat. The game rule for the player: each step of a record pays once in a career; once the clock has passed the committed flight that set a record that far, setting it again pays nothing. `CommittedMilestoneReach.Apply` therefore also raises each record node's band to the committed count at the clock: the pre-ledger seed plus the counted crossings of the ELS record rows at or before the clock (`CollectCommittedRecordCountsAtOrBefore`, the walk's own `RecordThresholdBook` over those rows in walk order), skipping a live Re-Fly session's suppressed subtree. It raises a band only when that count is above the band stock already holds (`KspStatePatcher.TryReadRepeatableRecordImpliedPaidCount`), through `TryRaiseRepeatableRecordBand` and the band patch, never lowers one, pays nothing and fires no event. The info line keeps its "marked N stock progress node(s) complete without reward" phrase and adds "raised R record band(s) without reward". `CommittedMilestoneClockCursor` also fires when the clock passes a committed record row that names or counts thresholds (`IsMirroredMilestoneRow`, part of `IsMirroredRow`, 3.15 step 5); a completion row (no threshold) is not mirrored. A re-climb before the committed row's UT is untouched: stock pays the earlier flight and the walk moves those thresholds' credit to it.

---

10. Contracts Module

Status: Designed — state transitions, reservation system, once-ever completion, deadline handling, and KSP state patching specified.

Game mode applicability: Not applicable in sandbox or Science mode. Active in Career mode only. Contracts are the primary source of funds and reputation income in career.

8.1 KSP's contract system

Contracts are procedurally generated offers that reward the player for completing objectives. Each contract has a state machine:

```
OFFERED → ACCEPTED → COMPLETED
                   → FAILED (deadline expiration or vessel loss)
                   → CANCELLED (player choice at KSC)
                   → ENDED (requirements no longer met; stock state OfferExpired)
```

Each transition has resource effects:

- **Accept**: advance payment (funds earning), contract slot consumed.
- **Complete**: reward (funds + reputation + sometimes science earning).
- **Fail**: penalty (funds + reputation loss).
- **Cancel**: partial penalty (funds + reputation loss). Stock scales it with time: with `t` the share of the accept-to-deadline span that had passed at the cancel, the funds penalty runs from the advance (`t = 0`) to the fail penalty (`t = 1`) and the reputation penalty from the career's decline loss to the fail loss (`Contract.PenalizeCancellation` , KSP 1.12.5). A contract with no deadline is charged at `t = 0` . The ledger records exactly that amount, never the full fail penalty.
- **Ended by stock (requirements no longer met)**: no penalty. Stock's `Contract.Update()` ends an Active contract whose requirements stop holding as `OfferExpired` . The contract leaves the active list, its slot frees, and nothing is charged.

Game rules for contract outcomes (0.10.6, todos CONTRACT-CANCEL-PENALTY-IGNORES-STOCK-TIME-LERP, CONTRACT-CANCEL-ALSO-FILES-A-FAIL-ROW, STOCK-OFFER-EXPIRED-ACTIVE-CONTRACT-NO-TERMINAL-ROW):

1. A contract that ends has exactly one outcome row: complete, fail, cancel, or ended.
2. A cancel costs what stock charged for it, the time-scaled share above, never more.
3. An Active contract stock ends because its requirements no longer hold is ended at that moment, with no penalty. It frees its slot then, not at its deadline, and it is never rebuilt as Active.

Capture: `GameStateRecorder.OnContractCancelled` records the lerped penalty (`GameActions/ContractCancellationPenalty.cs` , stock's float-narrowed dates included). Stock's `SetState(Cancelled)` fires `onCancelled` and then `onFailed` , so `OnContractFailed` skips a fire whose contract is already Cancelled. `Patches/ContractOfferExpiredCapturePatch.cs` (prefix + postfix on `Contract.SetState`) raises the Active → OfferExpired transition, which stock reports only through `onFinished` . The recorder files it as a ContractCancelled event with zero penalties and `ended=requirements` , converted to a `ContractCancel` row with `ContractRequirementsLost` set (serialized `requirementsLost = True`), shown as `Ended: <contract>` . A refused end (rule 1 of section 3.13) leaves the contract Active and files nothing.

Contracts are offered by KSP procedurally based on progress, reputation level, and randomness. Mission Control has a limited number of active contract slots (determined by building level). Each contract has a unique ID assigned by KSP.

8.2 Contract slot reservation - the committed-future slot forecast

Correction (0.10.5, stock-UI overlay program). An earlier draft reserved an accepted contract's Mission Control slot from UT=0 and showed the contract as accepted at every UT. That is not the shipped rule. Stock's contract state follows the clock: whenever committed rows lie after now, the recalculations that patch KSP state walk with a current-UT cutoff (a rewind passes the rewind UT to `LedgerOrchestrator.RecalculateAndPatch(utCutoff)` ; a KSP load, a commit, a time jump and a live KSC event go through `RecalculateAndPatchForCurrentTimelineUT` or its `...IfFutureActions` / `ForLiveTimelineEvent` wrappers). The cutoff walk sees only rows at or before now, and `KspStatePatcher.PatchContracts` restores exactly the contracts `ContractsModule.GetActiveContractIds` holds after that walk. A contract is therefore Active in Mission Control from its accept UT to its resolution or deadline, and a contract the committed timeline accepts later is still an offer (or absent) before that date.

The committed slots are protected at the stock control instead, by a forecast over the committed future (`ContractSlotReservation` , block C2 of `docs/dev/research/stock-ui-reservation-overlays-2026-09-25.md`). It reads the contracts Active in stock now and the committed rows of the committed-future index (section 14), so a live, pending or Re-Fly tree reserves nothing:

- Every contract Active now (counted as stock counts them: Active and not auto-accepted) holds its slot until the earlier of its first committed complete / fail / cancel row after now and its deadline, or throughout when it has neither.
- Every committed `ContractAccept` ahead adds its contract at its UT, until the earlier of its first committed resolution and its accepted deadline. A stock auto-accept contract is not counted (stock's `GetActiveContractCount` skips it).

- A deadline frees the slot AT the deadline UT, the rule `ContractsModule.CheckDeadlines` uses (section 10.6). On a UT tie, removals come first, then a committed Mission Control level change (an upgrade raises the limit to stock's limit for the new level, a Rebuild sets it to the lower level's limit), then accepts.

```
ContractSlotForecast (derived per query; ContractSlotReservation.Forecast)
  LimitNow:                 stock slot limit at the current Mission Control level
  ActiveNow:                contracts Active in stock now (auto-accept excluded)
  PeakCommitted:            most contracts active at once from now to the end of
                            the committed timeline, with no new accept
  FreeSlotsForNewAcceptNow: min over now and each committed accept ahead of
                            (limit at that UT - committed count at that UT)
  FirstStarvedAccept:       earliest committed accept a contract accepted now
                            would leave without a slot, or null

BlocksNewAccept(releaseUT) = ActiveNow < LimitNow
                             AND FirstStarvedAccept != null
                             AND FirstStarvedAccept.UT < releaseUT
```

A new contract holds its slot until its own deadline (`NewAcceptReleaseUT`, following stock's `Contract.Accept` deadline rules), so it is refused only when a committed accept before that release would be starved. When no slot is free now, stock's own rule already refuses the accept and Parsek adds nothing.

What Mission Control does with this (the screen side is section 14):

- **Accept of any other offer** is greyed and refused while `BlocksNewAccept` holds, with the reason `Slot needed from <date> for '<contract>' (<agent>), blocked by timeline.` Decline stays available.
- **An offer the committed timeline accepts later** has Accept and Decline greyed and refused, and its row reads e.g. `- Accepted on Y1, D03`. Accepting it early would accept it twice; declining it changes nothing (it becomes Active on the committed date anyway) but could cost reputation.
- **An Active contract the committed timeline completes, fails or cancels later** has Cancel greyed and refused, and its row reads e.g. `- Completed on Y2, D114`. A contract whose only future is its deadline stays cancellable.

The same forecast ends the Timeline's Contracts button hover in Career mode (`CareerSlotSummary`, e.g. `Contract slots: 4 of 7 free now (2 active, 1 reserved for later).`), which replaced the removed Career window.

8.3 Once-ever completion (like milestones)

A specific contract instance can only be completed once on the timeline. If two recordings both meet the completion conditions for the same contract, only the chronologically first one gets credit.

```
For each ContractComplete in UT order:
  if contractId not yet credited:
    action.effective = true
    mark contractId as credited
  else:
    action.effective = false  // duplicate completion, rewards zeroed
```

Effective completion rewards (funds, rep, science) flow into second-tier modules. Non-effective completions produce zero rewards but the contract is still resolved (slot freed).

Parameter rewards. Stock pays a contract in two parts: each `ContractParameter` that completes pays its own `FundsCompletion` / `ScienceCompletion` / `ReputationCompletion` (reason `ContractReward`) the moment it completes, and the contract then pays its contract-level reward. A World-Firsts exploration contract carries most of its money on its one "Achieve goal" parameter. Each paid parameter is its own `ContractParameterComplete` row at its own UT, credited once per (contract, parameter) and never once the walk has ended the contract (its completion, a deadline failure or an earlier explicit fail / cancel); it moves no slot and no terminal state. The capture is a Harmony prefix / postfix on `ContractParameter.AwardCompletion` (decompiled KSP 1.12.5: stock zeroes the three fields after paying them, before `onParameterChange` fires, so the event alone always reads zero). Tagging, the clock-cause scope, the direct-ledger forward, the supersede tombstone and the discard re-home follow `ContractComplete`. A leg stock skipped (a strategy's effect delta cancelling it, which leaves the field set) is not booked. Once a real clock (the walk's cutoff, else the committed-row clock; never NaN, so not during OnLoad) reaches a credited parameter row, the contract patch puts stock's live parameter in its post-payment state without paying (state Complete by reflection, rewards zeroed, disabled), so stock cannot pay it a second time when a later flight completes it again; the first row in walk order is the one counted. Built (section 3.15 build order step 3): a parameter row is also not counted once the walk has completed its contract (`ContractsModule.IsParameterAfterContractCompletion`, Timeline reason "the contract was already completed"). Stock pays a

contract's parameters before its contract-level reward, so the test follows the walk order: a row at a later UT is after the completion; at the completion's UT it is after only when both rows carry a `cseq` and the parameter's is higher. A parameter row with no capture order at that UT is taken as paid before the completion and still counts.

A contract (or group parameter) that silent write leaves fully satisfied is handed back to stock's own `ParameterStateUpdate` one frame later, in the recorder's clock-cause scope, so stock completes and pays it as it would have live, filed untagged; a contract the ledger already ended is never re-evaluated. Todo CONTRACT-PARAMETER-REWARDS-NOT-LEDGERED.

8.4 Contract action schemas

```
ContractAccept (KSC action — consumes a slot)
  ut:             double — frozen KSC time
  sequence:       int — order within same UT
  contractId:     string — KSP's unique contract instance ID
  contractType:   string — e.g. "ExploreBody", "PartTest", "TourismContract"
  title:          string — human-readable (e.g. "Explore the Mun")
  advanceFunds:   float — advance payment received (IMMUTABLE)
  deadlineAbsUT:  double or NULL — ABSOLUTE expiration UT (stock
                  `Contract.DateDeadline`), NULL if no deadline. The legacy key
                  `deadlineUT` carries a DURATION and is migrated on load — see
                  CONTRACT-DEADLINE-CAPTURED-AS-DURATION in
                  `docs/dev/todo-and-known-bugs.md`

ContractComplete (recording-associated earning)
  ut:             double — when completion conditions were met during flight
  recordingId:    string
  contractId:     string
  fundsReward:    float — IMMUTABLE
  repReward:      float — IMMUTABLE (nominal, pre-curve)
  scienceReward:  float — IMMUTABLE (some contracts award science)

ContractParameterComplete (recording-associated earning; one paid contract parameter)
  ut:             double - when stock paid the parameter
  recordingId:    string - same tagging rule as ContractComplete
  contractId:     string
  contractParameterId: string - the parameter's index path in its contract ("0", "1.0")
  fundsReward:    float - IMMUTABLE
  repReward:      float - IMMUTABLE (nominal, pre-curve)
  scienceReward:  float - IMMUTABLE (pool credit, no subject)

ContractFail (recording-associated or timeline event)
  ut:             double — when failure occurred
  recordingId:    string or NULL — NULL if deadline expiration
  contractId:     string
  fundsPenalty:   float — IMMUTABLE
  repPenalty:     float — IMMUTABLE (nominal, pre-curve)

ContractCancel (KSC action, or stock's requirements-lost end)
  ut:             double — frozen KSC time
  sequence:       int
  contractId:     string
  fundsPenalty:   float — IMMUTABLE (the time-scaled cancel penalty stock charged; 0 when ended)
  repPenalty:     float — IMMUTABLE (nominal, pre-curve; 0 when ended)
  requirementsLost: bool — true for stock's OfferExpired end of an Active contract
```

8.5 Contract recalculation

Contracts are a first-tier module. The walk processes contract actions and sets effective flags. Fund and reputation effects flow into second-tier modules.

```
RecalculateContracts():
  creditedContracts = {}
  activeContracts = {}

  For each contract action in UT order:
    if ACCEPT:
      activeContracts[contractId] = action
      // advance funds flow into Funds module

    if COMPLETE:
```

```
      if contractId not in creditedContracts:
        action.effective = true
        creditedContracts[contractId] = true
        // rewards flow into Funds, Reputation, Science modules
      else:
        action.effective = false
      remove contractId from activeContracts (slot freed)

    if FAIL:
      // penalties flow into Funds, Reputation modules
      remove contractId from activeContracts (slot freed)

    if CANCEL:
      // penalties flow into Funds, Reputation modules
      remove contractId from activeContracts (slot freed)
```

8.6 Deadline handling

Status: IMPLEMENTED. Both deadline slot-freeing and penalty application are implemented. `ContractsModule.CheckDeadlines` runs on every `ProcessAction` dispatch and removes expired contracts from `activeContracts` when the deadline UT is crossed. `ContractsModule.PrePass` scans accepted contracts for deadlines that expire without a prior completion or cancellation and injects a synthetic `ContractFail` action at the deadline UT carrying the contract's `FundsPenalty` and `RepPenalty`. Since 0.10.6 (committed-value rule phase 2, section 3.13) the visible walk gives that injected row to the contracts module only: it retires the contract and moves no pool, because stock files its own `ContractFail` row when it charges a deadline penalty (the walk withholds every row not in its input list from the funds, science and reputation modules and logs `Walk withheld N injected row(s)`). The projection keeps it out of the pool reserve the same way. An explicit `ContractFail` / `ContractCancel` at or before the deadline suppresses the synthetic fail, so a contract is never double-penalized.

Contracts with deadlines generate an automatic failure event when the deadline UT is crossed. During the recalculation walk, if an accepted contract's deadline UT is reached without a prior completion or cancellation, the walk inserts a `ContractFail` event at the deadline UT.

```
During walk, for each accepted contract:
  if deadlineUT is reached AND contract still in activeContracts:
    generate ContractFail at deadlineUT (recordingId = NULL)
    retire the contract (no pool moves: stock's own fail row carries the penalty)
    remove from activeContracts
```

This means deadline failures are derived events — they don't need to be stored in the ledger. They're produced during the walk whenever an accepted contract's deadline passes without resolution.

During fast-forward, the deadline crossing is visual only — the actual failure is processed on warp exit during recalculation.

Stock's own fail row is filed under no flight (owner ruling 2026-10-24, section 3.14). The `ContractFail` row stock writes when it charges a deadline penalty, and its strategy conversions, are untagged and written to the ledger at once, so no flight's discard, revert or Re-Fly merge can remove them. The same holds for a contract stock ends because its requirements no longer hold (the zero-penalty `ContractCancel` row with `requirementsLost`).

8.7 Funds reservation interaction

Contract advance payments are fund earnings — they increase the balance on accept. Contract rewards are fund earnings on completion. Contract penalties are fund losses on failure/cancel.

All of these participate in the funds reservation system (section 7.6). The advance payment increases available funds at the accept UT. Future rewards are not counted until the recording that completes the contract is committed (earnings are recording-associated).

Committed outcomes and open penalties (owner rulings 2026-10-21, section 3.13, built in 0.10.6). (1) A contract whose completion, failure or cancellation is committed cannot be failed before that outcome: stock's fail is refused and logged. Completing it earlier stays allowed. This extends D7 (section 14.4) from the Cancel button to in-flight fails. (2) An open contract's fail penalties (funds and reputation; stock contracts carry no science penalty) are reserved from its accept to its outcome while a committed purchase or strategy cost lies ahead. (3) Accept, Cancel and Decline are greyed with the reason, and refused, when the reservation they add, or the penalty they charge, would leave a later committed purchase or strategy cost unpaid. A deadline expiry needs nothing more: the committed timeline holds the same synthetic fail at the same UT (section 8.6), and stock's own fail row is filed under no flight (section 3.14).

8.8 KSP state patching

On warp exit / rewind, Parsek patches KSP's `ContractSystem` to match the ledger's contract state at the current UT:

- **Accepted contracts**: patched into KSP as active when the walk has them active, i.e. accepted at or before the walk's cutoff and not yet resolved or expired. A contract the committed timeline accepts later is not active before its accept UT; its slot is held at the stock control by the forecast in section 10.2.
- **Completed contracts**: patched as completed. KSP won't re-offer them.
- **Failed contracts**: patched as failed.
- **Cancelled contracts**: patched as cancelled.
- **Contracts ended by stock (requirements no longer met)**: not Active after their end row, so never rebuilt as Active; stock's own `OfferExpired` entry is left in place.
- **Contracts not yet on the timeline**: left to KSP's procedural generation. Parsek doesn't interfere with offerings.

This is the most complex patching operation across all modules. KSP's `Contract` objects have parameters, conditions, and internal state that must be correctly restored. The coding agent should investigate KSP's contract serialization API to determine how much of the contract state can be preserved and restored.

8.9 Procedural generation across rewinds

KSP generates contract offerings procedurally. After a rewind, KSP may generate different offerings than before. Parsek handles this by patching accepted/completed/failed contracts into KSP's state — these are removed from the offering pool. New offerings are KSP's domain.

This means: if the player accepted "Orbit Mun" in a previous commit, it won't be re-offered after rewind because Parsek patches it as accepted. KSP generates other contracts instead. The player doesn't see duplicate contracts.

Twins (planned, section 3.15 step 8).

- Stock can generate the same goal again under a new guid before the committed accept's UT, either as an offer or as a contract the clock accepts on its own (`AutoAccept`).
- The committed-future contract keeps such a twin out of the offer pool.
- Identity is never keyed on contract type and target, because repeatable contracts (part tests, tourism, rescues, Contract Configurator `maxCompletions > 1`) legitimately pay again.

12.10 Verified scenarios

Basic accept and complete:

```
UT=100: Accept "Orbit Mun". Advance +5k funds. Slot consumed.
UT=500: Recording A completes "Orbit Mun". +40k funds, +15 rep. Commit.
  Slot freed. Contract resolved.
```

Once-ever completion with retroactive priority:

```
UT=500: Accept "Orbit Mun".
UT=1000: Recording A completes it. effective=true. +40k, +15 rep.

Rewind to UT=600. Recording B also completes "Orbit Mun" at UT=700. Commit.
  Recalculate:
    UT=700 (B): effective=true. +40k, +15 rep.
    UT=1000 (A): effective=false. +0, +0.
  Credit shifts to B. Total unchanged. Contract resolved at UT=700.
```

Slot reservation across rewind:

```
Mission Control: 3 slots max.

Contract A: accepted UT=100, completed UT=500.
Contract B: accepted UT=200, pending (no resolution).
Contract C: accepted UT=300, cancelled UT=400.

Player rewinds to UT=50.
  Stock state at UT=50: none of A, B, C is active yet (all accepted later). ActiveNow = 0.
```

```
  Forecast: A from 100 to 500, B from 200 on, C from 300 to 400.
  Peak committed = 3 at UT=300. FreeSlotsForNewAcceptNow = 3 - 3 = 0.
  A new contract X with no deadline, accepted now: at UT=300 the count is 4 > 3,
    so FirstStarvedAccept = C. Accept on X is greyed and refused:
    "Slot needed from <C's date> for 'C', blocked by timeline."
  An X whose deadline is before UT=300 frees its slot in time and is allowed.
  A, B or C on offer: Accept and Decline greyed, row reads "- Accepted on <date>".

Player fast-forwards to UT=450.
  Warp exit walks to UT=450: A active (completes at 500), B active, C cancelled at 400.
  ActiveNow = 2. No committed accept ahead, so no accept is starved.
  Stock's own rule applies: 3 - 2 = 1 free slot. Player can accept one more.
  Cancel on A is greyed ("- Completed on <date>"); B has no committed resolution
  and stays cancellable.
```

Deadline failure during walk:

```
Contract D: accepted UT=100, deadline UT=800. No completion recorded.

Recalculate walk:
  UT=100: Accept. Slot consumed.
  UT=800: Deadline reached, contract still active → derived ContractFail.
    Apply penalties. Slot freed.
```

12.11 Open questions

- **Contract parameter preservation:** KSP contracts have complex internal state (completion conditions, parameters, waypoints). How much of this can Parsek serialize and restore on patching? If contract objects can't be fully reconstructed, Parsek may need to store a serialized snapshot of the contract at accept time.
- **Contract generation seeding:** KSP's procedural contract generation uses randomness. After a rewind, different contracts may be offered. Does Parsek need to seed the RNG to produce consistent offerings, or is divergence acceptable?
- **Contracts that reference recordings:** Some contracts require specific conditions (reach orbit, land on body) that are met during recordings. If a recording is completed but the contract completion is tied to it, and then a second recording also meets the conditions, the once-ever flag handles the duplicate — but does the contract's internal condition-tracking agree with Parsek's state?
- **Tourist contracts:** A tourist is a passenger of the flight that carried him. A tourist a committed future flight still carries is held for it (greyed in the crew dialog until that flight ends), so after a rewind he can never ride two vessels at once; a tourist never gets a stand-in or any replacement kerbal; a tourist missing from the roster when that flight's vessel spawns comes back as a tourist (KerbalType.Tourist, trait Tourist), whom stock sends home on recovery, never as a hired kerbal. His KerbalAssignment row carries role "Tourist" and no value. The hold half is one policy flag (`KerbalsModule.TouristsHeldByCommittedFlights`).
- **Rescue contracts:** Rescue contracts spawn a stranded kerbal. Completing the rescue adds them to the roster for free. This overlaps with the kerbals module's `KerbalRescue` action (section 11.10). The contract module records the contract completion; the kerbals module records the kerbal's addition to the roster. Both are recording-associated actions on the same recording.

---

11. Kerbals Module

Status: Designed — core reservation and replacement systems specified.

Kerbals are unique named entities with identity, state, and scarcity. They are the most complex resource module because they combine reservation (exclusive assignment), replacement (roster stability), and lifecycle management (XP, death, rescue).

Game mode applicability: Active in all game modes. In sandbox, kerbals are free and abundant but still have identity — the reservation system prevents the duplicate-kerbal paradox. Replacement is trivial (free, unlimited). In Science mode, kerbals exist and can die/be stranded, but there is no hiring cost (funds don't exist). In Career mode, kerbals cost funds to hire, gain XP, and are scarce — the full reservation, replacement chain, and hiring cost systems apply.

9.1 The duplicate kerbal problem

Parsek's rewind creates a unique problem with kerbals. Unlike science (fungible points) or funds (a number), kerbals have identity. The same kerbal cannot physically exist in two places at the same time on the timeline.

```
Without reservation:
  Recording A: Jeb launches UT=1000, stranded on Mun. Committed.
  Player rewinds to UT=500. KSP restores Jeb to roster.
  Recording B: Jeb launches UT=800, flies until UT=1500. Committed.
  Fast-forward: at UT=1000 ghost-Jeb becomes real (stranded on Mun).
  At UT=800–1500 recording-B Jeb is also real and flying.
  Two real Jebs exist from UT=1000 to UT=1500. Paradox.
```

The reservation system prevents this by construction.

9.2 Reservation rules

When a recording is committed with a kerbal onboard, that kerbal is **reserved from UT=0 until their latest endpoint across all recordings on the timeline.** The reservation starts at UT=0, not at the recording's launch time. This prevents any earlier mission from using the kerbal.

```
KerbalReservation (derived, per kerbal)
  kerbalName:     "Jeb"
  reservedFrom:   0 (always — invariant)
  reservedUntil:  max(endUT) across all recordings with this kerbal,
                  or INDEFINITE if any recording has DEAD, MIA, or STRANDED end state
                  (STRANDED becomes bounded when a rescue recording provides endUT)
```

The reservation is a single continuous block. There are no gaps. A kerbal is either free (after `reservedUntil`) or completely unavailable (UT=0 through `reservedUntil`). This eliminates any possibility of overlapping assignments.

Why not gap-based reservation? An alternative design would allow a kerbal to be used in gaps between recordings (e.g. Jeb recovers at UT=1500, next mission starts at UT=2000 — gap from 1500 to 2000 could be usable). This was rejected because the player cannot control mission duration at launch time. If a player launches Jeb at UT=1600 intending to return by UT=1900, but the mission runs long past UT=2000, it overlaps with the next recording. Enforcement would have to happen at commit time — after the player already flew the entire mission — which is unacceptable UX. The continuous block from UT=0 prevents this by making the kerbal unavailable until all recordings are resolved.

At commit time: the ledger checks whether each kerbal on the recording is currently reserved. If so, the commit is rejected — the player must use a different kerbal or revert.

On rewind: all reservations are recomputed from scratch, just like science recalculation. Walk all committed recordings, compute reservations per kerbal. Reservation status is always derived, never stored.

9.3 End states and reservation duration

End state	Reservation	Duration	Meaning
RECOVERED	Temporary	UT=0 → recovery UT	Kerbal returned to KSC, available after recovery
STRANDED	Temporary (open-ended)	UT=0 → until rescue recording closes it	Kerbal alive but stuck, rescuable by future mission
DEAD	Permanent, or temporary with stock respawn on	UT=0 → INDEFINITE (respawn off) / UT=0 → death UT + RespawnTimer (respawn on)	Kerbal gone; back after the stock respawn timer when respawn was on at the death
MIA	Permanent	UT=0 → INDEFINITE	Kerbal gone, KSP may make available for rehire later

STRANDED is temporary in principle even if long-lived in practice. The reservation stays open until a future recording rescues the kerbal (picks them up and recovers). If no rescue ever happens, it behaves like permanent reservation.

Implementation divergence: The actual `KerbalEndState` enum uses different values:

- `Aboard = 0` — kerbal is on a vessel that is still intact (covers design doc's STRANDED + active vessel states)
- `Dead = 1` — kerbal was killed
- `Recovered = 2` — kerbal was recovered with the vessel
- `Unknown = 3` — end state could not be determined (legacy recording, missing data; covers design doc's MIA)

The design doc's `STRANDED` is mapped to `Aboard` (crew still on vessel, open-ended reservation). `MIA` is mapped to `Unknown` (conservative: treated as open-ended temporary reservation).

Implementation note: when a temporary reservation ends (2026-09-23). The derived reservation keeps `ReservedUntilUT` (the Recovered flight's end, or `+inf`), and every consumer asks one predicate, `KerbalsModule.IsReservedAt(name, nowUT)`: a reservation exists AND it is permanent, open-ended, or `nowUT < ReservedUntilUT`. The kerbal is free AT the recovery instant (a commit usually runs at exactly that clock); with no readable clock the last decision stands for an unchanged hold and a new or changed hold is held. A cutoff recalculation's provisional first walk (cutoff-filtered rows) records no release transitions; the whole-ledger recompute is authoritative. A returned owner can fly again but stays protected from dismissal while a committed flight names him. "Now" is captured once per recalculation walk: the loaded save's time while a save is loading, else the walk's cutoff (rewind, time jump, current-UT walk), else the live clock. Scene loads, commits, rewinds and flight warp exit already recalculate; time passing in the Space Center or the Tracking Station, and the crew dialog opening, trigger one recalculation when a recovery time is crossed. When the hold ends the owner reclaims his slot through the 9.4 / 9.6 displacement rules, and a rewind to before the end reserves him again from the persisted chain (reusing its names, reactivating retired stand-ins). Before this, the code never compared `ReservedUntilUT` with the clock and a Recovered kerbal stayed reserved forever (todo KERBAL-RESERVATION-NEVER-LIFTS-WITH-TIME). Open-ended (Aboard / Unknown) holds are not ended by time; a recovery bounds them (next note).

Implementation note: a death follows the stock crew respawn (owner ruling S8, 2026-09-26). Stock KSP marks a killed kerbal Dead and, only when `Difficulty.MissingCrewsRespawn` is on, Missing until death + `Difficulty.RespawnTimer`, then Available (decompiled `Part` death path, `ProtoCrewMember.StartRespawnPeriod` / `CheckRespawnTimer`). Parsek follows the policy AS IT STOOD AT THE DEATH: `KerbalsModule.PopulateCrewEndStates` stamps it on the recording the first time the end states record a death (`Recording.CrewDeathRespawns` / `CrewDeathRespawnSeconds`, persisted as the sparse keys `crewDeathRespawn` / `crewDeathRespawnSec`), keeps an existing stamp on every re-inference, and the optimizer split / merge carry it with the end states. The ledger `KerbalAssignment` rows are re-derived from the recording on load, which is why the recording and not the row holds it. The walk turns a Dead row whose recording is stamped ON into a temporary hold ending at the recording EndUT + the stamped timer (`KerbalReservation.DeathRespawnUT`); stamped OFF, or unstamped (deaths recorded before the stamp existed), stays permanent, and so does a death merged with an open-ended co-row that starts before the respawn (9.5); a looped segment elsewhere in the chain changes nothing (the loop is visual only, operator ruling 2026-09-27, which retired the looping-chain-death rule of 2026-09-26); the Timeline's respawn wording follows that resolved hold. While the respawn is pending the kerbal reads Lost everywhere (`KerbalsModule.IsLossHold`) and no stand-in covers him (9.5, as stock leaves a missing kerbal unreplaced); the 9.3 time release frees him at the respawn instant. A LATER flight of the same kerbal that outlasts the respawn extends the merged hold past it (9.2: one continuous block); the loss is then decided against the walk clock (`KerbalsModule.IsLossHoldAt`, trace item G6, fixed 2026-09-27): before the respawn he reads Lost until the respawn date with no stand-in, from the respawn to the later flight's end the hold is an ordinary reservation with a stand-in. The walk records that answer on the rebuilt reservation (`KerbalReservation.LossAtWalkClock`) so the clock-free `IsLossHold` readers agree, every Lost date reads `DeathRespawnUT` (not the hold's end), and `ComputeNextReleaseUT` also returns a respawn still ahead of the clock so crossing it recalculates. A future death (after a rewind) holds him from now until its respawn, and a tombstoned death row takes its respawn window with it. Parsek never writes rosterStatus: after the respawn the kerbal is whatever stock left him (Available).

Implementation note: a recovery closes an open-ended hold (2026-09-23). A STRANDED (Aboard) or Unknown hold is closed by the recovery of the real vessel that continues the flight, which is the "rescue recording closes it" rule for the case the recording itself cannot express: with auto-merge on, stock's in-flight Recover commits the flight at the scene change BEFORE it recovers the vessel, so the committed flight ends Landed with the crew aboard, and committed recordings are never re-stamped. At `onVesselRecovered` Parsek writes one `KerbalRecovered` ledger action per recovered kerbal (owner name after the stand-in reverse map; UT = the recovery; recordingId = the committed recording the vessel continues, identified by a POSITIVE launch-guid match or as the vessel Parsek spawned from it). The reservation walk ends that kerbal's Aboard / Unknown holds from the owner's tree that ended by the recovery at the recovery UT, so the block becomes UT 0 → recovery UT and the 9.3 release rule above frees him; it is derived like every other hold, so a rewind to before the recovery reserves him again. Holds from other missions, later flights (the max-end merge stands) and permanent (Dead) holds are untouched, and a hold whose vessel is never recovered stays open. A re-fly that supersedes the owner recording tombstones the action with it. Parsek's own programmatic recoveries (crew-suppressed) and rewind strips write nothing. Todo KERBAL-ABOARD-RESERVATION-OUTLIVES-THE-REAL-VESSEL.

Implementation note: a flight retired at KSC frees its crew at its end (operator ruling 2026-09-23). A recording whose vessel ENDS its flight parked in the KSC exclusion zone is retired and never becomes a real vessel (flight-recorder design 13.1), so nothing can ever recover it. Its aboard crew are freed at the recording's EndUT as if recovered: `KerbalsModule.PrePass` marks every committed recording for which `VesselSpawner.IsKscRetiredFinalFlight` holds (the SAME final-segment notion as the spawn side, `GhostPlaybackLogic.IsFinalSpawnSegment`, plus the same position predicate) and adds, per Aboard / Unknown `KerbalAssignment` row of that recording, an IN-MEMORY closure owned by it at its EndUT (`KerbalsModule.CollectKscRetirementClosures`). The existing closure rule above then bounds the kerbal's open-ended holds from that recording and from the earlier segments of the same tree that ended by then; a later flight and another mission keep their holds. No ledger row is written and no recovery funds are paid: the ledger stays exactly as recorded, and the reservation is re-derived on every walk, so a rewind to before the end reserves the kerbal again. Mirror guard: a recording that is materialized as a real vessel (spawned or adopted, spawned pid non-zero) or

whose recorded launch still exists in the save (the real counterpart still sits on the pad; the spawn path would adopt it rather than retire it) is not treated as retired, so a kerbal aboard a real vessel is never freed by this rule. The crew side and the spawn side read the same position: a recording the spawn side has already settled as a KSC retirement is taken as is, and otherwise the evaluation re-hydrates a dropped vessel snapshot exactly as the spawn gate does (only when the trajectory endpoint lies in a circle). The live-counterpart probe here is silent (no relaunch-rejection log per walk).

Implementation note: a kerbal who leaves through a split is handed off, not lost (2026-10-22). A kerbal can leave a vessel mid-flight inside another piece of it: a crew cabin with no command module decoupled or undocked with him inside, or any split child he moved into before the split. He is in the parent recording's start crew and missing from its end crew, which on an intact flight used to read Dead (a permanent loss, though he was alive in the cabin). The rule now: a kerbal in the start crew of a child of one of the recording's own split branch points (undock, EVA, joint break, breakup; read from the child's start snapshot or its EVA kerbal, stand-ins mapped back to the owner) and missing from the recording's end crew is not Dead there; his row reads Unknown and its hold ends at the split, where the child's recording takes him over (`KerbalsModule.ApplySplitDeparture`, `CollectCrewCarriedAwayBySplit`, `ResolveMidFlightSplitHandoffUT`). The child's own end state then decides his fate: recovered frees him at the recovery, aboard keeps the open-ended hold, a death in the child is a death. A parent destroyed after he left does not kill him. The rule holds across the recording's chain: when the flight he left is later cut into segments, a later segment reads the split of an earlier one the same way. A kerbal still aboard at the parent's end, or one who left without a split child carrying him, reads as before.

Implementation note: another flight's claim hands an open-ended hold over (2026-10-17, cargo adoption 2026-10-23). A kerbal left aboard a station, or standing on EVA, by a committed flight is held open-ended (Aboard / Unknown), and reservations merge by max end, so only something that ends that hold frees him. Besides a same-tree recovery, a split and a KSC retirement, a later flight in ANOTHER tree that claims his vessel ends it: a Dock or a Board into that vessel, or an EVA kerbal of the later flight moving cargo into or out of that vessel's (or that EVA kerbal's own) inventory, which adopts it as an `InventoryTransfer` member recording (flight-recorder design 4.12). The hold ends at the earliest such claim whose merged child (for a cargo adoption, the member recording) is the same launch of his vessel and carries him (`KerbalsModule.CollectDockClaims` / `ResolveDockClaimHandoffUT`); from there the claiming flight's own rows hold him until its own end. A claim on another vessel, another launch of the same craft, or one that does not carry him leaves the hold as it was. Derived like every hold: discarding or superseding the claiming flight brings the earlier hold back. Log line `Reservation bounded by cross-tree dock handoff:`. A later flight that records the vessel ITSELF claims it the same way at that recording's start (2026-10-24): a recording whose vessel joined its tree instead of coming out of it (the tree's root, a parentless recording, or the child of a post-switch Launch branch point, e.g. after a `]` switch to the station), same launch or a Spawn-linked guid, carrying him, in another tree only (`KerbalsModule.IsJoinedVesselRecording`); without it the earlier Aboard row stayed open, so a death on the later flight read as a permanent loss while stock respawned him. Log line `Reservation bounded by foreign recording of the vessel:`; todo GC6-FOREIGN-TREE-RECORDING-KEEPS-OPEN-HOLD.

9.4 Replacement system — temporary reservations

When a kerbal is temporarily reserved (RECOVERED or STRANDED), the roster would shrink without intervention. Parsek auto-generates a **free stand-in kerbal** to fill the slot.

Stand-in properties:

- Same class as the reserved kerbal (pilot/engineer/scientist).
- **Randomized** attributes (courage, stupidity, name). Not cloned from the original — this prevents roster homogenization over time.
- 0 XP (level 0), as with any new kerbal.
- Free — no funds cost. The reservation is a Parsek artifact, not a player choice, so Parsek covers the cost.

The stand-in exists for the duration of the reservation. When the original kerbal's reservation ends (recovered, or rescued from stranding), the original reclaims their slot:

- If the stand-in was **never used** in any recording → **deleted** from roster.
- If the stand-in **was used** in a recording → **retired** (unassignable, but remains in KSP's roster so recordings referencing them stay valid).

9.5 Replacement system — permanent reservations

When a **slot owner** is permanently reserved (DEAD or MIA), or dead and waiting for the stock crew respawn (S8 note in 9.3), Parsek does **not** auto-generate a replacement. A recorded death is a finite respawn-pending hold only when it was stamped with respawn on AND nothing keeps the kerbal committed across the dead window. A death merged with an open-ended co-row of the same kerbal (an un-closed Aboard / Unknown row) that starts BEFORE the respawn UT is a permanent loss even with respawn on (owner decision 2026-09-26; `ResolveOpenEndedRespawnDeaths` on the merged set, comparing the earliest open-ended co-row start carried

through the merge). A looped segment in the dead kerbal's chain no longer makes the death permanent (operator ruling 2026-09-27: the loop is visual only), so an overlapping open-ended hold can never hand a dead owner a stand-in. An open-ended co-row that starts at or after the respawn UT is a post-respawn reuse: the respawn stays intact and the later flight is an ordinary hold. Before the respawn instant such a merged hold (open-ended, or finite and ending after the respawn) still reads as a loss with no replacement; the stand-in appears only from the respawn on (G6, 9.3 note). The roster shrinks by one. The player must manually hire a new kerbal at KSC, at standard hiring cost, as a normal spending action on the timeline.

The hired kerbal is their own entity — not part of any replacement chain, just a new roster member. This matches vanilla KSP behavior: death has real consequences, the player pays to rebuild their crew.

Important distinction: if a **stand-in** dies within a temporary chain (the slot owner is temporarily reserved and will return), the chain continues. A new stand-in is generated to keep the slot filled, because the slot is still needed — the original will eventually reclaim it. The "no auto-replacement" rule only applies when the slot owner themselves is permanently gone.

```
Slot owner dies → no auto-replacement, roster shrinks, player hires manually.
Stand-in dies within temporary chain → chain continues, new stand-in generated.
```

Sandbox mode: kerbals are free and the player can create new ones at will. Permanent loss has no mechanical cost — the player simply creates a replacement.

Science mode: funds don't exist, so hiring cost doesn't apply. Permanent loss behaves like sandbox — the player creates a replacement kerbal for free. The roster shrink is cosmetic until the player acts.

Career mode: the full permanent-loss mechanics apply — roster shrinks, player pays hiring cost to replace.

9.6 Replacement chains

Temporary replacements form a per-slot chain. Each original kerbal owns a roster slot. Stand-ins fill the slot when the original is reserved.

```
Slot example:
  Owner: Jeb (Pilot)
  Chain: [Hanley, Kirrim]

  Jeb reserved UT=0→2000 (recording A, recovered).
  Hanley generated to fill slot. Free.
  Hanley reserved UT=0→800 (recording B, recovered).
  Kirrim generated to fill slot. Free.

  Active occupant at UT=0: Kirrim (deepest in chain, both above reserved).
  At UT=800: Hanley reclaims (Kirrim deleted if unused, retired if used).
  At UT=2000: Jeb reclaims (Hanley deleted if unused, retired if used).
```

Chain rules:

- Chains are per-slot, ordered by generation.
- The active occupant is always the deepest free kerbal in the chain.
- When a kerbal reclaims their slot, all deeper stand-ins are displaced (deleted or retired).
- Chains only exist for temporary reservations. Permanent loss exits the chain system.

9.7 Retired pool

Retired kerbals are stand-ins that were used in recordings and then displaced when their predecessor reclaimed the slot. They have status RETIRED:

- Remain in KSP's roster (recording reference integrity).
- Unassignable to new missions by Parsek.
- Do not count toward active roster size.

Retired status is **derived, not stored**. On rewind, all kerbal state is recomputed from scratch. A kerbal that was retired at UT=3000 may be reactivated as RESERVED at UT=500 if the timeline still has recordings using them.

The retired pool only grows from temporary-reservation stand-ins that were used in recordings. In practice, this means the pool grows slowly — only when a player uses a stand-in in a mission and the original later returns.

9.8 Kerbal action schemas

```
KerbalAssignment (recording-associated action — per kerbal per recording)
  recordingId:    string
  kerbalName:     string
  role:           PILOT | ENGINEER | SCIENTIST
  startUT:        float — mission start
  endUT:          float or NULL — recovery/death/MIA UT, NULL if stranded
  endState:       RECOVERED | DEAD | MIA | STRANDED
  xpGained:       float — XP earned during this recording

KerbalHire (spending action — career only)
  ut:             double — frozen KSC time
  sequence:       int — order within that UT
  kerbalName:     string
  cost:           float — funds spent
  role:           PILOT | ENGINEER | SCIENTIST

KerbalRescue (recording-associated action)
  recordingId:    string — the recording that rescued this kerbal
  kerbalName:     string — the rescued kerbal's name
  role:           PILOT | ENGINEER | SCIENTIST
  recoveryUT:     float — when the rescue recording recovered

KerbalRecovered (ledger action written at a real-vessel recovery; implementation 2026-09-23)
  ut:             double — the recovery instant
  recordingId:    string — the committed recording the recovered vessel continues
  kerbalName:     string — the recovered kerbal (reservation owner's name)
  role:           PILOT | ENGINEER | SCIENTIST
  effect:         ends the kerbal's STRANDED / Unknown holds from that recording's tree
                  that ended by the recovery, at ut

VesselRecovered (ledger action written at a player recovery of a real vessel; owner rulings 2026-10-03)
  ut:             double - the recovery instant
  recordingId:    string - the committed recording the recovered vessel continues (one row per owning tree)
  vesselName:     string - the recovered vessel's name (diagnostic)
  vesselPid:      uint - the recovered live vessel's persistentId (diagnostic)
  effect:         spawn evidence only: RecoveredRecordingEvidence reads the recording as
                  recovered, so its vessel is never spawned again; moves no pool

ResourceScanUnlocked (ledger action written when a resource scan unlocks a body or biome; owner ruling 2026-10-06)
  ut:             double - the unlock instant
  recordingId:    string - the live recording that made the scan; empty at the Space Center
  scanBody:       string - the scanned body's name (resolved to the stock planet id at patch time)
  scanBiome:      string - the biome a surface scanner unlocked; absent for a whole-body survey
  effect:         the state patcher unlocks the body / biome in ResourceMap once the clock
                  reaches ut, add-only; moves no pool

ScienceLabEarning (ledger action written when the science lab pays out stored science)
  ut:             double - the payout instant
  recordingId:    string - the live recording; empty at the Space Center (a recovery there)
  scienceAwarded: float  - the pool credit (R&D science after minus before the stock call)
  method:         Transmitted | Recovered
  effect:         credits the science pool uncapped when the walk reaches ut

KerbalStandIn (generated by Parsek, not a player action)
  kerbalName:     string — procedurally generated
  role:           PILOT | ENGINEER | SCIENTIST
  replacesKerbal: string — who this stand-in fills for
  courage:        float — randomized
  stupidity:      float — randomized
```

9.9 XP accumulation

KSP's XP system is achievement-based: a kerbal earns XP for flyby, orbit, landing, and flag planting on each celestial body, but only once per body per achievement type. Repeat visits to the same body yield no additional XP. This deduplication is handled natively by KSP — the ledger stores whatever KSP reported as `xpGained`, same pattern as `scienceAwarded`.

XP banks on recovery only. A kerbal must return to Kerbin for XP to be credited. A stranded kerbal accumulates achievements during their mission but does not receive the XP until a rescue recording recovers them. The `xpGained` field on a `KerbalAssignment` with endState=STRANDED is 0 until a subsequent rescue recording resolves it.

Because the reservation system enforces strictly sequential usage (no overlapping missions), XP accumulates in chronological order with no conflicts. The ledger walks all `KerbalAssignment` entries for a kerbal, sorted by UT, and sums `xpGained`. The final XP is the total across all recordings.

9.10 Kerbal existence timeline

Not all kerbals exist from UT=0:

- **Default crew** (Jeb, Bill, Val, Bob): exist from UT=0.
- **Hired kerbals**: exist from their hire UT onward.
- **Rescued kerbals**: exist from the recovery UT of the recording that rescued them. Rescue contracts spawn a stranded kerbal in orbit or on a surface; completing the rescue and recovering adds them to the roster for free (no hiring cost). This is a recording-associated action — the rescued kerbal's appearance is tied to the recording that brought them home.
- **Stand-in kerbals**: exist from the moment they're generated (the commit that triggered the reservation).

A kerbal cannot be assigned to a recording that starts before their existence UT. This is naturally enforced: hired kerbals don't appear in the roster until after the hire action, rescued kerbals don't appear until their rescue recording recovers, and stand-ins don't appear until they're generated.

9.11 Verified scenarios

Simple reservation and return:

```
Starting roster: Jeb (P), Bill (E), Val (P), Bob (S). Size = 4.

COMMIT Recording A: Jeb, UT=1000→2000 (recovered).
  Jeb reserved UT=0→2000. Generate Hanley (P, random attrs). Free.
  Active: [Hanley, Bill, Val, Bob] = 4 ✓

FAST-FORWARD to UT=2100:
  Jeb free. Reclaims slot. Hanley unused → DELETE.
  Active: [Jeb, Bill, Val, Bob] = 4 ✓
```

Deep chain, all stand-ins used:

```
COMMIT Recording A: Jeb, UT=1000→3000 (recovered).
  Jeb reserved. Generate Hanley (P).
  Active: [Hanley, Bill, Val, Bob] = 4 ✓

COMMIT Recording B: Hanley, UT=500→2500 (recovered).
  Hanley reserved. Generate Kirrim (P).
  Active: [Kirrim, Bill, Val, Bob] = 4 ✓

COMMIT Recording C: Kirrim, UT=200→2000 (recovered).
  Kirrim reserved. Generate Dunford (P).
  Active: [Dunford, Bill, Val, Bob] = 4 ✓

FAST-FORWARD to UT=2100:
  Kirrim free. Reclaims from Dunford. Dunford unused → DELETE.
  Active: [Kirrim, Bill, Val, Bob] = 4 ✓

FAST-FORWARD to UT=2600:
  Hanley free. Reclaims from Kirrim. Kirrim used → RETIRED.
  Active: [Hanley, Bill, Val, Bob] = 4 ✓

FAST-FORWARD to UT=3100:
  Jeb free. Reclaims from Hanley. Hanley used → RETIRED.
  Active: [Jeb, Bill, Val, Bob] = 4 ✓
  Retired: [Kirrim, Hanley]
```

Stranded then rescued:

```
COMMIT Recording A: Jeb, UT=1000→STRANDED on Mun.
  Jeb reserved UT=0→INDEFINITE (open-ended temporary).
  Generate Hanley (P). Free.
  Active: [Hanley, Bill, Val, Bob] = 4 ✓
```

```
COMMIT Recording B: Rescue mission, picks up Jeb at UT=3000, recovers UT=3500.
  Jeb's reservation updated: UT=0→3500.
  Active: [Hanley, Bill, Val, Bob] = 4 ✓ (Jeb still reserved until 3500)

FAST-FORWARD to UT=3600:
  Jeb free. Reclaims from Hanley. Hanley used → RETIRED.
  Active: [Jeb, Bill, Val, Bob] = 4 ✓
```

Rewind recomputes everything:

```
State at UT=3100 from deep chain scenario:
  Active: [Jeb, Bill, Val, Bob] = 4
  Retired: [Kirrim, Hanley]

REWIND to UT=500:
  Recalculate all reservations from ledger:
    Jeb: reserved UT=0→3000 (recording A)
    Hanley: reserved UT=0→2500 (recording B)
    Kirrim: reserved UT=0→2000 (recording C)
  All three reserved at UT=500. Need stand-in.
  Hanley reactivated as RESERVED (was retired).
  Kirrim reactivated as RESERVED (was retired).
  Generate new Dunford (P) for active slot.
  Active: [Dunford, Bill, Val, Bob] = 4 ✓
  Retired: [] (all reserved, none retired at this UT)
```

Multi-crew mission:

```
COMMIT Recording A: Jeb + Bill, UT=1000→2000 (both recovered).
  Jeb reserved UT=0→2000. Generate Hanley (P).
  Bill reserved UT=0→2000. Generate Derrick (E).
  Active: [Hanley, Derrick, Val, Bob] = 4 ✓

FAST-FORWARD to UT=2100:
  Jeb reclaims from Hanley. Hanley unused → DELETE.
  Bill reclaims from Derrick. Derrick unused → DELETE.
  Active: [Jeb, Bill, Val, Bob] = 4 ✓
```

Stand-in dies within a temporary chain:

```
COMMIT Recording A: Jeb, UT=1000→2000 (recovered).
  Jeb reserved UT=0→2000. Generate Hanley (P).
  Active: [Hanley, Bill, Val, Bob] = 4 ✓

COMMIT Recording B: Hanley, UT=500→700 (DEAD).
  Hanley reserved UT=0→INDEFINITE. But Jeb's slot still needs filling.
  Chain continues: generate Kirrim (P).
  Chain: Jeb → [Hanley (dead), Kirrim]
  Active: [Kirrim, Bill, Val, Bob] = 4 ✓

FAST-FORWARD to UT=2100:
  Jeb free at UT=2000. Reclaims slot.
  Kirrim unused → DELETE.
  Hanley dead + used in recording B → RETIRED (dead).
  Active: [Jeb, Bill, Val, Bob] = 4 ✓
  Retired: [Hanley (dead)]
```

The slot owner (Jeb) is temporary, so the chain continues even though the stand-in (Hanley) died. This contrasts with permanent loss of a slot owner:

Permanent loss — slot owner dies:

```
COMMIT Recording A: Jeb, UT=1000→1200 (DEAD).
  Jeb reserved UT=0→INDEFINITE. Permanent.
  No auto-replacement. No chain. Roster shrinks.
  Active: [_, Bill, Val, Bob] = 3.

Player hires Wehrner (P) at KSC. Spending action: -25000 funds.
```

```
Wehrner is independent — not in any chain.
Active: [Wehrner, Bill, Val, Bob] = 4 ✓
```

9.12 Dismissal rules

In vanilla KSP, the Astronaut Complex allows the player to dismiss (fire) kerbals from the roster. With Parsek's reservation and chain system, dismissal interacts with several kerbal categories differently.

Parsek prevents dismissal of any kerbal it manages. Reserved kerbals, active stand-ins, and retired kerbals are all protected from dismissal. The player can only dismiss kerbals that are completely outside Parsek's tracking — regular unassigned crew with no recordings and no chain involvement.

Rationale by category:

- **Reserved kerbals**: their reservation is a system invariant. Dismissing them would break recording reference integrity.
- **Active stand-ins**: they exist to maintain roster stability. Dismissing one would leave a slot empty, defeating the purpose of the replacement system.
- **Retired kerbals**: they exist in the roster for recording reference integrity. Ghosts are raw GameObjects rendered from sidecar data and don't need live roster entries for playback, but the roster entry preserves the kerbal's identity in KSP's crew system. Blocking dismissal is the safe default for v1.

9.13 Astronaut Complex capacity

The Astronaut Complex building level caps how many kerbals the player can hire (Level 1: 5, Level 2: 12, Level 3: unlimited). This cap only restricts player-initiated hiring. In vanilla KSP, rescued kerbals bypass the cap.

Parsek's system-generated kerbals (stand-ins and retired) also bypass the Astronaut Complex cap. They are not player hires — they are Parsek's internal roster management. The cap only governs what the player can do at the hiring screen.

This means the total number of kerbals in KSP's roster can exceed the Astronaut Complex cap due to stand-ins and retired kerbals. This is consistent with vanilla behavior (rescued kerbals already exceed the cap) and should not cause KSP issues.

9.14 Open questions

- **Rescue mechanics:** How does the ledger associate a rescue recording with a stranded kerbal? The rescue recording would need to reference the stranded kerbal by name and provide the recovery UT. This likely requires the recording system to detect when a vessel docks with or picks up a stranded kerbal.
- **KSP's MIA respawn:** KSP can respawn MIA kerbals after a configurable delay. Should Parsek model this, or treat MIA as permanently gone and let KSP handle the respawn outside the ledger? RESOLVED (owner ruling S8, 2026-09-26): a recorded death follows the stock respawn policy stamped at the death; see the 9.3 implementation note.
- **Stand-in naming:** Should Parsek use KSP's procedural name generator, or maintain its own? KSP's generator avoids name collisions with existing roster members. RESOLVED: stand-in names are persisted in KERBAL_SLOTS ConfigNode and reused across recalculations.
- **Sandbox mode:** The reservation system applies (prevents duplicate kerbals), but replacement is trivial (free, unlimited). Does sandbox need the chain system at all, or just the reservation check? RESOLVED: sandbox uses the full chain system (prevents duplicate kerbals regardless of game mode).
- **Retired kerbal cleanup:** Should there be a mechanism for the player to acknowledge and permanently remove retired kerbals from the roster (beyond blocking dismissal)? The retired pool could grow over a long career.

15.15 Implementation Architecture (Post-Build)

The kerbals module participates in the unified recalculation walk as a registered first-tier `IResourceModule`. `KerbalsModule` implements `IResourceModule` and is registered at `RecalculationEngine.ModuleTier.FirstTier` in `LedgerOrchestrator.Initialize` (the migration tracked as T42 is complete). Its `ProcessAction` consumes `KerbalAssignment` actions during the walk, and its `PostWalk` builds the replacement chains and computes the retired stand-in set. It also reads crew presence from `RecordingStore.CommittedRecordings` (vessel snapshots) to drive reservation, because kerbal identity is anchored to which crew are on which vessel, not to discrete actions alone.

Why: Kerbal reservation depends on crew presence in vessel snapshots (which crew members are on which vessel), not on discrete game actions alone. `KerbalAssignment` actions are generated at commit time via `LedgerOrchestrator.CreateKerbalAssignmentActions` (from crew snapshot data) and are consumed by `KerbalsModule.ProcessAction` in the walk. `KerbalHire` IS produced by `GameStateEventConverter` and consumed by `FundsModule`

(for hiring cost deduction). `KerbalRescue` and `KerbalStandIn` action types exist in the `GameActionType` enum with full serialization, reserved for future use.

Chain and reservation system: Operates as designed (UT=0 reservation, replacement chains, retired pool, dismissal protection). Fully integrated into the engine walk as a first-tier IResourceModule (T42). ProcessAction consumes KerbalAssignment actions; PostWalk builds chains and retired set; ApplyToRoster called by orchestrator after the walk.

---

12. Facilities Module

Status: Designed — spending schemas, visual state management, and destruction/repair lifecycle specified.

Game mode applicability: Not applicable in sandbox (all facilities max level, indestructible). Not applicable in Science mode (facilities are not upgradeable, and destruction/repair has no funds cost). Active in Career mode only.

KSC facilities are buildings with discrete levels that gate player capabilities. The facility module tracks upgrade costs, destruction events, and repair costs for funds accounting, and manages building visual state during fast-forward replay.

10.1 KSP's facility system

KSP has approximately 9 facilities, each with 3 levels (1 = basic, 2 = upgraded, 3 = fully upgraded). Key facilities and what they gate:

- **Launchpad / Runway**: vessel size and mass limits.
- **VAB / SPH**: part count limits.
- **Tracking Station**: tracking range, patched conics detail.
- **Astronaut Complex**: roster capacity, EVA capability.
- **Mission Control**: active contract slot limits.
- **R&D**: tech node access, science experiment availability.
- **Administration**: strategy slot limits.

Upgrading costs funds. Buildings can also be destroyed (player crashes into KSC) and repaired (costs funds).

10.2 Facility action schemas

```
FacilityUpgrade (spending action)
  ut:           double - frozen KSC time
  sequence:     int - order within that UT
  facilityId:   string - "LaunchPad" / "VehicleAssemblyBuilding" / etc.
  toLevel:      int - the facility's level from this UT on (2 or 3; 1 or 2 for a downgrade)
  facilityCost: float - the funds stock deducted for it (code field: FacilityCost)
  facilityDowngrade: bool, sparse - true when the row LOWERS the level (code field: FacilityDowngrade)

FacilityDestruction (recording-associated action, or a direct KSC row)
  ut:           double - when the building collapsed (stock OnKSCStructureCollapsing)
  recordingId:  string - the recording live at the collapse; null when none was
  facilityId:   string - the DestructibleBuilding id ("SpaceCenter/LaunchPad/Facility/<part>")

FacilityRepair (spending action)
  ut:           double - frozen KSC time (stock OnKSCStructureRepairing)
  sequence:     int - order within that UT
  facilityId:   string - the DestructibleBuilding id
  facilityCost: float - this building's share of the funds stock deducted (code field: FacilityCost)
```

Upgrades and repairs are KSC spending actions (frozen UT, sequenced). A destruction during a recorded flight is recording-associated: it becomes a ledger row when that recording commits, is dropped with the recording on a revert discard, re-homed as a direct row on a non-revert discard (stock has already saved the building down), and retired by a Re-Fly that supersedes the flight. A destruction with no live recorder (before a flight's recording starts, or the KSC debug Demolish) is a direct row at its UT. A KSP load to before a direct row prunes it like any other KSC action; a Parsek rewind keeps it as a future row that the walk applies only once the clock passes it.

Destruction and repair are keyed by the single DestructibleBuilding id; a facility is several buildings, and it is destroyed while any of them is. One stock repair (`SpaceCenterBuilding.RepairFacility`) repairs every destroyed building of the facility and deducts one `FundsChanged(StructureRepair)` total (`RepairCost` summed over the destroyed buildings, times `Career.FundsLossMultiplier`); Parsek writes one FacilityRepair per building carrying its share, as one batch, so the KSC reconciliation sums them against the

single debit. That FundsChanged event is not converted to a funds row by any other path, so the repair row is the one place the spend is counted. Upgrading a destroyed facility repairs it for free (stock `ResetStructures`), recorded as zero-cost repair rows, written in one batch with the upgrade row.

One stock upgrade (`SpaceCenterBuilding.UpgradeFacility`) deducts one `FundsChanged(StructureConstruction)` (`upgradeLevels[level + 1].levelCost` times `Career.FundsLossMultiplier`, nothing in a game without funds) BEFORE `SetLevel` raises the event the upgrade row is recorded from. Parsek reads that debit as the funds handler observes it inside the call, so the row carries what the pool actually lost (net of a strategy discount, whose funds share no strategy row records for this event-derived reason) and the KSC reconciliation pairs it with that one event. A level change no `UpgradeFacility` call observed (the scene-change poll) carries cost 0 rather than a guessed one. Rows written at cost 0 before this capture take their debit on load when the saved `FundsChanged(StructureConstruction)` event of the row's own tag and UT still proves it, and are left as stored otherwise.

A paid downgrade is the same kind of row. Stock's KSC facility menu, opened with Left Ctrl held on an out-of-service facility above level 1 (outside Mission mode), shows a "Rebuild lvl N" button that calls the private `SpaceCenterBuilding.DowngradeFacility(Funding.Instance != null)`: one `FundsChanged(StructureConstruction)` debit of 0.667 times `upgradeLevels[level - 1].levelCost` times `Career.FundsLossMultiplier`, the free `ResetStructures`, then `SetLevel(level - 1)`. The same capture scope, opened in downgrade mode by a prefix on that method, stamps the FacilityDowngraded event with the observed debit, and the event becomes a FacilityUpgrade row to the lower tier marked `facilityDowngrade` (written in one batch with the free repairs, or at commit when tagged). The row type is the upgrade's because every reader of it reads "the facility is at tier `toLevel` from this UT on, for `facilityCost`": the facility walk assigns the level, the funds walk charges the cost, the KSC reconciliation pairs it with the StructureConstruction debit, and the tombstone, patch and slot readers key on the facility and the level. The marker is read where the word "upgrade" would be wrong: the description ("Downgrade X → Lv.N"), the Timeline's legacy-event twin (FacilityDowngraded), the stock-screen reservation index (a Rebuild row is indexed as a facility level change like an upgrade, its `CommittedFutureEntry.FacilityDowngrade` giving the direction, which the slot forecasts read to set the limit to the lower level's and the reservation text reads to say `Rebuilt to level N`), and the legacy cost-0 upgrade repair and estimate (never applied to a downgrade). A committed Rebuild therefore holds the facility's Upgrade and Rebuild buttons until it passes, as a committed upgrade does (owner ruling 2026-10-07, section 14.4). A level drop no `DowngradeFacility` call made (the scene-change poll, another mod, Mission mode's facility limits) stays an informational event with no row.

10.3 Funds accounting

All three action types participate in the unified funds walk:

- **Upgrade**: deducts funds at the upgrade UT (a downgrade row the same way, at its UT).
- **Repair**: deducts funds at the repair UT.
- **Destruction**: no direct funds cost (the destruction itself is free; the cost comes from the subsequent repair). A new flight's destruction that makes a committed repair chargeable needs no rule of its own: the repair cost falls under the reserved floor and debt of section 3.13 (owner ruling 2026-10-21).

10.4 Visual state management

Building visual state updates **in real time during warp**, following the same pattern as ghost vessel playback (see section 5.3). As the warp timeline crosses facility events, Parsek sets building visuals immediately — the player sees buildings upgrade, get destroyed, and get repaired at the correct UT. This is purely visual; KSP's actual facility state is patched on warp exit.

```
For each facility, walk events in UT order:
  UPGRADE:     set visual to new level
  DESTRUCTION: set visual to destroyed
  REPAIR:      set visual to pre-destruction level
```

Between a destruction and its repair, the building is visually destroyed. This window can be any duration — the player repairs whenever they choose.

```
UT=500:  Upgrade launchpad to level 2       visual: level 2
UT=1000: Destruction (recording A crash)    visual: destroyed
UT=1000 to UT=1100: launchpad visually destroyed
UT=1100: Repair                             visual: level 2
UT=2000: Upgrade to level 3                 visual: level 3
```

Ghost vessels whose recordings include launches during the destroyed window still replay normally — ghosts are rendered from sidecar trajectory data and don't interact with KSP's facility system.

On warp exit, Parsek patches KSP's actual facility state to match the derived state at the current UT (see section 5.2).

Live building patch contract (as built). Facility LEVELS are patched from the walk. Building destroyed / intact state is not: a walk with no UT cutoff (commit, scene load, a cold load before the clock is ready, warp start) contains rows dated after now. `FacilityStatePatcher.PatchLiveDestructionState` instead folds the effective ledger, per DestructibleBuilding, to the last FacilityDestruction / FacilityRepair row at or before live UT, and acts only where that row contradicts the live building: demolish an intact building whose last row is a destruction, repair a destroyed one whose last row is a repair. A building with no row at or before now is never touched (nothing is restored from the absence of a row: stock's own state travels with every save, rewind and revert, and the ledger cannot tell a building stock restored from one it should restore), a building mid-collapse or mid-repair is left to stock's animation, and nothing is patched while a flight is recording, its tree is uncommitted or pending (that flight's collapses are not in the ledger yet), or before the universe clock is ready, or while `ParsekScenario.OnLoad` is on the stack (it runs before stock loads the save into the buildings, with the pre-load clock); a contradicting row after a scene-change load therefore waits for the next recalc. A building not yet registered with the current `ScenarioDestructibles` is skipped too, since until registration it reads its default intact state.

10.5 Derived facility level

The facility level at any UT is derivable by walking the action history: start at default level (1), apply upgrades, destructions, and repairs in order. During warp, this drives the visual state. On warp exit, the full recalculation produces the final derived level, which is patched into KSP's state.

10.6 Open questions

- **Destruction detection:** How does Parsek detect building destruction during a recording? RESOLVED: `GameStateFacilityRecorder` subscribes to the synchronous `GameEvents.OnKSCStructureCollapsing` / `OnKSCStructureRepairing` (fired inside `DestructibleBuilding.Demolish` / `Repair`), plus a Harmony scope on `SpaceCenterBuilding.RepairFacility` for the cost and a `ResetStructures` hook for the free repair inside an upgrade. The scene-load `IsDestroyed` poll remains a same-instance safety net only (a fresh recorder seeds its cache and polls in the same call).
- **Partial destruction:** Can individual buildings be destroyed independently, or does KSP group them? RESOLVED: independently - each `DestructibleBuilding` collapses on its own, and the actions are keyed per building (see 10.2).
- **CustomBarnKit interaction:** CustomBarnKit modifies facility costs and progression tiers. Does the ledger need to account for non-standard level counts or costs, or does it just record what KSP reports?

---

13. Strategies Module

Status: Designed — time-windowed transforms, reservation system, and interaction with contract rewards specified.

Game mode applicability: Not applicable in sandbox or Science mode (no Administration building). Active in Career mode only. Strategies are lightly used by most players but affect resource balances when active, so Parsek must track them for correctness.

11.1 KSP's strategy system

Strategies are policies set in the Administration building that divert a percentage of one resource earned from contracts into another resource. There are eight stock strategies, all following the same pattern:

- **Source resource**: the resource being diverted from (funds, science, or reputation).
- **Target resource**: the resource being diverted to.
- **Commitment slider**: 1% to 25%, controls the diversion percentage.
- **Setup cost**: a one-time cost in the source resource, paid at activation.

Strategies only transform **contract rewards** — not milestone bonuses, not experiment science, not vessel recovery funds. Milestones pass through unaffected.

Constraints:

- Limited active strategy slots (Administration building level: 1 at level 1, 3 at level 2, 5 at level 3).
- Conflicting strategies (same source resource) cannot be active simultaneously. KSP enforces this natively.
- Strategies are activated and deactivated at KSC (frozen UT).

11.2 Strategy action schemas

```
StrategyActivate (KSC action)
  ut:             double — frozen KSC time
  sequence:       int — order within same UT
  strategyId:     string — e.g. "UnpaidResearch" / "PatentsLicensing"
  sourceResource: enum — FUNDS | SCIENCE | REPUTATION
  targetResource: enum — FUNDS | SCIENCE | REPUTATION
  commitment:     float — 0.01 to 0.25 (slider percentage)
  setupCost:      float — one-time cost in source resource (IMMUTABLE)

StrategyDeactivate (KSC action)
  ut:             double — frozen KSC time
  sequence:       int
  strategyId:     string
```

The setup cost is deducted from funds by the `FundsModule` when processing `StrategyActivate` actions. **Note (v0.3): KSP stock strategies always cost funds for setup. Reputation-based setup costs are not implemented.**

11.3 Strategy blocks against the committed timeline

Correction (2026-09-25, stock-UI overlay program PR 5). An earlier draft specified a UT=0 reservation that blocked every new strategy activation while any strategy was on the timeline. No code ever implemented it (`StrategiesModule.GetAvailableSlots` had no caller), and it is not the rule. The shipped rule lives at the stock control (`StrategyReservationPredicates`, reference `docs/dev/research/stock-ui-reservation-overlays-2026-09-25.md` section 10 step 5):

- **Activation of S is refused** when the committed timeline activates S later, when activating S now leaves no free slot at a committed future activation, or when, with S still active at a committed activation of another strategy, stock's own conflict rule (`StrategySystem.HasConflictingActiveStrategies`, over group tags) would refuse that activation. The slot check is a peak walk: the strategies active in stock now plus S, then the committed future activations and deactivations in UT order (deactivations first at the same UT), with the limit raised at each committed Administration upgrade and set to the lower level's at a committed Rebuild; refuse when the count exceeds the limit at any committed activation.
- **Deactivation of S by the player is refused** while a committed StrategyActivate or StrategyDeactivate row for S is still ahead. No other committed row depends on S being active: contract rewards and strategy conversions are captured after stock applied the strategy and replay as recorded (section 11.4 note).
- **Stock auto-expiry is never refused.** Expiry exists only with KSPCommunityFixes' StrategyDuration fix; it goes through `Strategy.Deactivate()` gated on `CanBeDeactivated`, which Parsek leaves unpatched, and it is captured as a StrategyDeactivate row like any deactivation, filed under no flight because the clock caused it (section 3.14).

The original rationale (a new strategy could divert earnings that committed spendings downstream rely on) does not hold for stock strategies: the walk never re-derives a committed reward from the active strategy set, so a present-day activation or deactivation cannot change a committed row's value.

These predicates read the committed-future index shared by every stock-screen block (`StrategyReservationGate` over `CommittedFutureIndex`). How the Administration building shows them (stock's own reason line, greyed Accept / Cancel with a tooltip, stock-first precedence, the shared explanation text) is described with the other stock screens in section 14.

11.4 Transforms during recalculation

Note (as shipped, #439 Phase A): the walk does not apply this transform. Stock strategies transform a contract reward through `GameEvents.Modifiers.OnCurrencyModifierQuery` before the completion is captured, so the captured reward is already post-transform and `StrategiesModule.TransformContractReward` is a logged identity no-op; strategy currency conversions are captured as their own rows (StrategyScienceDebit / Credit, the StrategyConverter funds and reputation sources). The design below is kept for reference.

Planned (section 3.15 step 7).

- Today a conversion leg is applied as captured even when the walk zeroes its cause as a duplicate.
- Under the plan a leg captured for a known cause row carries that row's id, and the replay scales the leg by the cause's effective share.
- A leg with no positive evidence of a cause (a mod paying with a strategy reason) stays as captured.

During the recalculation walk, when a contract reward is processed, the walk checks which strategies are active at that UT. If a matching strategy is active (its source resource matches one of the reward types), the reward is transformed:

```
For each contract reward at this UT:
  For each active strategy at this UT:
    if strategy.sourceResource matches a reward type on this contract:
      diverted = rewardAmount × strategy.commitment
      rewardAmount -= diverted
      targetRewardAmount += diverted × conversionRate
```

Note (v0.3): `conversionRate` is hardcoded to 1.0 (deferred item D2). Actual KSP strategy conversion rates need to be extracted from strategy definitions.

The transform happens between the contracts first-tier resolution (which sets the effective flag) and the second-tier crediting (which adds effective values to funds/rep/science running balances). Only effective contract completions are transformed — non-effective duplicates produce zero rewards and therefore zero transforms.

The conversion rate (how much target resource you get per unit of diverted source resource) is defined by KSP's strategy configuration. The coding agent should extract these rates from KSP's strategy definitions.

Implementation mechanism: Strategy transforms write to `TransformedFundsReward`, `TransformedScienceReward`, and `TransformedRepReward` derived fields on the `GameAction` object — NOT the persisted fields. The `RecalculationEngine` resets these to the original values before each walk. Second-tier modules (`FundsModule`, `ReputationModule`) read the Transformed fields when processing `ContractComplete` actions.

11.5 What strategies do NOT affect

Strategies only transform contract rewards. The following are unaffected:

- **Milestone fund and reputation awards** — pass through at full value.
- **Science from experiments** — `scienceAwarded` is unaffected by strategies.
- **Vessel recovery funds** — full recovery value credited.
- **Contract advance payments** — paid at accept, not transformed.
- **Penalties** — contract failure/cancel penalties are not transformed.

11.6 Verified scenarios

Basic transform within active window:

```
Strategy "Unpaid Research" activated at UT=300 (10% REP→SCIENCE, setup cost: -5 rep).

UT=100: Milestone +10k funds, +15 rep.         (no transform — milestone)
UT=300: Strategy activates. Setup: -5 rep.
UT=400: Contract reward +40k funds, +50 rep.    → transformed: +45 rep, +5 science.
UT=600: Contract reward +30k funds, +30 rep.    → transformed: +27 rep, +3 science.
UT=700: Strategy deactivated.
UT=800: Contract reward +20k funds, +40 rep.    (no transform — outside window)
```

Strategy window and retroactive commit:

```
Strategy active UT=200 to UT=600.
Contract "Orbit Mun" completed at UT=300 (inside window). +50 rep → 45 rep + 5 science.

Retroactive commit: rec_B completes same contract at UT=100 (BEFORE window).
  UT=100: effective=true. +50 rep (full — outside window). +0 science.
  UT=300: effective=false (duplicate). +0 rep, +0 science.

Result: credit shifted to UT=100. No strategy diversion applied.
Rep goes UP (lost the 5 rep that was diverted). Science goes DOWN (lost the 5 science bonus).
Total resource value redistributed — not a paradox, just a different outcome.
```

A committed activation holds the slot it needs:

```
Admin building: 1 slot. Strategy A activated at UT=1000 (committed).

Player rewinds to UT=500. Wants to activate Strategy B.
  Peak walk: {B} at 500, {B, A} at 1000 -> 2 > 1. BLOCKED:
```

```
   "A committed activation of 'A' on <date> needs this slot."

With a committed Admin upgrade to level 2 (3 slots) at UT=800, the same
activation is allowed.
```

11.7 KSP state patching

After every ledger recalculation that patches KSP state, `KspStatePatcher.PatchStrategies` (`StrategyStatePatcher.cs`) makes `StrategySystem` match the walk's active set:

- A ledger-active, stock-inactive strategy is switched on through `Strategy.Load(ConfigNode)` with the committed activation date and commitment factor: no setup charge (the walk already charged it) and no `Activate()` call, so no capture.
- A stock-active strategy the ledger has inactive is switched off through `Unregister()` plus the private `isActive` flag: no refund, and no `Deactivate()` call (which KSPCF's minimum-duration gate could refuse).
- A strategy the ledger never activated (active before the save had a ledger) is left alone.
- An activation dated after now, or a strategy whose next committed row is a deactivation, is left as is: a recalculation without a current-UT cutoff walks past now, and those rows apply when the clock reaches them. Since 0.10.7 the patcher rebuilds the active set at the current clock from the committed rows first (`ResolveLedgerActiveAtNow`), and `StrategyActivationGate` switches a committed activation on in the frame the clock crosses its UT, so no recalculation has to come first; a one-time currency exchanger is never switched on again (section 3.14).
- Stock fills its strategy list one frame after `StrategySystem.OnLoad` (a coroutine), after the synchronous ksp-load recalculation. A patch that finds the list not loaded waits, and a postfix on the private `StrategySystem.LoadStrategies` runs it with the requesting walk's snapshot once the list exists (Career only; dropped by a newer recalculation, a scene change or another save).
- A strategy switched on after its `ActivateUT + LongestDuration` expires on stock's next tick (KSPCF) and records a real StrategyDeactivate row at now: benign and one-shot.

Conflict checking (group tags) is KSP-native, and the activation block mirrors it against committed activations. `Strategy.Load` bypasses it, so a ledger state stock's rule would not allow is applied as the ledger has it and warned once, never resolved by switching a strategy off.

11.8 Open questions

- **Conversion rates:** The exact conversion rate per strategy (how much target resource per unit of diverted source) needs to be extracted from KSP's strategy configuration files or decompiled code.
- **Strategy mods:** Popular mods like Strategia completely replace the stock strategy system. Should Parsek support modded strategies, or only stock? For v1, stock only is sufficient.
- **Player usage patterns:** Strategies are lightly used by most players. The module is designed and specified, but implementation priority may be lower than other modules.

---

14. Stock-Screen Reservation Layer

Status: Implemented (0.10.5). Plan, block audit and owner rulings: `docs/dev/research/stock-ui-reservation-overlays-2026-09-25.md`.

Game mode applicability: A block applies wherever its stock screen exists and the effective ledger holds a committed future row of its kind; most kinds (contracts, strategies, facility upgrades, hires, part purchases) exist only in Career. Parsek is inert in Mission and Scenario games (section 1.5).

Several reservations in this document protect committed history that lies AFTER the current UT: a tech node, part purchase, hire, facility upgrade or repair, contract accept or resolution, or strategy change that the committed timeline makes later, and a kerbal a committed flight holds. Repeating such an action now would apply or charge it twice when the clock reaches the committed row. Parsek refuses these actions at the stock control the player is about to click, and explains the refusal on that same control.

14.1 The committed-future index

Every stock-screen block and mark reads one index, `CommittedFutureIndex` (`CommittedFutureIndex.cs`), built over the effective ledger:

- **Committed.** A row counts when it is in the effective ledger (`EffectiveState.ComputeELS` : not tombstoned) AND either has no recording (a KSC-origin row, written the moment the player acted) or belongs to a recording in the Effective Recording Set (`EffectiveState.ComputeERS` : committed, not superseded, not session-suppressed). A row tagged to the live or pending tree or to a Re-Fly provisional reserves nothing: a flight still in progress never blocks a Space Center action.
- **Future.** A committed row is future when `row.UT > now` , strictly (`CommittedFutureIndex.IsFuture`). A row AT now is already applied, matching the walk's `UT <= cutoff` rule (section 3.8).
- **Keys.** Rows are grouped by kind and key and sorted by UT: tech id, facility id, destructible building id, contract guid (accept, complete, fail, cancel), kerbal name (hire, plus a retire kind read from committed `CrewRemoved` milestone events), strategy id (activate, deactivate), part name (entry-cost purchase) and part upgrade name (upgrade purchase). The index also keeps every committed `KerbalAssignment` , so a kerbal's explanation can name the flight that holds him (`ResolveHoldAssignment`).
- **Lifetime.** `CommittedFutureIndexCache.Current` rebuilds only when an input changes (the identity of the ELS / ERS lists, a milestone fingerprint, or an explicit `Invalidate` from `LedgerOrchestrator.OnTimelineDataChanged`). Every query takes `now` explicitly, so a block lifts as soon as the clock passes the last committed row, with no rebuild. When the clock cannot be read, `now` is UT 0, which over-blocks rather than under-blocks.

Kerbal holds themselves are the derived reservations of `KerbalsModule` (section 11); the index only supplies the committed assignments for their text.

14.2 The pairing rule and the stock-mechanism rule

- **Pairing.** For every clickable kind, the mark on the control and the click-block behind it read the SAME predicate over the SAME index instance (`StockUiReservationPredicates` , `StrategyReservationPredicates` , `StockUiDecorationQuery`), so they cannot disagree. A mark with no block would mislead; a block with no mark is a surprise. Each block also has a Harmony backstop on the stock method behind the control (for example `RDTech.UnlockTech` , `Contract.Accept` / `Decline` / `Cancel` , `SpaceCenterBuilding.UpgradeFacility` , `KerbalRoster.SackAvailable` , `FlightEVA.spawnEVA` , `CrewTransfer.Create`), so a call from anywhere else, Contract Configurator's Accept included, is refused with the same text.
- **Stock mechanisms only** (owner ruling D1, 2026-09-25; the stock-control annotation exception to the "no new player-facing UI surfaces" rule in `.claude/CLAUDE.md`). An annotation uses only what the stock control already has: its disabled / greyed state, text appended to its existing stock tooltip or description, stock's own reason field (`Strategy.CanBeActivated` reason, `CrewListItem.SetButtonEnabled` caption), the row's own status or label text, or a tint of its existing icon. A greyed button gets its reason on a stock tooltip (`StockUiReasonTooltip`), and gets its exact stock look back when the block lifts (`StockUiGreyedButton`). No Parsek-drawn box, badge, counter or panel appears on a stock screen; an informational mark (a committed completion on an Active contract, a lost or stand-in kerbal) lives only in the row's own label.
- **Stock first.** Where stock itself already refuses the action (no free slot, a strategy conflict, the cost), stock's result and reason stand and Parsek adds at most its dated fact.

14.3 One explanation text

`ReservationExplanation` builds one short sentence per reservation kind, and the control's hover, the greyed button's tooltip, the row label and the refused-click dialog (`CommittedActionDialog.ShowBlocked` , titled "Action Blocked") all show that same text. The sentence is the participle, the date and time (`KSPUtil.PrintDateCompact` with hour and minute), then "blocked by timeline until then.", e.g. `Researched on Y1, D06, 14:05, blocked by timeline until then.` or, for a kerbal, `Reserved by timeline for 'Mun Lander' until Y1, D09, 18:40.` It never says "committed" or "your timeline". Block reasons are drawn in stock's own reason orange. The Timeline window's row hover reads the same check to say which stock control a future row holds (`Holds Research in R&D until ...`).

14.4 Screen by screen

Screen	Refused while a committed row is ahead	Mark
R&D	Research of a node the timeline researches later; purchase of a part it buys later (with entry purchases on); Purchase-all skips such parts and names them, and is greyed when only they remain	Tinted node icon; the reason in the node tooltip and side-panel description; greyed Research / Purchase-all with a tooltip; the part tooltip greys its purchase button (also in the VAB / SPH part list), and the upgrade tooltip greys its purchase button for an upgrade the timeline buys later

Screen	Refused while a committed row is ahead	Mark
Mission Control	Accept and Decline of an offer the timeline accepts later; Cancel of an Active contract it completes, fails or cancels later; Accept of any other offer while the slot forecast says it would starve a committed accept (section 10.2)	Row label `- Accepted on Y1, D03` / `- Completed on Y2, D114`; the reason at the end of the detail panel; greyed buttons with tooltips. The slot block has no row mark by design
Administration	Activating a strategy the timeline activates later, one that would take a slot a committed activation needs, or one that would conflict with a committed activation; cancelling one the timeline deactivates or re-activates later (section 13.3)	Stock's own reason line in the description panel; greyed Accept / Cancel with a tooltip
Astronaut Complex	Hiring an applicant the timeline hires later; dismissing any kerbal Parsek manages (reserved, stand-in, retired; section 11.12)	The row's own status label (`Hired on ...`, `Reserved until ...`, `Lost`, `Retired`, `Stand-in for Bill Kerman`); stock's locked-with-reason button state. A stand-in's dismissal reads e.g. `Standing in for Bill Kerman, reserved by timeline for 'Mun Lander' until Y1, D09, 18:40.`
KSC facility menu	Upgrade and Rebuild (stock's Left Ctrl "Rebuild lvl N", a paid downgrade) of a facility while a later committed level change of it, an upgrade or a Rebuild, is ahead: levels are absolute, so an earlier level change would rewrite the recorded one (owner ruling 2026-10-07; lifts after the last one). Repair of a building whose current destruction a committed repair already covers, and Upgrade and Rebuild of its facility while that repair is ahead: both level changes run stock's free `ResetStructures` repair first, so they would repair the building now and rewrite the recorded repair	Greyed Upgrade / Rebuild / Repair with a tooltip (Rebuild only while stock shows it); the reason in the menu's description, one line per distinct reason. A level change alone reads `Upgraded to level 2 on ...` or `Rebuilt to level 1 on ...`, a covering repair alone `Repaired on ...`; mixed rows are each named with their own participle, earliest first
VAB / SPH crew dialog	Seating a kerbal the timeline reserves, has lost, or retired as a stand-in (`KerbalsModule.ShouldFilterFromCrewDialog`)	The kerbal is listed, not hidden, with stock's inactive-crew look, the status in place of his trait (`Reserved until Y1, D07`) and stock's locked-with-reason tooltip
Flight	EVA or crew transfer of a kerbal a committed flight holds, aboard a vessel that does not continue a committed flight; stands down during a Re-Fly session (`StockUiFlightCrewDecoration.IsMoveRefused`)	Greyed EVA on the portrait (reason in stock's EVA tooltip); greyed EVA / Transfer in the hatch dialog, with the status on the row's name label

Ruled 2026-10-21, built in 0.10.6 (section 3.13): Mission Control also greys Accept, Cancel and Decline when the reservation or penalty would leave a later committed cost unpaid (decoration kind `ContractReserve`, the reason in the detail panel), and every purchase, strategy setup and launch is greyed with the reason when it would spend reserved money.

The part-purchase and strategy blocks have a state half too: `KspStatePatcher.PatchPurchasedParts` marks a committed purchase purchased when the clock reaches it (no second charge), `KspStatePatcher.PatchUnlockedUpgrades` does the same for a committed part upgrade, and the strategy patch makes stock's active set follow the ledger (section 13.7).

15. Gameplay Simulation Findings (v0.4)

Edge cases and gaps discovered by simulating concrete KSP gameplay scenarios against the implementation.

15.1 ~~Critical Gaps~~ Resolved

~~**Vessel build cost not captured.**~~ FIXED (v0.4). `LedgerOrchestrator.CreateVesselCostActions` injects vessel build cost at commit time from `Recording.PreLaunchFunds` delta as a `FundsSpending` with `source=VESSEL_BUILD`.

~~**Vessel recovery funds not captured.**~~ FIXED (v0.4). `CreateVesselCostActions` also handles recovery — when `rec.TerminalStateValue == TerminalState.Recovered`, recovery funds are created from trajectory data.

~~**Science and reputation wiped on mid-career Parsek install.**~~ FIXED (v0.4). `Ledger.SeedInitialScience` and `Ledger.SeedInitialReputation` are called in `RecalculateAndPatch`, creating `ScienceInitial` and `ReputationInitial` seed actions alongside the existing `FundsInitial`.

~~**Contract science rewards not credited.**~~ FIXED. `ScienceModule.ProcessAction` dispatches `ContractComplete` to `ProcessContractScienceReward`, which credits `action.TransformedScienceReward` (the post-strategy-transform value) into effective earnings when `action.Effective == true` (the chronologically first completion gets the credit; later duplicate completions are skipped).

15.2 Edge Cases Discovered

~~**Facility destroyed state not patched.**~~ FIXED (v0.4). `KspStatePatcher.PatchDestructionState` is fully implemented — iterates `DestructibleBuilding` objects and calls `db.Demolish()` / `db.Repair()` to sync visual state with the ledger's derived destruction state.

Sequence numbers not assigned during event conversion. `GameStateEventConverter` creates actions with `Sequence=0` for all KSC events at the same UT. If a player unlocks two tech nodes at the same frozen KSC time, their relative order is undefined. Benign for independent spendings but could matter if one spending's affordability depends on the order. Planned fix: `cseq` as the tie key (section 3.15 step 2).

ContractAccept sorts with spendings, not earnings. `ContractAccept` is not in `IsEarningType` (it's neither earning nor spending in the classifier). At the same UT, the advance payment processes after explicit earnings. Functionally harmless (the advance still adds to balance before spendings), but semantically inconsistent with "credits before debits."

Multiple strategy transforms stack multiplicatively. If two strategies divert from different source resources on the same contract completion, each reads the previously-transformed value. Strategy A diverts 10% of funds, then Strategy B diverts 20% of the remaining funds = 28% total, not 30%. This may differ from KSP's actual multi-strategy behavior. Needs in-game verification.

Recording-associated spending survives reconciliation when recording is pruned. If recording A has a vessel build cost (`FundsSpending` with `RecordingId=A`), and recording A is removed via KSP load, reconciliation prunes earnings by recording ID but spending pruning is by UT only. The spending persists even though its associated recording is gone. Acceptable since KSP load is authoritative and the loaded save's state accounts for the spending.

Empty ledger + mid-career = no replay of past events. When Parsek is installed mid-career, only the funds seed is captured. All prior tech unlocks, facility upgrades, contract completions, and kerbal assignments are NOT in the ledger. The recalculation walk only knows about events captured by Parsek from this point forward. This is by design (the ledger starts fresh) but means the game actions list will be incomplete for the pre-Parsek period.

15.3 Performance Notes

The recalculation walk is O(n log n) sort + O(n × m) dispatch (n=actions, m=9 modules). For 500 actions, this completes in well under 1ms. The most expensive per-action operation is the reputation curve evaluation (integer-step loop per nominal rep value), but even with 50 rep actions this is trivially fast. LINQ `SortActions` allocates a new list each call — minor GC pressure, could be optimized to sort in-place if needed.

No batching for rapid commits: 10 quick commits = 10 full recalculations. Acceptable for typical gameplay (commits are rare events, not per-frame).